Department of Computer Science

Software Engineering

Final Project (30%)

Phase 2: Software Design Specification Document (10%)

Rihla

Desert and City Experience Booking Platform for Dubai

Software Design Specification (SDS)

Team members and parts

Salem Al Tamimi: Sections 1 and 2

Hamzah and Hamad: Section 3

Mir Umer: Section 4

Omar Arif: Section 5

Oumar Ibrahim: Section 6

Built with Python 3.12: a Tkinter desktop client, a Flask REST server and an SQLite database

Table of Contents

1.0 Introduction 4
1.1 Goals and objectives 4
1.2 Statement of scope 5
1.3 Software context 7
1.4 Major constraints 8
2.0 Data design 10
2.1 Internal software data structure 10
2.2 Global data structure 11
2.3 Temporary data structure 11
2.4 Database description 12
3.0 Architectural and component-level design 25
3.1 System Structure 27
3.2 Description for Components 32
3.2 Description for Component “Accounts and Sign-in” 33
3.2 Description for Component “Events and Venues” 40
3.2 Description for Component “Booking and Shopping List” 46
3.2 Description for Component “Pricing, Payment and Receipts” 53
3.2 Description for Component “Cancellations and Refunds” 58
3.2 Description for Component “Venue Operations” 64
3.2 Description for Component “Fleet and Dispatch” 70
3.2 Description for Component “Administration and Reports” 76
3.2 Description for Component “Reviews and Support Tickets” 82
3.3 Dynamic Behavior for Components 86
4.0 User interface design 97
4.1 Description of the user interface 97

4.2 Interface design rules 108
4.3 Components available 109
4.4 UIDS description 109
5.0 Restrictions, limitations, and constraints 110
6.0 Testing Issues 112
6.1 Classes of tests 113
6.2 Expected software response 114
6.3 Performance bounds 118
6.4 Identification of critical components 119
7.0 Work distribution 120
Appendix A: Server endpoint reference 120

Section 1

1.0 Introduction

Rihla is a booking platform for experiences in Dubai, such as evening desert safaris, dinners with a view, yacht cruises and overnight camps. In Phase 1 we wrote down what the system should do. This document explains how we actually built it: how the data is stored, how the program is split into parts, what the screens look like and how we test it.

One thing changed a lot since Phase 1. Our first plan was to write a custom module for Odoo on top of PostgreSQL. When we started coding, we decided that a full business framework was more than a team of our size could learn and finish in one semester, so we moved to plain Python. Rihla is now a Tkinter desktop app that talks to a small Flask server, and the server keeps everything in one SQLite file. Everything in this document describes that final version.

Looking back, our way of working was closer to the evolutionary, or iterative and incremental, model than to the waterfall model. We did not design everything first and code it once. We built a small working version, tested it, and then added one feature at a time: VAT and currencies, cancellations and refunds, the admin role, the SSN rule, staff venues. Each step ended with the full test suite passing. The known risk of this model is problems with the architecture, because not all requirements are known at the start. We felt that risk ourselves. For example, the users table was designed before the admin and staff venue features existed, which is why their columns were added later by init_db.

1.1 Goals and objectives

Small tour companies in Dubai still take many bookings over the phone and on WhatsApp. When a tourist, a hotel desk and a travel agency all want the last seats of the same safari, somebody usually gets a seat that does not exist. The main goal of Rihla is to stop this by keeping every booking in one place and checking the seats on the server, at the moment of booking.

The second goal is to give each person in the process a screen made for their job. A tourist does not need to see the vehicle fleet, and the driver team does not need the payment window. We ended up with six roles: Customer, Vendor, Staff, Partner (travel agency), Dispatcher and Admin.

Our objectives are:

- **No double booking:** seats are counted again inside a database transaction when the booking is saved.
- **Honest prices:** the 5% UAE VAT is always shown as its own line, and the user can see prices in AED, USD, EUR or GBP.
- **Simple tickets:** every booking gets a 16-digit code that venue staff can check in once.
- **A clear refund policy:** the same cancellation rules for everybody, with a record of who cancelled and why.
- **Safety:** an SSN for every account that is not a customer, a Report button for everyone, and bans handled by the admin.

1.2 Statement of scope

Rihla follows a booking from start to end. A vendor publishes an event and the dispatcher gives it vehicles. A customer or a partner books it and pays. On the day, staff at the venue check the ticket in, and afterwards the customer can leave a review. Cancellations, refunds, support questions and reports also live inside the system.

The main inputs are account details, event details (title, category, price in AED, venue, date or weekly days, time, capacity and a picture), booking requests, payment details, ticket codes, vehicle data, support messages and reports. The main outputs are the event list with live seats, the booking confirmation with its ticket code, a VAT breakdown, a PDF receipt, check-in results, the day's schedule for a venue and the refund amount when a booking is cancelled.

We ranked the functions from our point of view as developers. Essential means it is finished and tested. Desirable means we wanted it and got most of it done. Future means it is not in the code yet.

Function	Priority	Where it stands
Sign up, log in and password reset for six roles	Essential	Done. Passwords are salted PBKDF2 hashes.
16-digit SSN for non-customer accounts	Essential	Done. Only a hash and the last 4 digits are kept.
Events, venues and categories for vendors	Essential	Done, with pictures and weekly repeats.
Explore, search and filter tours	Essential	Done (category, maximum price, search box).
Seat check for each date	Essential	Done. Uses vehicle seats or the event capacity.
Booking with 5% VAT and a currency choice	Essential	Done. Totals are saved in AED with VAT.
Payment by Visa, Mastercard or bank transfer	Essential	Demo only. Checked against fixed details.
Booking Details window with PDF download	Essential	Done. The PDF is written by our own code.
Ticket check-in by venue staff	Essential	Done. Staff only see their own venue.
Cancellation and refund rules	Essential	Done. 100%, 50% or not allowed, with a record.
Partner shopping list paid in one go	Essential	Done. All items or none are booked.
Vehicle register and event dispatch	Essential	Done. Vehicle seats become the capacity.
Admin: accounts, bans, reports, all bookings	Essential	Done.
Reviews and average ratings	Desirable	Done, 0 to 5 stars in half steps.
Support tickets answered at the venue	Desirable	Done. Staff can mark them resolved.
Google Maps link for each venue	Desirable	Done.
Real email and SMS codes	Desirable	Not done. The code is always 000000.
Real payment gateway and real refunds	Future	Not done.
QR tickets, driver app, live vehicle map	Future	Not done.
Arabic and other languages	Future	Not done. English only.

Table 1: Ranking of the major functions

User stories

We also wrote the requirements as user stories in the form 'As a <role>, I need <capability>, so that <business value>'. This form helped us check that each role really gets something from the system. A few examples:

Role	User story
Customer	As a customer, I need to see how many seats are left on a date, so that I do not try to book a full event.
Partner	As a travel agency, I need to pay for several events in one go, so that my client group is either fully booked or not booked at all.
Vendor	As a vendor, I need to cancel a booking with a reason, so that the customer knows why and gets a full refund.

Role	User story
Staff	As a staff member, I need to check a ticket in only once and only at my venue, so that nobody enters twice or at the wrong place.
Dispatcher	As a dispatcher, I need the event's seats to follow the vehicles I assign, so that we never sell more seats than we can drive.
Admin	As an admin, I need to ban an account from a report, so that a bad user is stopped at once.

Table 2: Example user stories

Non-functional requirements (FURPS+)

We grouped the non-functional requirements as FURPS+: Functionality (here mainly security), Usability, Reliability, Performance and Supportability, plus extra constraints. This is how Rihla meets each of them:

Quality	How the design meets it
Functionality (security)	Salted PBKDF2 hashes for passwords and SSNs, the _blocked guard for banned and unknown accounts, staff limited to their own venue.
Usability	One frame for every role, quick demo accounts, clear messages from the server, a calendar that only allows valid dates.
Reliability	Seat checks and the cart checkout run inside one database transaction, a ticket can only be used once.
Performance	Short transactions, a 10 second client timeout, pictures shrunk to 1400 pixels.
Supportability	init_db upgrades older databases, dummy_data.ps1 resets the demo data, 70 automated tests.
+ (constraints)	Python 3.12 on Windows, money in AED with 5% VAT, no real payments.

Table 3: FURPS+ quality requirements

1.3 Software context

Tourism is one of the largest industries in Dubai, and most visitors book at least one desert or sea trip. The companies that run these trips are often small and sell through many channels at once: their own customers, hotel desks and travel agencies. Rihla puts all of these on one system. Customers book for themselves. Partners book groups for their clients and are allowed more than 10 guests per booking. Vendors see every booking for their events, dispatchers decide how many seats an event has by giving it vehicles, and staff at the venue see who is coming today.

On the technical side, Rihla is a client and server program (Figure 1). The client in the src folder is a Tkinter app that the user starts with start_client.py. It never opens the database itself. Every request goes through api_client.py as HTTP with JSON to the Flask server in the server folder, which listens on 127.0.0.1 port 5000. Because

only the server touches users.db, all the rules about seats, VAT, refunds, bans and venues are written once, in one place.

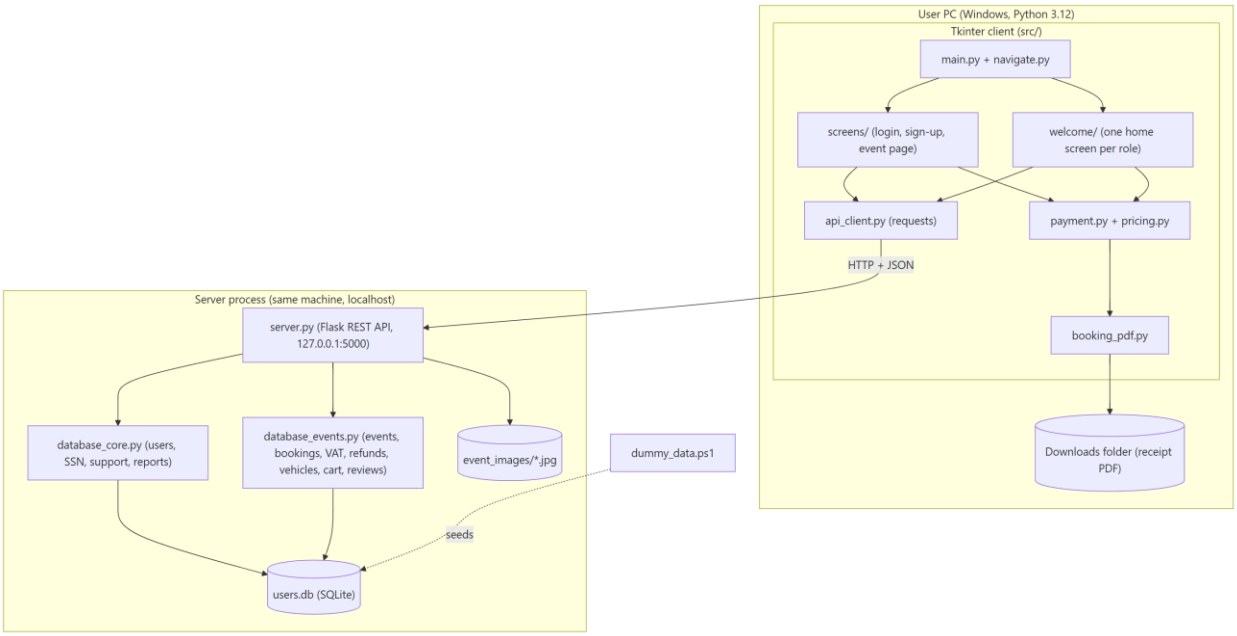


Figure 1: Component and deployment view of Rihla

1.4 Major constraints

A few things shaped the design more than others.

- **Time and team size:** we had one semester, so we chose tools the whole team already knew (Python, Tkinter, Flask and SQLite) instead of learning a big framework.
- **UAE VAT:** VAT is 5% and has to appear as a separate line. All money is kept in AED. Other currencies are only for showing prices.
- **No real money:** a course project cannot process real cards, so the payment step is a demo that compares the form with fixed demo details.
- **Privacy:** an SSN is sensitive, so the server never stores the number itself.
- **Windows:** the team works on Windows 10 and 11, so that is what we tested on. A few details, like finding the Downloads folder, use the Windows registry.
- **Easy to demo:** the server creates six demo accounts when it starts, and dummy_data.ps1 fills the database with events, bookings and reviews whose dates are relative to today.

Poor configuration management and too little testing are among the main reasons why software projects fail, so we treated both as constraints from the start. The whole project lives in one Git repository, which is our software configuration management tool. Every commit is a version we can compare with or go back to, and the commit messages say what changed and why. We worked on a single main branch instead of topic branches, which is the simple centralized workflow. It is simple and fits a small team, but every change has to start from the latest version of shared files such as the database modules, or work gets lost in a merge.

Section 2

2.0 Data design

This section describes the data inside Rihla: what moves between the parts of the program, what is shared by all of them, which files are only kept for a while, and how the database is built.

2.1 Internal software data structure

Inside Rihla, data travels as Python dictionaries. Between the client and the server the same dictionaries are sent as JSON. Every answer from the server has a success flag, and when something goes wrong it also has a message written for the user, which the client shows as it is. This kept our error handling simple, because the screens never need to guess what went wrong.

The most important structure is the event. It holds the title, category, price, description, venue (with its name and map link), the date for a one-time event or the weekdays and time for a repeating one, and the capacity. Almost every screen starts from a list of events.

Close to it is the capacity info, which the event page asks for each time the user picks a date. It says how many seats the event has in total, where that number comes from (the assigned vehicles or the event itself), how many are already booked on that date and how many remain. The capacity bar on the screen is drawn straight from these numbers.

The booking is the structure that most roles share. Besides the event, date, guests, total price and ticket code, it carries a status and the cancellation details. The status is worked out when the booking is read, not stored as such. The database only keeps 'Confirmed' or 'Cancelled' for the booking and 'Valid', 'Used' or 'Cancelled' for its ticket. The function _booking_dict combines the two with today's date and shows the user 'Confirmed', 'Checked in', 'Expired' or 'Cancelled'. It also adds can_cancel and refund_if_cancelled, so the screen can show the refund before the user decides.

Smaller structures connect single steps. The payment result (txn_id, method, currency) goes from the payment window to the Booking Details window. The receipt is a list of lines, each with its text, font size and bold flag, and the same list is used for the window and for the PDF. A refund is returned as percent, base, VAT and total. A cart item, a vehicle assignment and a user row for the admin table follow the same idea.

2.2 Global data structure

The global store is one SQLite database, server/users.db. No screen opens it directly. The client asks the server, and the server uses database_core.py (accounts, SSN, staff venues, support tickets and reports) or database_events.py (everything about events and bookings). Event pictures are kept next to it as files in server/event_images, named by the event id.

A few modules hold values that large parts of the program depend on:

- **config.py:** the six roles, the demo accounts, the roles allowed at sign-up (all except Admin) and the minimum password length of 8. There is one copy for the client and one for the server.
- **pricing.py:** VAT_RATE = 0.05 and the fixed display rates for 1 AED (USD 0.2723, EUR 0.2510, GBP 0.2150).
- **database_events.py:** the refund rules (48 and 24 hours) and the 300 character limit for a cancel reason.
- **database_core.py:** default capacity 20, maximum capacity 5000, 10 guests per customer booking, 3 open support tickets per customer and SSN length 16.
- **theme.py:** colours and the Segoe UI font, so every screen matches.

2.3 Temporary data structure

Some data only lives for a short time:

- **Partner cart:** the partner_cart table holds the events a partner has picked but not paid for. The rows are deleted after a successful checkout.

- **Picture cache:** the client keeps downloaded event pictures in memory and drops one when the vendor changes it, so lists do not download the same picture again.
- **Uploaded picture:** before upload the client shrinks the picture to at most 1400 pixels and sends it as base64 inside the JSON. The server checks that it is a JPEG under 4 MB before writing the file.
- **Receipt PDF:** saved in the user's Downloads folder as Rihla_Booking_<transaction id>.pdf. If the file exists, a number is added to the name, so old receipts are never overwritten.
- **Test databases:** the unit tests point the database path to a temporary file, so they never touch the real data.
- **Seed record:** dummy_data.ps1 writes down every row it adds in a demo_seed table, so the next run can remove exactly those rows and nothing else.

2.4 Database description

The database is created by init_db in database_core.py. It runs every time the server starts. It creates the tables that are missing, adds new columns to databases made by older versions of our code (for example the cancellation columns), and fills in the default data: five vehicle types, two venues, three categories, two demo events, seven vehicles, a demo ticket and the six demo accounts. The reviews table is the exception. It is created the first time someone reads or writes a review.

Figure 2 shows the tables and how they relate. We should be honest about one design choice here. The users table has no number id, so most tables point to a user by email address (for example customer_email or vendor_email). This was quick to build, but it means an email cannot be changed later without touching many tables. Section 5 comes back to this.

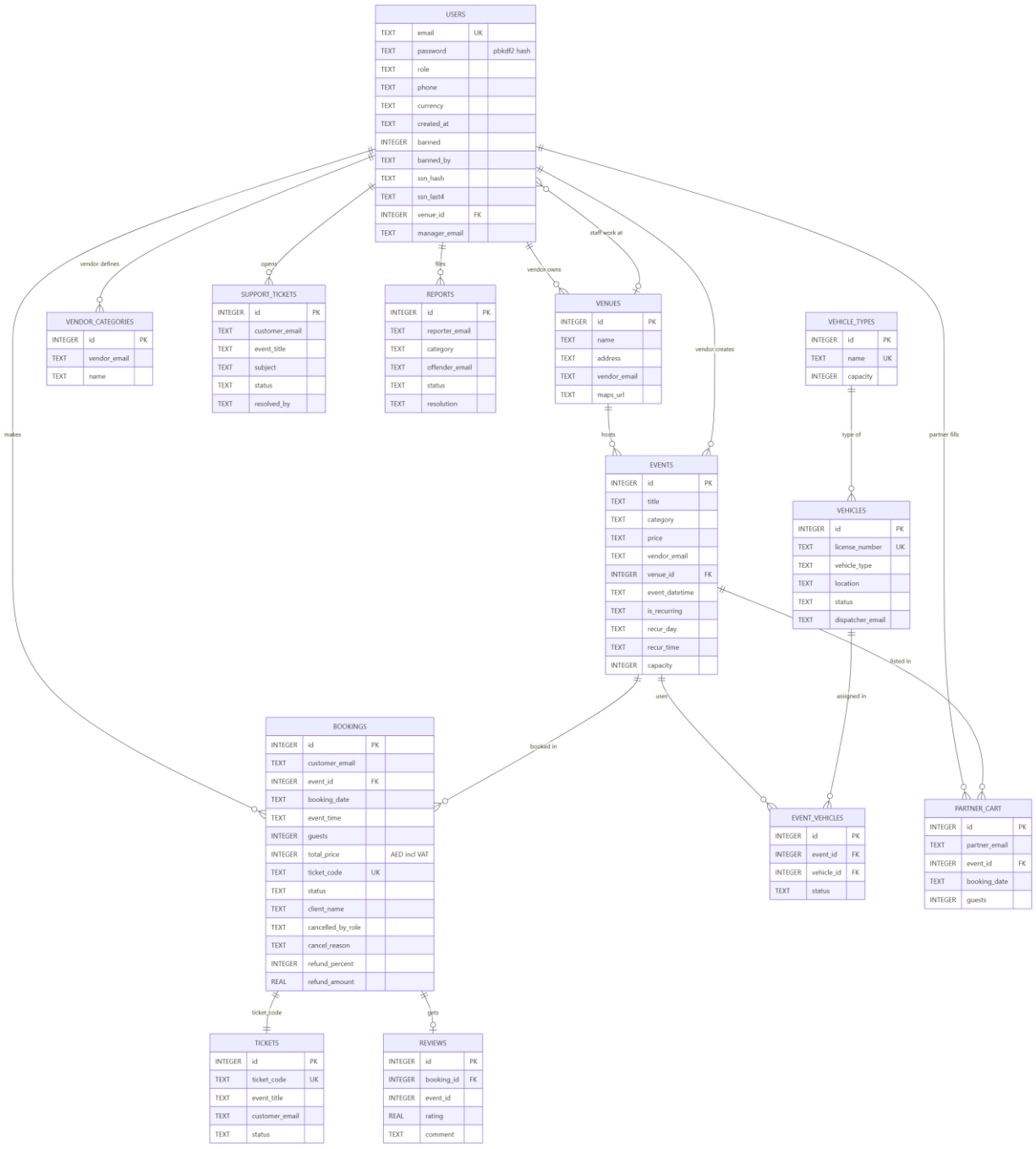


Figure 2: Entity relationship diagram of server/users.db

EER diagram

The ER diagram above shows the tables as they are stored. To show the meaning behind them, we also drew an Enhanced ER (EER) diagram in Chen notation. Rectangles are entities, diamonds are relationships and ovals are attributes. A key attribute is underlined, a double oval is multi-valued, and a dashed oval is derived,

which means the code works it out instead of storing it. A double line means total participation, so every row on that side must take part in the relationship. The numbers 1, N and M on the lines are the cardinalities.

The most important part of the EER is the specialization of USER. In the code every account has exactly one role, stored in the role column, so we drew a disjoint (d) and total specialization. We added one level in between, SSN_HOLDER, because every role except Customer must give a 16-digit SSN, and only those accounts have ssn_hash and ssn_last4. STAFF is the only role with its own extra attribute (manager_email) and its own relationship (WORKS_AT a venue). The other roles take part in relationships that only they are allowed to use: VENDOR owns venues and creates events, DISPATCHER registers vehicles, PARTNER fills the shopping list and ADMIN bans users and handles reports.

A complete EER with every attribute does not fit on one page in a readable size, so we split it the same way a map book does. Figure 3 is the overview with all entities and relationships and no attributes. Figures 4 to 8 each show one area with all of its attributes. An entity drawn in grey is only there to show a link, and its attributes are in another part.

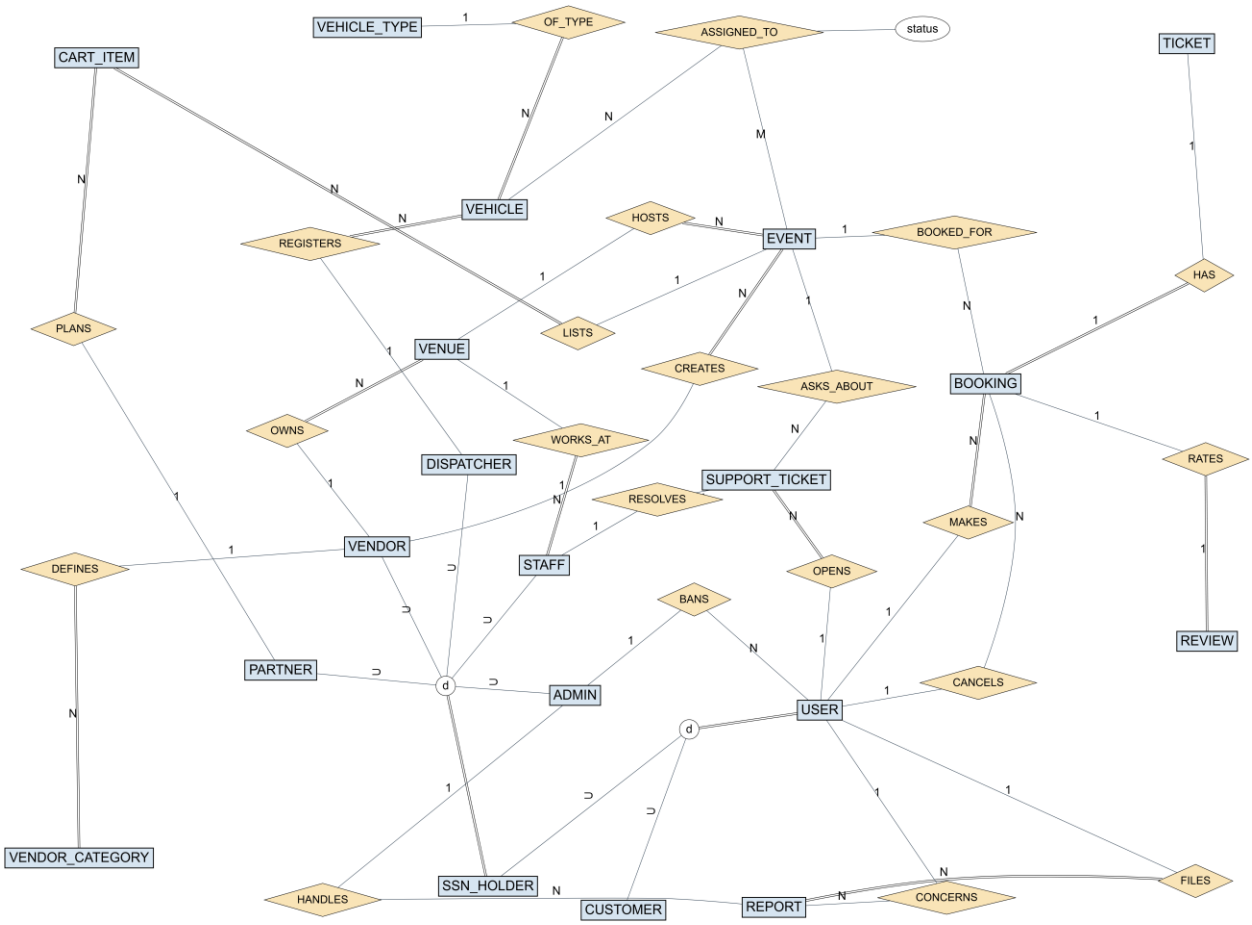


Figure 3: EER overview: all entities, relationships, cardinalities and the role specialization

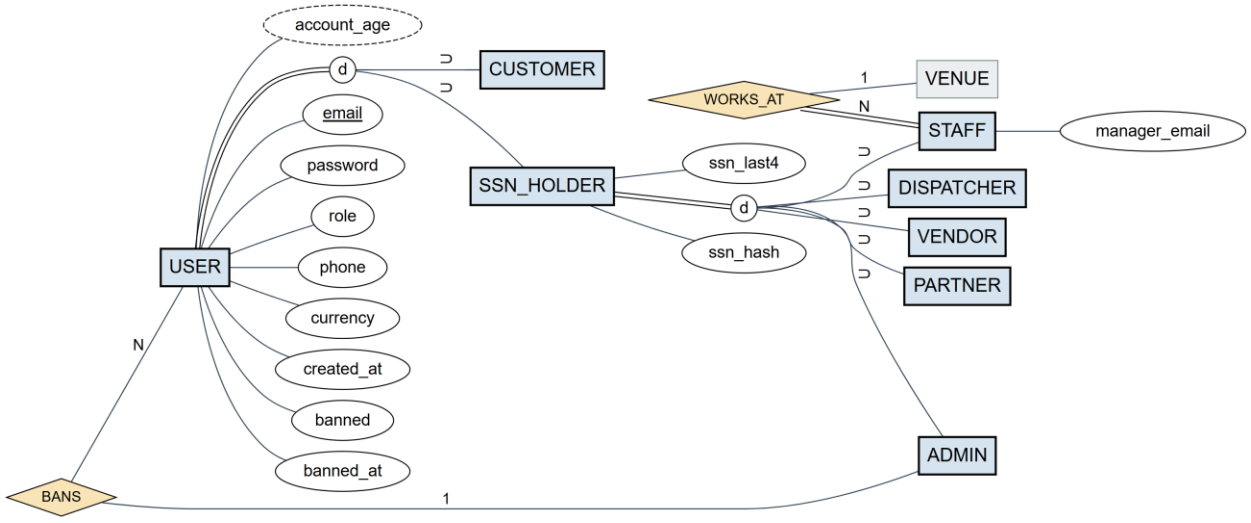


Figure 4: EER part 1: users and roles, with all attributes

Figure 4: account_age is derived from created_at on the admin page. BANS is a relationship between an admin and any user, and banned_at belongs to the user because it describes the account.

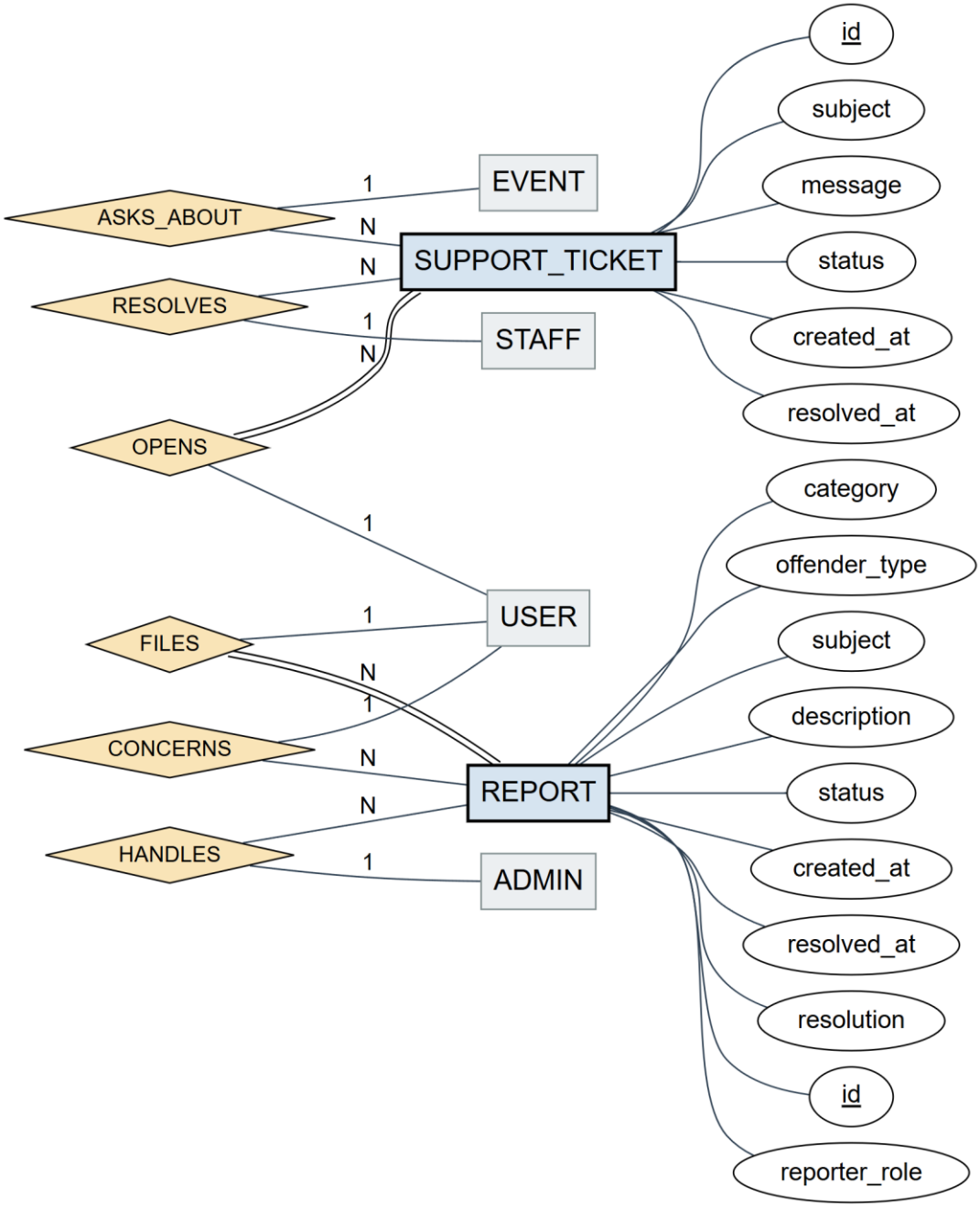


Figure 5: EER part 2: support tickets and reports, with all attributes

Figure 5: ASKS_ABOUT is partial, because old support tickets from before customers had to pick an event name no event at all. CONCERNS is partial too, because the offender's email in a report is optional.

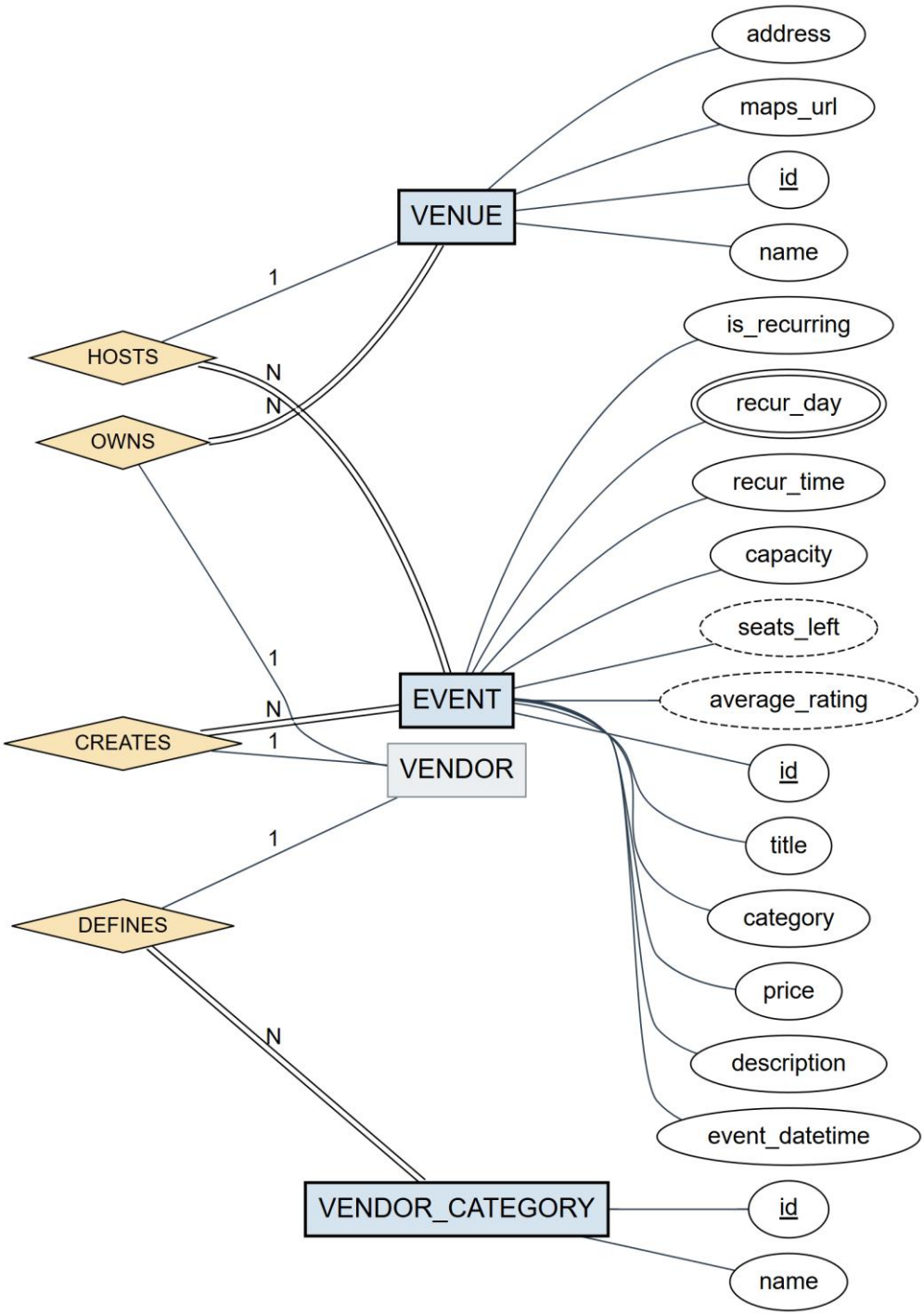


Figure 6: EER part 3: venues and events, with all attributes

Figure 6: recur_day is multi-valued because a repeating event can run on several weekdays, for example Tuesday and Thursday. seats_left and average_rating are derived. Every event is hosted at a venue and created by a vendor, so EVENT has total participation in both.

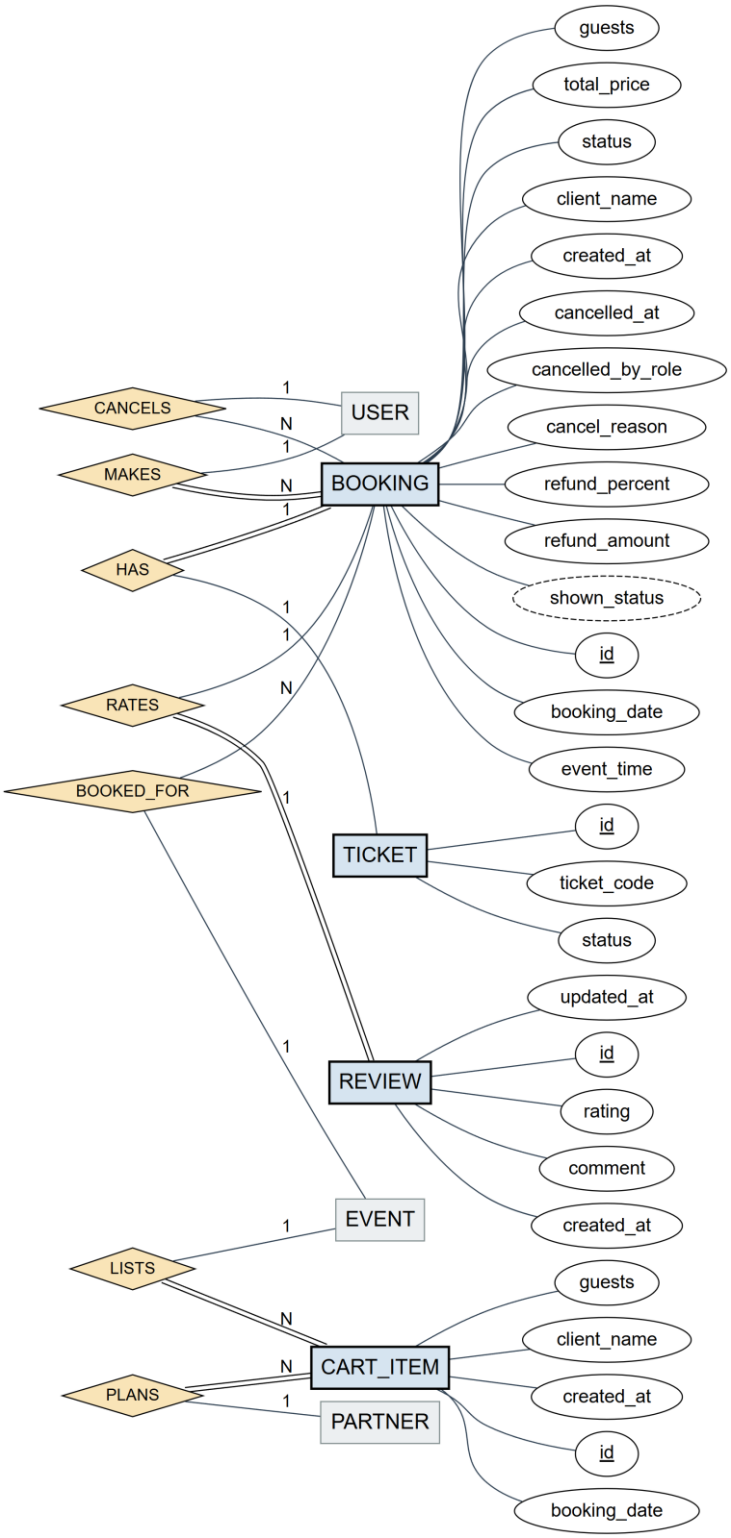


Figure 7: EER part 4: bookings, tickets, reviews and the partner cart, with all attributes

Figure 7: every booking has exactly one ticket, but the demo ticket shows that a ticket can exist without a booking, so only the booking side is total. A review

belongs to exactly one booking. BOOKED_FOR is partial for bookings because they stay in the database after their event is deleted. shown_status (Confirmed, Checked in, Expired or Cancelled) is derived from the booking, its ticket and today's date.

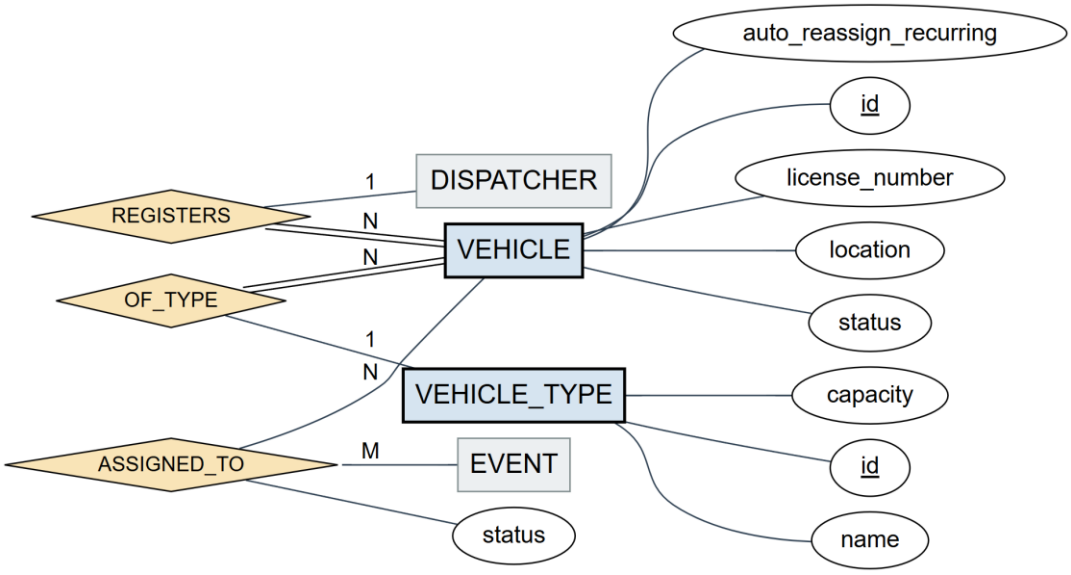


Figure 8: EER part 5: vehicles and vehicle types, with all attributes

Figure 8: ASSIGNED_TO is the only many-to-many relationship. It is stored in the event_vehicles table, and its status (Assigned or Concluded) is an attribute of the relationship, not of the event or the vehicle.

The tables below give enough detail to create the database again.

Table Name: users

Attribute	Data Type	Constraint
email	TEXT	UNIQUE, compared without letter case
password	TEXT	pbkdf2$200000$salt$hash
role	TEXT	NOT NULL, default 'Customer'
phone	TEXT	UAE mobile number
currency	TEXT	AED, USD, EUR or GBP (default AED)
created_at	TEXT	sign-up time
banned / banned_at / banned_by	INTEGER / TEXT / TEXT	banned is 0 or 1
ssn_hash / ssn_last4	TEXT	salted hash and last 4 digits
venue_id	INTEGER	references venues(id), staff only
manager_email	TEXT	staff only, just stored

Table 4: The users table holds one row per account. The SSN columns are only filled for non-customer roles, and the venue columns only for staff.

Table Name: venues

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name / address	TEXT	name NOT NULL
vendor_email	TEXT	owner
maps_url	TEXT	Google Maps https link or ''

Table 5: The venues table holds the places where events happen and the vendor that owns them.

Table Name: events

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
title	TEXT	NOT NULL
category / price / description	TEXT	
vendor_email	TEXT	owner
venue_id	INTEGER	FOREIGN KEY venues(id)
event_datetime	TEXT	'YYYY-MM-DD HH:MM' for one-time events
is_recurring / recur_day / recur_time	TEXT	'Recurring' or 'One-Time', weekdays, time
capacity	INTEGER	NOT NULL default 20, from 1 to 5000

Table 6: The events table holds the experiences that can be booked. Price is text such as '240 AED'.

Table Name: bookings

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
customer_email	TEXT	NOT NULL, customer or partner
event_id	INTEGER	FOREIGN KEY events(id)
event_title / venue_name	TEXT	copies kept for history
booking_date / event_time	TEXT	booking_date NOT NULL
guests	INTEGER	NOT NULL default 1
total_price	INTEGER	NOT NULL, AED with VAT
ticket_code	TEXT	UNIQUE NOT NULL, 16 digits
status	TEXT	'Confirmed' or 'Cancelled'
client_name	TEXT	the partner's client group
created_at	TEXT	default CURRENT_TIMESTAMP
cancelled_at / cancelled_by / cancelled_by_role / cancel_reason	TEXT	cancellation record
refund_percent / refund_amount	INTEGER / REAL	100 or 50, amount in AED

Table 7: The bookings table holds every paid booking and, when it is cancelled, who did it, why and how much was refunded.

Table Name: tickets

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
ticket_code	TEXT	UNIQUE
event_title / customer_email	TEXT	
status	TEXT	'Valid', 'Used' or 'Cancelled'

Table 8: The tickets table holds the entry tickets that staff check in.

Table Name: reviews

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
booking_id	INTEGER	NOT NULL UNIQUE, FOREIGN KEY bookings(id)
event_id / customer_email	INTEGER / TEXT	index on event_id
rating	REAL	0 to 5 in steps of 0.5
comment	TEXT	up to 500 characters
created_at / updated_at	TEXT	default CURRENT_TIMESTAMP

Table 9: The reviews table holds one rating per booking. A second review of the same booking replaces the first.

Table Name: support_tickets

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
customer_email / subject / message	TEXT	NOT NULL
event_title	TEXT	the event it is about
status	TEXT	'Open' or 'Resolved'
created_at / resolved_by / resolved_at	TEXT	

Table 10: The support_tickets table holds questions from customers about an event.

Table Name: reports

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
reporter_email / reporter_role	TEXT	NOT NULL
category / offender_type / offender_email	TEXT	what happened and who it is about
subject / description	TEXT	NOT NULL
status / resolution	TEXT	'Open' or 'Resolved'

Attribute	Data Type	Constraint
created_at / resolved_by / resolved_at	TEXT	

Table 11: The reports table holds the problems that users report to the admin.

Table Name: vehicle_types

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name	TEXT	UNIQUE NOT NULL
capacity	INTEGER	NOT NULL

Table 12: The vehicle_types table says how many seats each kind of vehicle has (desert buggy 2, 4x4 Land Cruiser 6, Tesla sedan 4, mini-yacht 20, RTA bus 50).

Table Name: vehicles

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
license_number	TEXT	UNIQUE NOT NULL
vehicle_type / location	TEXT	NOT NULL
status	TEXT	'Available', 'Assigned' or 'Maintenance'
dispatcher_email	TEXT	owner
auto_reassign_recurring	INTEGER	0 or 1

Table 13: The vehicles table holds the fleet of each dispatcher.

Table Name: event_vehicles

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
event_id / vehicle_id	INTEGER	FOREIGN KEYs
status	TEXT	'Assigned' or 'Concluded'

Table 14: The event_vehicles table links vehicles to the events they serve.

Table Name: partner_cart

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
partner_email	TEXT	NOT NULL, indexed
event_id	INTEGER	NOT NULL, FOREIGN KEY events(id)
booking_date / guests / client_name	TEXT / INTEGER / TEXT	
created_at	TEXT	default CURRENT_TIMESTAMP

Table 15: The partner_cart table is the partner's shopping list.

Table Name: vendor_categories

Attribute	Data Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
vendor_email / name	TEXT	NOT NULL, no repeated name per vendor

Table 16: The vendor_categories table holds the categories a vendor can pick for events.

We also added four indexes for the queries that run most often: bookings by customer, bookings by event and date, the cart by partner and reviews by event.

Section 3

3.0 Architectural and component-level design

Suitable architecture style: layered architecture, split over a client and a server (three tiers).

Reasoning: We have six roles that all work on the same seats and the same bookings. If each screen had its own copy of the rules, sooner or later two screens would disagree, for example about how many seats are left. Putting the rules in one server layer, under all the screens, avoids that. Layers also let us work in parallel. One of us could change a screen while another changed a SQL query, and as long as the JSON stayed the same nothing broke. The cost is an extra step for every action, because the client has to call the server even for small things. On one machine this delay is too small to notice.

Our layered architecture has three layers:

Presentation layer (Layer 2): the Tkinter screens in the src folder. navigate.py switches between the login, sign-up and password screens, and routing.py opens the home screen that matches the user's role. Every home screen extends BaseWelcomeScreen, which draws the dark side menu, the top bar with the clock, and the Report, Settings and Log Out buttons.

Application layer (Layer 1): server.py, a Flask REST API. Each endpoint reads the JSON body, checks the caller with a small guard called _blocked (unknown or banned accounts and wrong roles get HTTP 403), calls one data function and returns JSON.

Data layer (Layer 0): database_core.py and database_events.py, which hold all SQL and the business rules: password hashing, SSN checks, seats, VAT, ticket codes, refunds, venue scope and bans.

A good example of why this matters is the seat check. The event page checks the remaining seats before it opens the payment window, but that is only for the user's comfort. The real check happens again in the data layer inside a BEGIN IMMEDIATE transaction, when the booking is saved. That is how we prevent double booking. In

Phase 1 we had planned a 15-minute seat lock instead, but the transaction turned out to be simpler and safer.

We see the architecture as the set of high-level decisions that are made early and are hard to change later. An architectural style is made of components, connectors, interfaces and a configuration. In Rihla the components are the screens, the Flask server and the two data modules. The connectors are HTTP requests with JSON between client and server, and plain Python calls and SQL on the server. The interfaces are the server endpoints listed in Appendix A, and the configuration is the picture in Figure 1.

Rihla mixes three well-known architectural styles. It is client/server, because the Tkinter client is the triggering process and the Flask server is the reactive one that accepts or refuses each request. It is layered, because each layer only uses the one below it. And it has a central repository, because all data lives in one database that only the server opens. Our API also borrows from REST: events, bookings and venues are resources with their own paths, and GET, POST, PUT and DELETE are used on them. We are not fully RESTful, though. Actions such as /cancel or /checkout are verbs in the path, which a strict REST design would avoid.

Design decisions and alternatives

While building Rihla we had to choose between several options more than once. The table below lists the main choices, what we picked and why. Some of them have a price, and we come back to those in section 5.

Decision	What we chose	Alternative we considered	Why
Platform	Plain Python: Tkinter client and Flask server	Odoo module with PostgreSQL (Phase 1 plan)	Smaller to learn and finish in one semester, and every team member knows Python.
Where the rules live	Only in the server's data layer	Checks in each screen	Six roles share the same seats and bookings, so the rules must exist once.
Stopping double booking	Seat check inside a BEGIN IMMEDIATE transaction	A 15-minute seat lock (Phase 1 plan)	No lock to clean up when a user leaves halfway.
Database	One SQLite file	PostgreSQL or MySQL server	Nothing to install, easy to copy and reset for the demo.

Decision	What we chose	Alternative we considered	Why
Keys for users	Email address	A number id per user	Quick to build, but an email can no longer be changed.
Sign-in state	The client remembers the email	Session tokens	Not done in time. It is the first thing we would add.
Money	Stored in AED with VAT included	Store each booking in the user's currency	One currency keeps refunds and reports simple.
Receipt PDF	Our own small PDF writer	A PDF library such as ReportLab	One less library to install for a one-page receipt.
Event capacity	Worked out from assigned vehicles, or the vendor's number	A seats column that is updated by hand	It can never be out of date.
Ticket codes	Random 16 digits, checked to be unique	QR codes	Easy to type at the gate, and no camera is needed.

Table 17: Main design decisions

3.1 System Structure

The client is organised by screen and the server by data area. The welcome package in the client has one module per role and a few shared views: ReportView, VehicleView, DispatchView, the cancellation helpers and the shared table style. The server has the API module and the two data modules.

When we split the code we tried to follow the coupling and cohesion rule: each module should have one clear purpose (high cohesion) and depend on as few other modules as possible (low coupling). pricing.py only does money, booking_pdf.py only writes PDF files, and cancellation.py only holds the cancel dialogs. On the coupling side, no screen talks to the database. They all go through api_client.py, so the server can change its SQL without any screen noticing. Cyclic dependencies between packages should also be avoided. We met this problem directly: the login screen opens the home screens and the home screens open the login screen again on Log Out. navigate.py solves it by importing each screen only inside the function that shows it, so the modules never import each other at load time.

3.1.1 Architecture diagram

The full class diagram did not fit on one page, so we split it in two. Figure 9 shows the home screens and how they inherit from BaseWelcomeScreen. Figure 10 shows

the modules behind them and the server. Python modules that are not classes are marked with <<module>>.

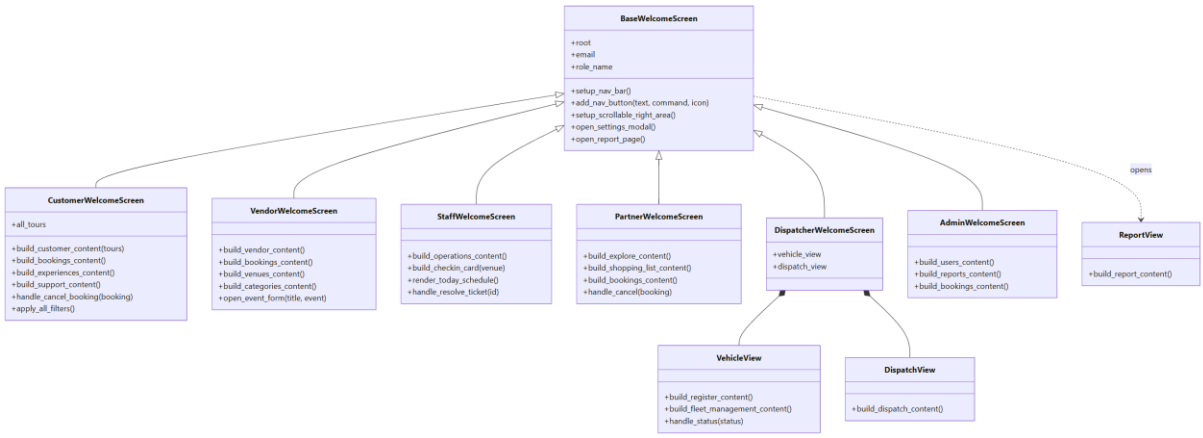


Figure 9: Class diagram, part 1: the home screens and shared views

Figure 9: every role gets its own subclass of BaseWelcomeScreen, so the menu, top bar and settings look the same everywhere and each subclass only adds its own pages. The dispatcher screen is the only one that is built from two helper views, because its fleet and dispatch pages are large.

The diagram also follows the rule of using inheritance for 'is a' relationships and composition for 'has a' relationships. A CustomerWelcomeScreen is a BaseWelcomeScreen, so it inherits. A DispatcherWelcomeScreen has a VehicleView and a DispatchView, which cannot exist without it, so they are drawn with a filled diamond (composition).

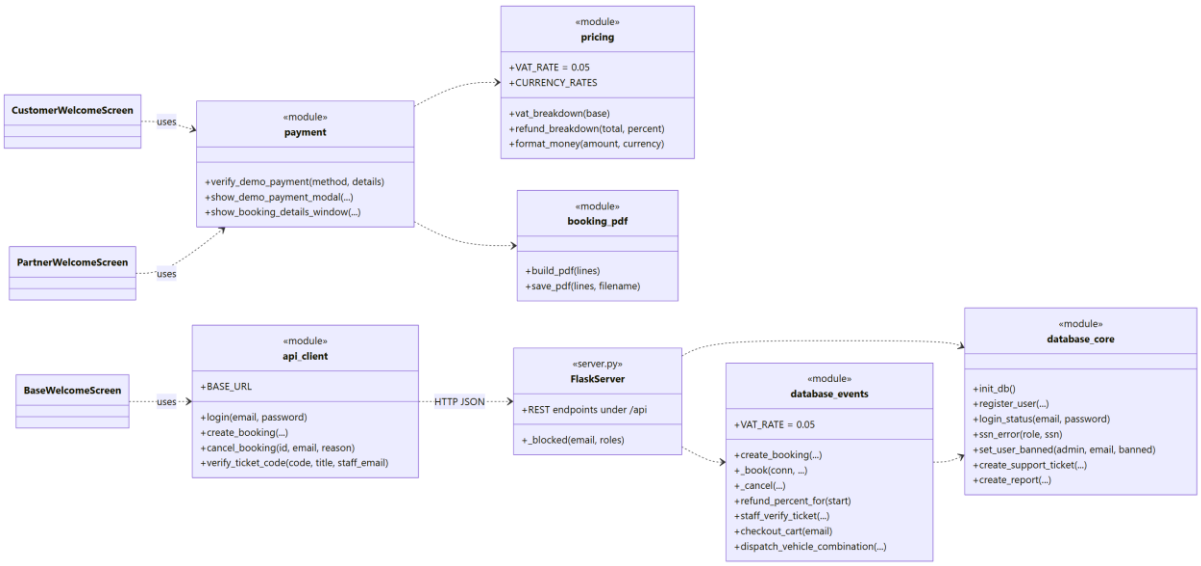


Figure 10: Class diagram, part 2: client modules, Flask server and data modules

Figure 10: the screens only know api_client. The payment window uses pricing for VAT and currency and booking_pdf for the receipt. On the server side, server.py calls the two data modules.

Figures 11 to 13 are the use case diagrams, drawn in PlantUML. We drew two roles per diagram to keep them readable. The components in section 3.2 follow these use cases.

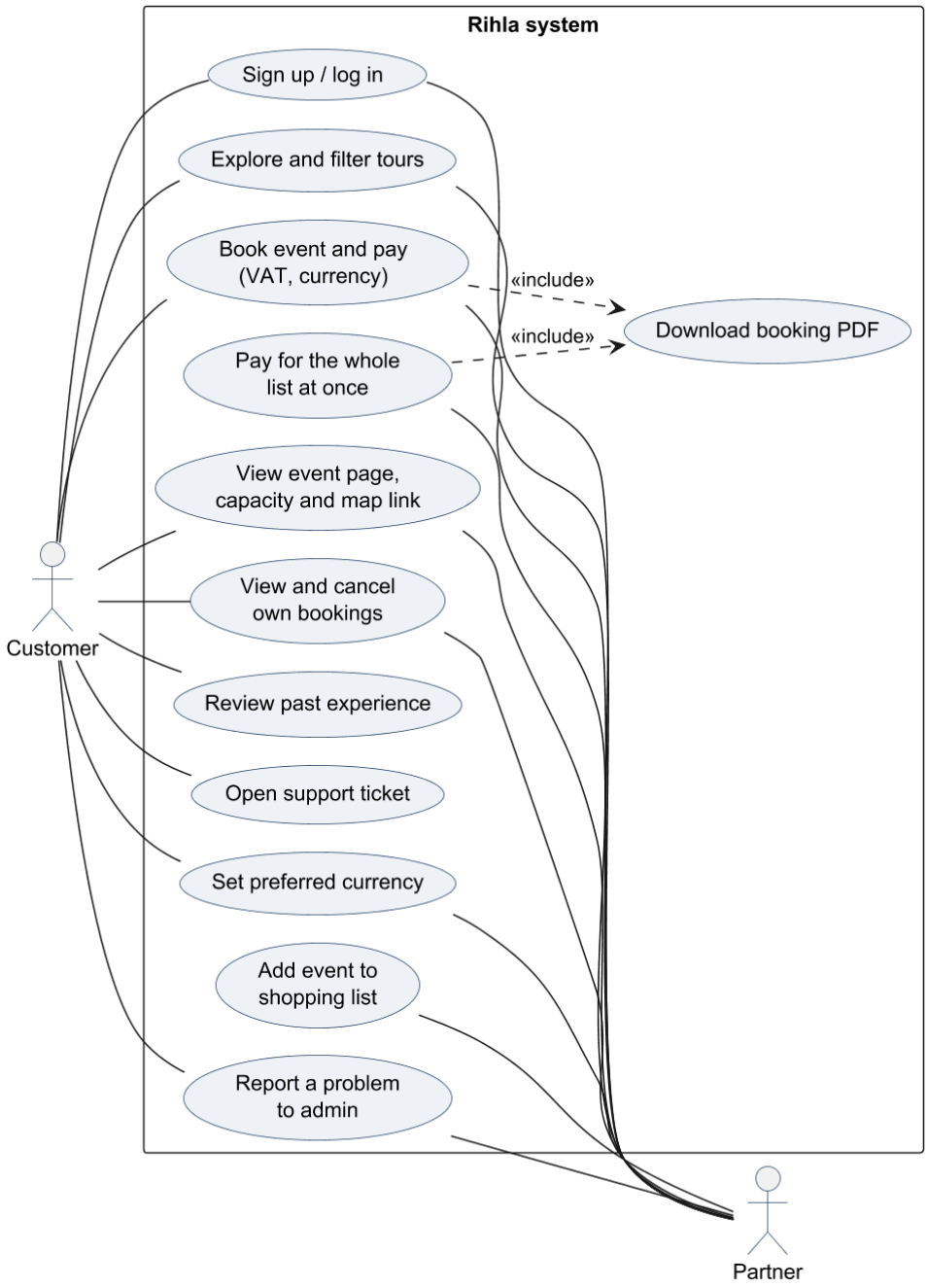


Figure 11: Use cases of the Customer and the Partner

Customers and partners share most of their use cases. The partner has two extra ones: the shopping list and paying for the whole list at once. Both paths end with the Booking Details window, where the PDF can be downloaded.

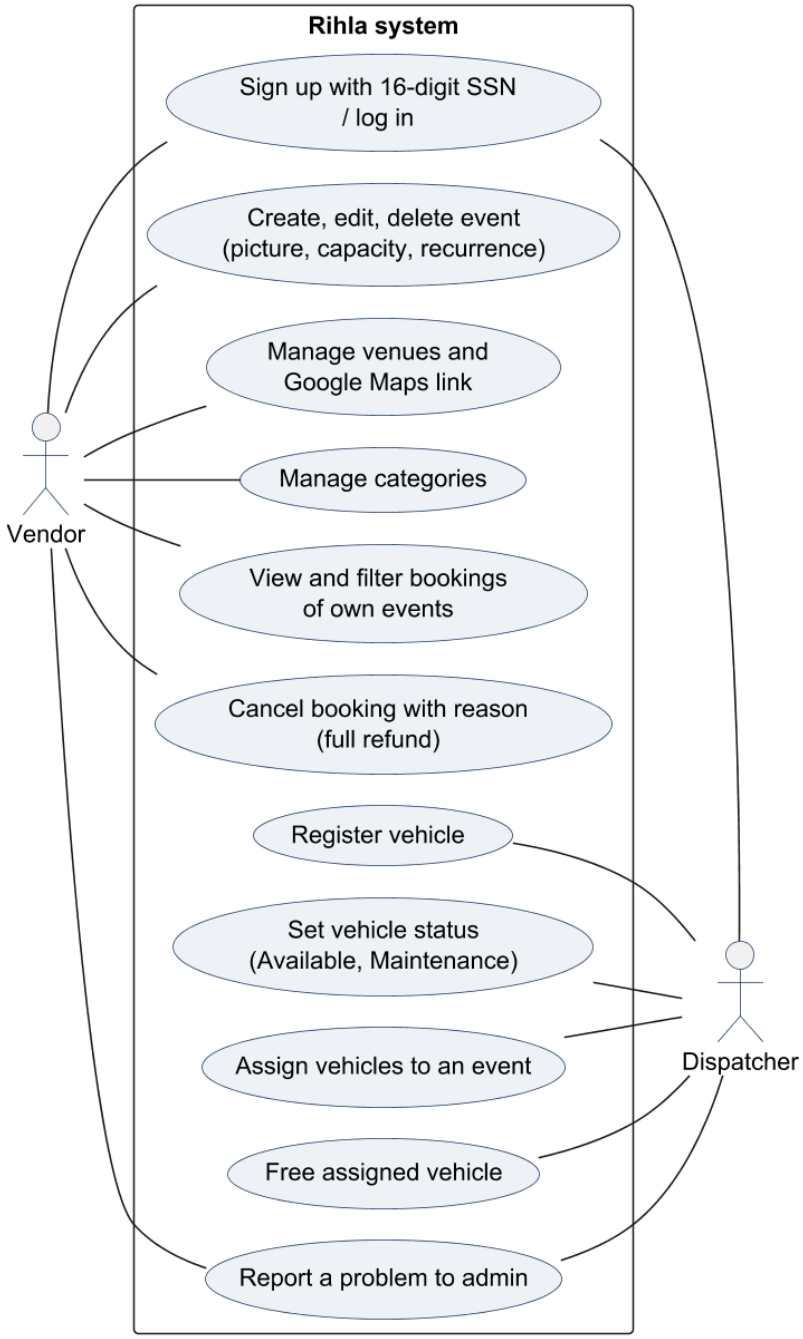


Figure 12: Use cases of the Vendor and the Dispatcher

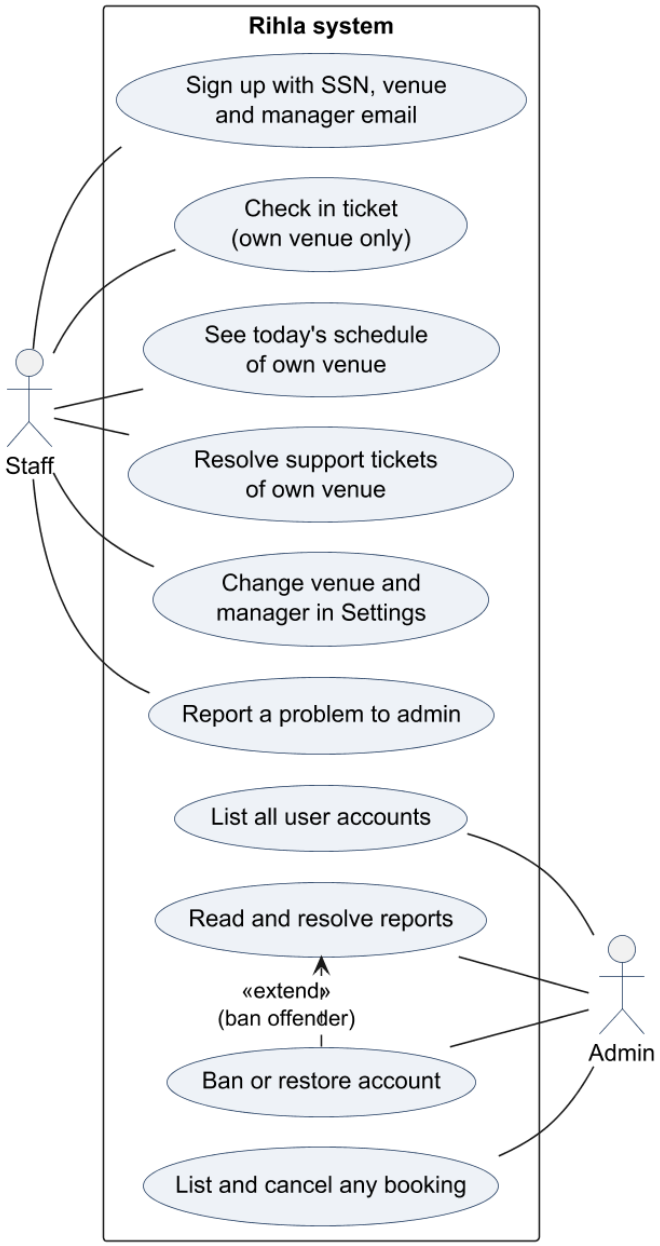


Figure 13: Use cases of the Staff and the Admin

The admin does not have a Report button, because admins are the ones who read reports. Banning an account is often done straight from a report, which is why Ban or restore account is drawn as an extension of Read and resolve reports.

3.2 Description for Components

We split Rihla into nine components. Most of them follow a role (the vendor, the staff, the dispatcher, the admin), and a few follow a piece of logic that several roles

share (booking, payment, cancelling). Each component is described with the template headings. Under 3.2.3.1 we show a small class diagram of the classes and modules that make up the component, followed by an activity diagram of its main flow. Each operation is given with its real Python name, so it can be found in the code.

For each component we also asked the question: which object should be responsible for each step? A class that checks something should stay focused on checking, and the result should be acted on by the part that needs it anyway. The screens show the refund and ask for confirmation, but the decision about the refund percent belongs only to _cancel on the server, because it needs the booking, the time and the caller's role, and because the screens could be bypassed.

3.2 Description for Component “Accounts and Sign-in”

3.2.1 Processing narrative (PSPEC) for component

This component covers everything about who the user is. It creates accounts, signs people in, resets passwords and keeps each user's preferred currency. The login screen has quick buttons for the six demo accounts, which made testing much faster.

Sign-up has an account type picker with five roles. Admin is not one of them, so nobody can make an admin account from the app. Customers sign up with email, phone and password. Every other role must also give a 16-digit SSN, and staff must pick the venue they work at and the email of its manager. We store the SSN the same way as a password: only a salted hash, plus the last 4 digits.

Passwords are hashed with PBKDF2 and 200,000 rounds. Some very early accounts still had plain text passwords, so the login quietly upgrades such a password to a hash the first time it is used.

3.2.2 Component interface description

The login screen is the first thing every user sees. From there the user can open Create Account or Forgot Password. After a correct login the client asks the server for the role and opens the matching home screen. A banned user gets a clear

message instead. Later, every role can change the preferred currency and the password from the Settings window, and staff can also change their venue there.

Endpoint	Sends	Gets back
POST /api/auth/login	email, password	stored email and role, or 401 / 403
POST /api/users/register	email, password, role, phone, ssn, venue_id, manager_email	success or the reason
GET /api/users/email_exists/<email>		exists flag
PUT /api/users/password	email, new_password	success
GET and PUT /api/users/currency	email, currency	currency code
GET and PUT /api/staff/venue	email, venue_id, manager_email	venue and manager

Table 18: Server endpoints used by “Accounts and Sign-in”

3.2.3 Component processing detail

- Start
- User opens the app and sees the login screen
- User chooses to log in, sign up or reset the password
- If signing up, user picks an account type and fills in the form (SSN for non-customers, venue and manager email for staff)
- User verifies the email and phone with the code 000000
- System checks the details and saves the account
- If logging in, system checks the email and password
- Based on the result, user enters the home screen of their role, or sees an error
- User can change the currency or password from Settings and log out at any time
- End

3.2.3.1 Design Class hierarchy for component Accounts and Sign-in

On the client this component is made of screen functions rather than classes, so in Figure 14 we show each screen file as one box. The SSN is typed into DigitBoxes, a small widget we wrote that shows one box per digit. All checks that matter are repeated on the server in database_core.

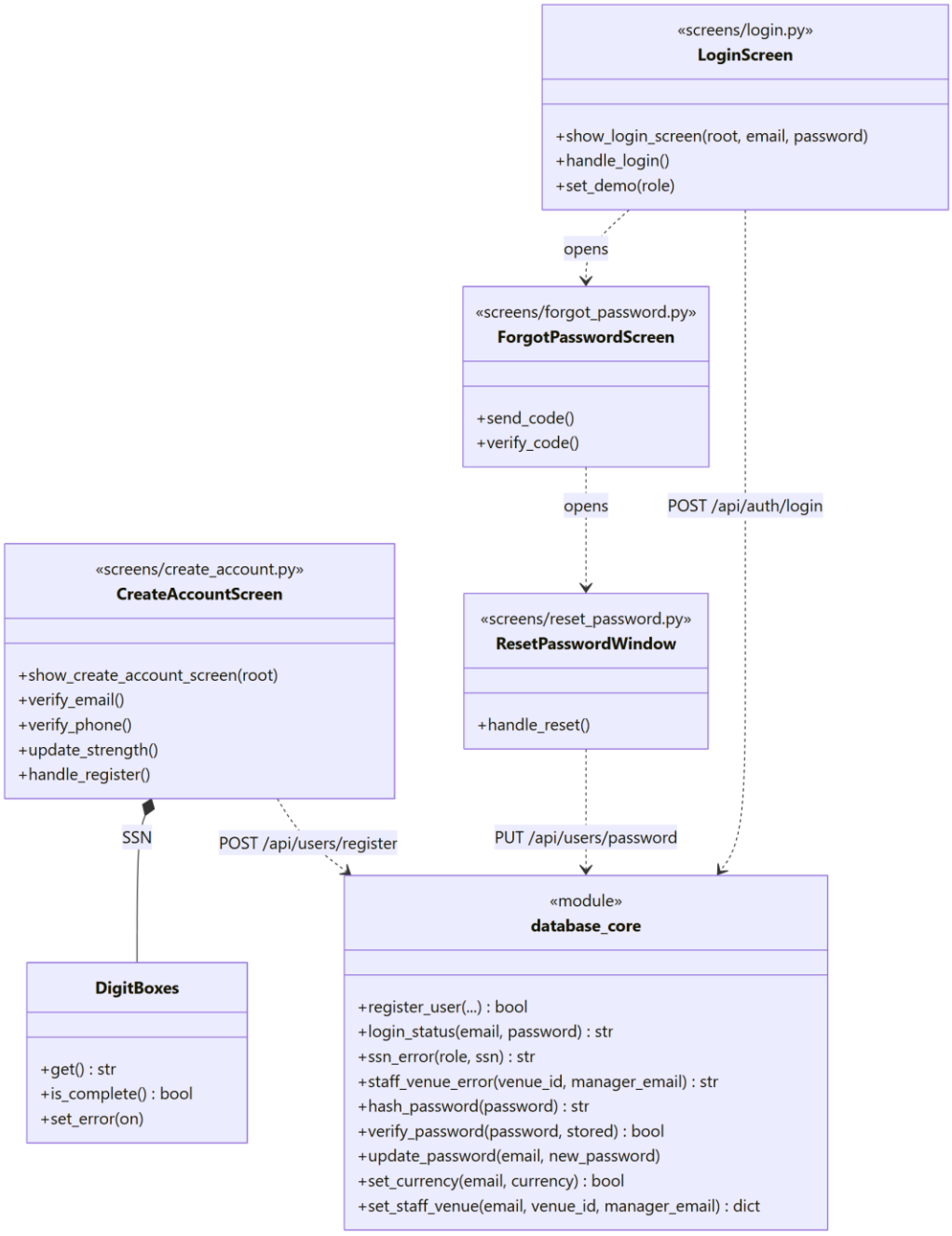


Figure 14: Class diagram of the Accounts and Sign-in component

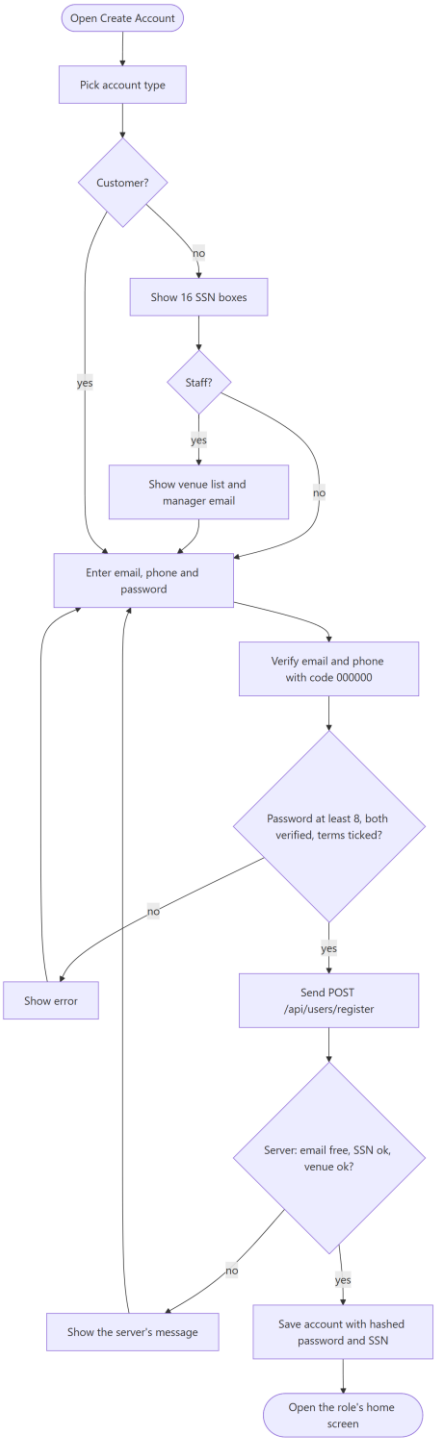


Figure 15: Activity diagram: creating an account

Figure 15: the client checks the form first, so most mistakes are caught without a server call. The server then checks the email, SSN and venue again, because the client could be bypassed.

3.2.3.2 Restrictions/limitations for component Accounts and Sign-in

There is no session token. After login the client only remembers the email and sends it with later requests. The email and phone codes are always 000000, and the Google and Apple buttons only open an ordinary email and password form. The password change endpoint does not ask for the old password, and anyone who knows a registered email and the fixed code could reset that password.

3.2.3.3 Performance issues for component Accounts and Sign-in

Hashing with 200,000 rounds is slow on purpose, so a login takes a moment. It happens once per login, so it does not slow down the rest of the app.

3.2.3.4 Design constraints for component Accounts and Sign-in

Passwords need at least 8 characters and cannot be words like 'password'. The SSN must be exactly 16 digits and cannot be the same digit 16 times. The phone number is a UAE mobile number with 9 digits after +971.

3.2.3.5 Processing detail for each operation of component Accounts and Sign-in

The component has the operations listed below:

```
+ login_status(email: str, password: str): str
+ register_user(email, password, role, phone, ssn, venue_id,
manager_email): bool
+ ssn_error(role: str, ssn: str): str or None
+ staff_venue_error(venue_id, manager_email): str or None
+ hash_password(password: str): str
+ verify_password(password: str, stored: str): bool
+ update_password(email: str, new_password: str): None
+ set_currency(email: str, currency: str): bool
```

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. login_status() is invoked when the user presses Sign In. It returns 'ok', 'invalid' or 'banned'.

2. register_user() is invoked when the user presses Create Account. It saves the account only if every rule passes.

3. ssn_error() is invoked by sign-up on the server. It returns the message to show, or nothing when the SSN is fine.

4. staff_venue_error() is invoked at staff sign-up and when staff change their venue in Settings.

5. hash_password() is invoked whenever a password or an SSN is stored.

6. verify_password() is invoked by login_status.

7. update_password() is invoked by the reset window and by Settings.

8. set_currency() is invoked when the user saves a preferred currency in Settings.

3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

1. login_status()

```
Start
Find the account by email, ignoring letter case
If there is no account or the password does not match, return 'invalid'
If the stored password is old plain text, replace it with a hash
If the account is banned, return 'banned'
Return 'ok'
End
```

2. register_user()

```
Start
If the role is not allowed at sign-up, use 'Customer'
If the email is taken or the SSN is wrong for this role, return false
If the role is Staff, check the venue and manager email
Save the account with the hashed password and hashed SSN (last 4 digits
kept)
Return true
End
```

3. ssn_error()

```
Start
If the role is Customer, return nothing
Remove spaces and dashes
If the SSN is empty, not 16 digits, or one digit repeated, return the
message
Return nothing
End
```

4. staff_venue_error()

```
Start
If no venue was chosen or it does not exist, return the message
If the manager email is empty or not an email address, return the message
Return nothing
End
```

5. hash_password()

```
Start
Make a random 16-byte salt
Run PBKDF2-SHA256 with 200,000 rounds
Return 'pbkdf2$200000$salt$hash'
End
```

6. verify_password()

```
Start
If the stored text is an old plain password, compare it safely
Else split it into rounds, salt and hash and hash the typed password the
same way
Return true if both hashes match
End
```

7. update_password()

```
Start
Hash the new password
Save it for the account with that email
End
```

8. set_currency()

```
Start
If the currency is not AED, USD, EUR or GBP, return false
Save it for the account
Return true
End
```

3.2 Description for Component “Events and Venues”

3.2.1 Processing narrative (PSPEC) for component

This is the vendor's side of Rihla. A vendor creates events with a title, category, price in AED, description, venue, a time, a capacity and a picture. An event either happens once on a date or repeats on chosen weekdays, for example every Tuesday and Thursday at 15:30. The vendor also keeps a list of venues, each with an optional Google Maps link, and a list of categories.

Deleting an event is the most careful operation here, because customers may already have paid. The server frees the event's vehicles, cancels its upcoming bookings that were not used yet with a full refund, cancels their tickets, and only then deletes the event and its picture.

3.2.2 Component interface description

The vendor works from the pages My Events, Bookings, Manage Venues, Manage Categories and Add New Event. One form is used both for adding and for editing an event. Pictures are picked with the normal file dialog and sent with the rest of the form. The Bookings page is a table of every booking on the vendor's events, and its columns can be filtered.

Endpoint	Sends	Gets back
GET /api/events, /api/events/vendor/<email>		events
POST /api/events	event fields, picture (base64)	success (vendors only)
PUT and DELETE /api/events/<id>	event fields, or nothing	success
GET /api/events/<id>/image		JPEG or 404
GET /api/bookings/vendor/<email>		bookings on the vendor's events
POST /api/venues, DELETE /api/venues/<id>	name, address, maps_url	success or 'in use'
PUT /api/venues/<id>/maps_url	vendor_email, maps_url	cleaned link or error
GET, POST, DELETE /api/vendor_categories	vendor_email, name	categories

Table 19: Server endpoints used by “Events and Venues”

3.2.3 Component processing detail

- Start
- Vendor opens My Events

- Vendor adds, edits or deletes an event, a venue or a category
- System checks the capacity, the picture and the map link
- System saves the change and renames the event in old bookings and tickets if the title changed
- If an event is deleted, system cancels its upcoming bookings with a full refund
- The updated event appears for customers and partners
- End

3.2.3.1 Design Class hierarchy for component Events and Venues

VendorWelcomeScreen builds all five pages, with open_event_form used for both adding and editing (Figure 16). The event_images module loads, shrinks and shows pictures, and CoverImage is the widget that scales a picture to fill a card.

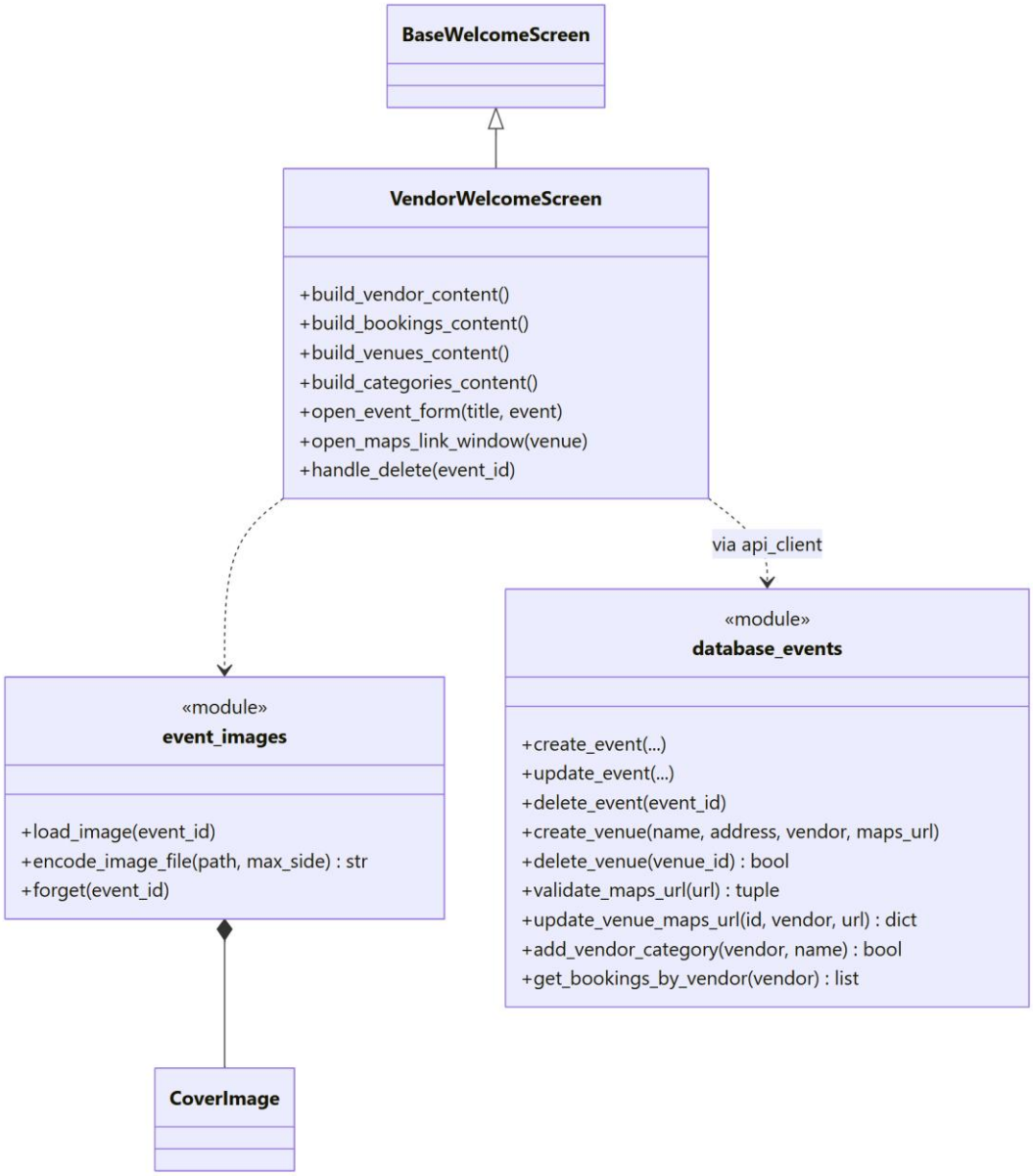


Figure 16: Class diagram of the Events and Venues component

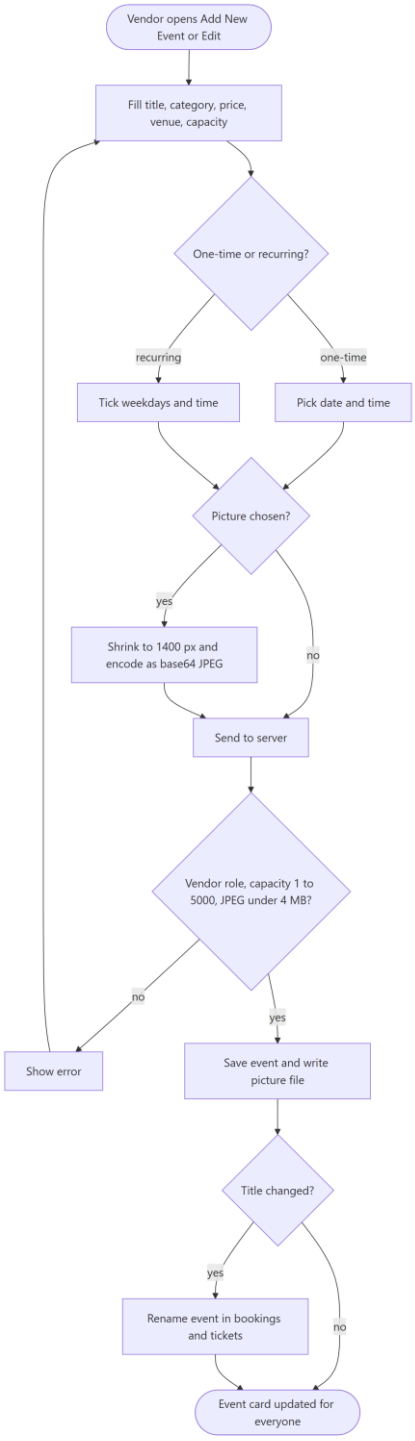


Figure 17: Activity diagram: a vendor saves an event

Figure 17: the picture is shrunk on the client before it is sent, and the server checks it again. Renaming an event also renames it inside old bookings and tickets, so customers never see an old title.

3.2.3.2 Restrictions/limitations for component Events and Venues

Updating or deleting an event and deleting a venue do not check that the caller owns it. The client only shows a vendor their own records, but the server does not enforce it yet. Prices are saved as text and only the digits are read, so a price like 99.50 is not possible.

3.2.3.3 Performance issues for component Events and Venues

Pictures are shrunk to 1400 pixels before upload and limited to 4 MB, so the lists with many cards still load quickly.

3.2.3.4 Design constraints for component Events and Venues

Capacity must be between 1 and 5000. Pictures must be JPEG. A venue cannot be deleted while events use it or staff work there. Map links must be https links to Google Maps.

3.2.3.5 Processing detail for each operation of component Events and Venues

The component has the operations listed below:

```
+ create_event(title, category, price, description, vendor_email,
venue_id, event_datetime, is_recurring, recur_day, recur_time, capacity):
None
+ update_event(event_id, title, ..., capacity): None
+ delete_event(event_id: int): None
+ create_venue(name, address, vendor_email, maps_url): None
+ delete_venue(venue_id: int): bool
+ validate_maps_url(url: str): (str, str or None)
+ add_vendor_category(vendor_email: str, name: str): bool
```

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. create_event() is invoked when the vendor saves a new event.

2. update_event() is invoked when the vendor saves an edited event.

3. delete_event() is invoked when the vendor presses Delete on an event card and confirms.

4. create_venue() is invoked from Manage Venues after the map link was checked.

5. delete_venue() is invoked from Manage Venues.

6. validate_maps_url() is invoked when a venue is created or its map link is changed. It returns the cleaned link or a message.

7. add_vendor_category() is invoked from Manage Categories.

3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

1. create_event()

```
Start
Check the capacity (empty means 20, otherwise 1 to 5000)
Save the event
End
```

2. update_event()

```
Start
If a capacity was sent, check and save it
Save the other fields
Copy the new title into the event's bookings and tickets
End
```

3. delete_event()

```
Start
Set the event's assigned vehicles back to Available
Remove the event's vehicle assignments
Cancel upcoming bookings with an unused ticket (100% refund, reason given)
Cancel their tickets
Delete the event
End
```

4. create_venue()

```
Start
Save the venue with its owner and map link
End
```

5. delete_venue()

```
Start
If an event uses the venue or a staff member works there, return false
Delete the venue
Return true
End
```

6. validate_maps_url()

```
Start
If the link is empty, accept it
If it is too long or has spaces, refuse it
Add https:// when needed
If the host is a Google Maps address, accept it, otherwise refuse it
End
```

7. add_vendor_category()

```
Start
If the vendor already has a category with this name (any letter case),
return false
Save the category
Return true
End
```

3.2 Description for Component “Booking and Shopping List”

3.2.1 Processing narrative (PSPEC) for component

This component turns a choice of event, date and guests into a booking with a ticket. It is used by customers and partners.

The event page shows a calendar where only the allowed dates can be picked, a capacity bar such as '12 of 30 remaining' (with 'Hurry up!' when it is almost full) and the total price. When the payment is approved, the server checks the seats once more, saves the booking and creates a new 16-digit ticket code that is not used by any other ticket.

Partners work a little differently. They have no limit of 10 guests per booking, they can type a client name for the group, and they can collect several events in a shopping list and pay for all of them at once. A customer may also book the same event more than once, for example for two groups of friends.

3.2.2 Component interface description

The customer starts from Explore Tours and opens an event with View Details. The partner has the same page with Add to List and Pay Now instead of one booking

button. After payment both see the Booking Details window, and the booking appears in My Bookings or Partner Bookings.

Endpoint	Sends	Gets back
GET /api/events/<id>/capacity?date=	event id, date	total, booked, remaining
POST /api/bookings	email, event_id, booking_date, guests, client_name	booking with base, VAT, total, ticket code
GET /api/bookings/customer/<email>		bookings with status and refund info
POST /api/cart, GET /api/cart/<email>	partner_email, event_id, date, guests, client_name	cart items
POST /api/cart/<id>/remove, .../clear	partner_email	success
POST /api/cart/<email>/checkout		all bookings, or none

Table 20: Server endpoints used by "Booking and Shopping List"

3.2.3 Component processing detail

- Start
- User opens an event and picks a date and the number of guests
- System shows the seats left on that date and the price
- User presses Proceed to Booking (or Add to List, for partners)
- User pays in the payment window (see the next component)
- System checks the seats again, saves the booking and creates a ticket code
- User sees the Booking Details window and the booking in their list
- End

3.2.3.1 Design Class hierarchy for component Booking and Shopping List

Figure 18 shows the classes. CustomerWelcomeScreen and PartnerWelcomeScreen list the events and the bookings, and both open the same event page. The event page holds a CapacityBar and calls the payment window. On the server the work is done by create_booking and _book.

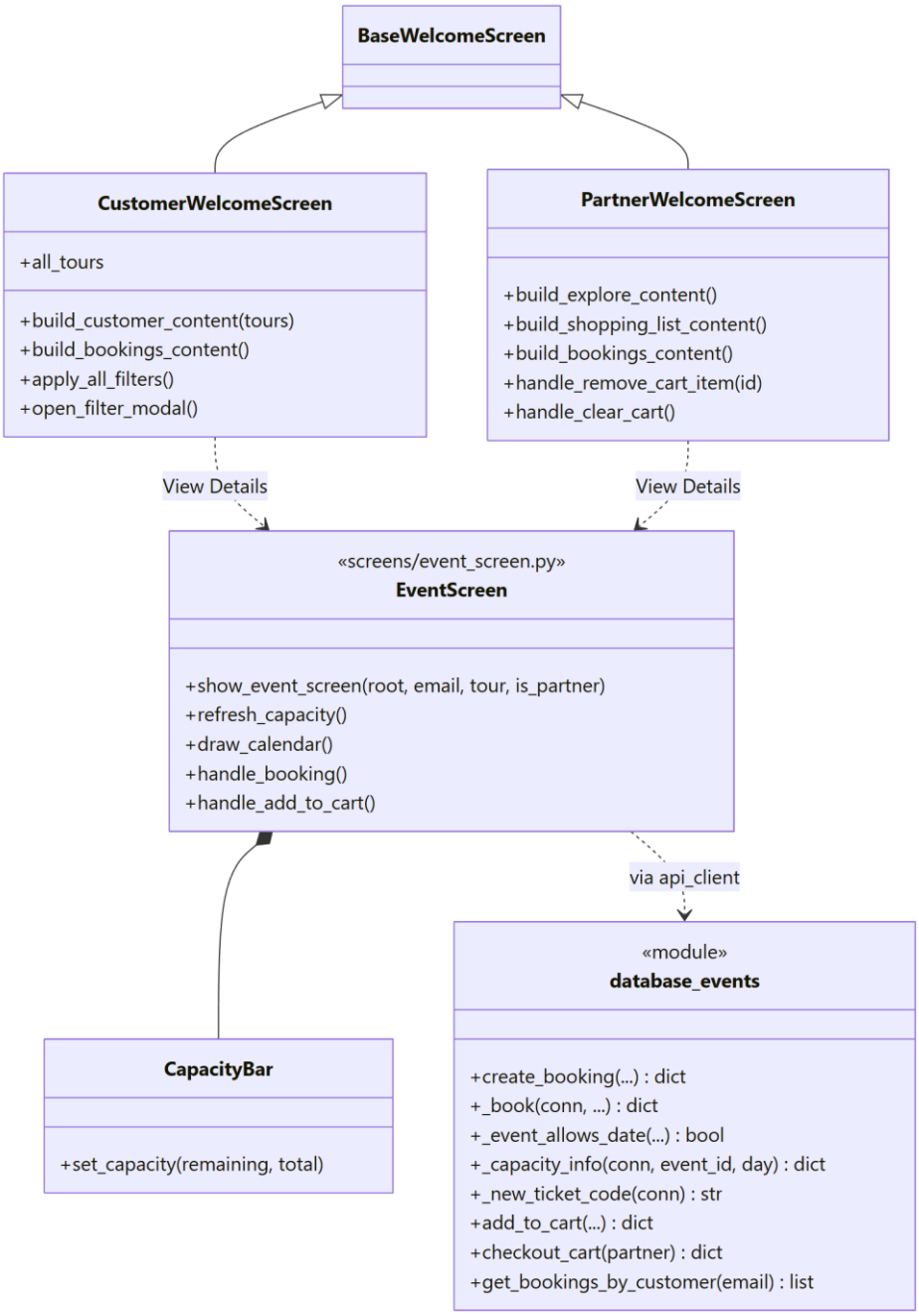


Figure 18: Class diagram of the Booking and Shopping List component

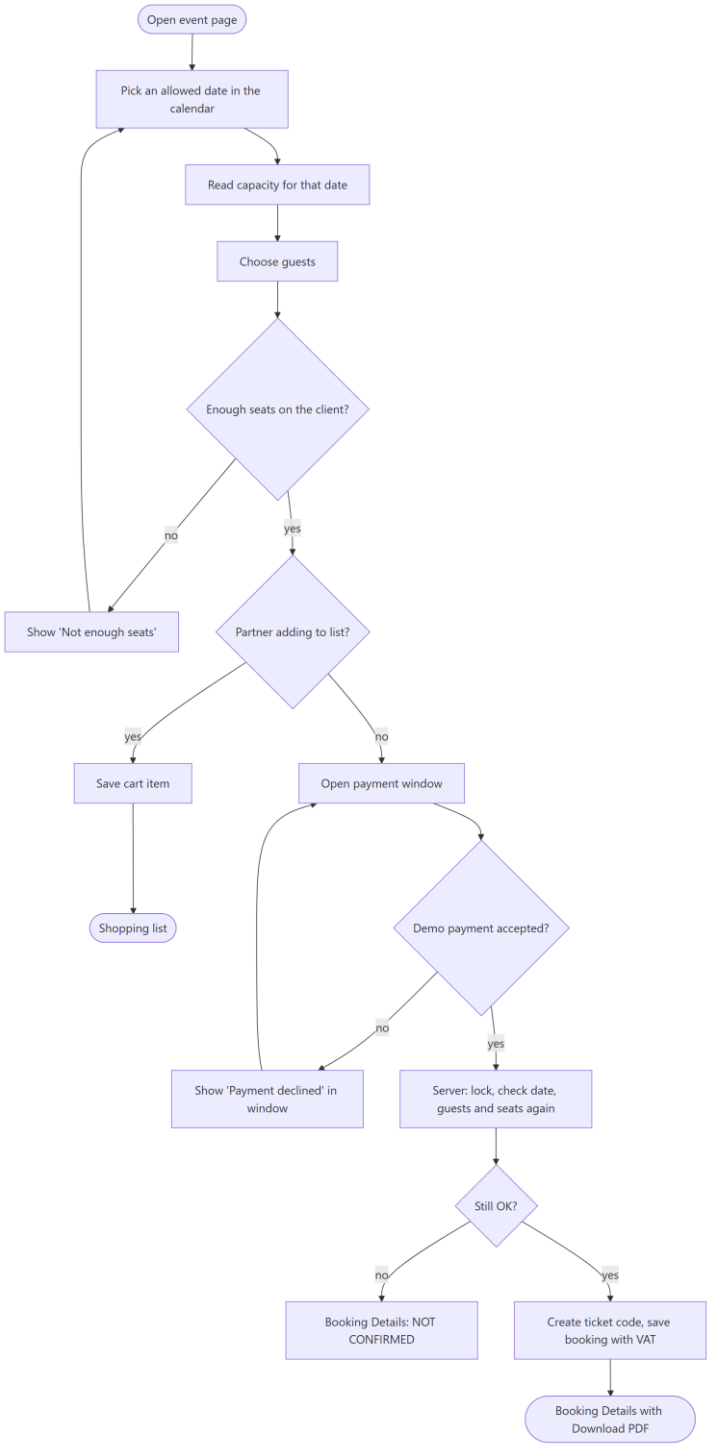


Figure 19: Activity diagram: booking an event

Figure 19: the seats are checked twice, once on the client for a quick answer and once on the server inside the transaction. Only the second check counts.

3.2.3.2 Restrictions/limitations for component Booking and Shopping List

The payment is approved on the client before the server saves the booking. If the booking fails at that point, the Booking Details window says NOT CONFIRMED and asks the user to contact support. With a real payment provider this order would have to change.

3.2.3.3 Performance issues for component Booking and Shopping List

A booking is one short write. BEGIN IMMEDIATE takes the database write lock at the start, so two people booking the last seats at the same time are handled one after the other instead of both getting them.

3.2.3.4 Design constraints for component Booking and Shopping List

The date must be today or later and must fit the event: the right weekday for a repeating event or the exact date for a one-time event. Customers book 1 to 10 guests. Nobody can book more than the seats left on that date.

3.2.3.5 Processing detail for each operation of component Booking and Shopping List

The component has the operations listed below:

```
+ create_booking(email, event_id, booking_date, guests, is_partner,
client_name): dict
- _book(conn, email, event_id, day, guests, is_partner, client_name): dict
- _event_allows_date(is_recurring, recur_day, event_datetime, day): bool
- _capacity_info(conn, event_id, day): dict
- _new_ticket_code(conn): str
+ add_to_cart(partner_email, event_id, booking_date, guests, client_name):
dict
+ checkout_cart(partner_email: str): dict
+ get_bookings_by_customer(email: str): list
```

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. create_booking() is invoked after the demo payment is approved. It opens a transaction and calls _book. The server decides is_partner from the account's role, not from what the client sends.

2. _book() is invoked by create_booking and by checkout_cart. It does the checks and saves the booking inside the transaction it is given.

3. _event_allows_date() is invoked by _book and add_to_cart.

4. _capacity_info() is invoked whenever seats are needed: on the event page, when booking and when adding to the cart.

5. _new_ticket_code() is invoked by _book.

6. add_to_cart() is invoked by Add to List on the event page.

7. checkout_cart() is invoked when the partner pays for the shopping list.

8. get_bookings_by_customer() is invoked by My Bookings and Partner Bookings.

3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

1. create_booking()

```
Start
Open a write transaction
Call _book with the request
Return its result
End
```

2. _book()

```
Start
Check the date and number of guests (1 to 10 unless partner)
Refuse dates in the past or dates the event does not run on
Get the seats left on that date
If there are not enough seats, refuse with the numbers
Make a new random 16-digit ticket code that is not used yet
Work out base = price x guests, VAT = 5% of base, total = base + VAT
Save the ticket as Valid and the booking as Confirmed with the total
Return the booking with base, VAT and total
End
```

3. _event_allows_date()

```
Start
If the event repeats, return true when the weekday of the day is in its
list
Else return true only when the day is the event's own date
End
```

4. _capacity_info()

```
Start
If the event has assigned vehicles, total = sum of their seats
Else total = the event's own capacity
Booked = guests of confirmed bookings on that day
Remaining = total - booked (never below 0)
End
```

5. _new_ticket_code()

```
Start
Repeat
   Make a random 16-digit number that does not start with 0
Until no ticket has this code
Return it
End
```

6. add_to_cart()

```
Start
Check date, guests and that the event runs on that day
If there are not enough seats now, refuse
Save the item in the partner's cart
End
```

7. checkout_cart()

```
Start
If the cart is empty, refuse
Open a write transaction
Book every item with _book
If one item fails, undo everything and refuse
Empty the cart and return all bookings
End
```

8. get_bookings_by_customer()

```
Start
Read the user's bookings with their ticket status
For each one work out the shown status and the refund if cancelled now
Return the list
End
```

3.2 Description for Component “Pricing, Payment and Receipts”

3.2.1 Processing narrative (PSPEC) for component

This component handles money on the client side. It works out VAT, shows prices in the user's currency, runs the demo payment and builds the receipt.

Every price in the database is in AED. The 5% VAT is added on top of the base price, rounded to two decimals and shown as its own line. USD, EUR and GBP are only for display, using fixed rates. The user can switch the currency inside the payment window, and the default is the preferred currency saved in Settings.

The receipt is built once as a list of lines and used twice: for the Booking Details window and for the PDF. We wrote the PDF writer ourselves in booking_pdf.py, with plain PDF objects and the built-in Helvetica font, so the app does not need an extra library for it.

3.2.2 Component interface description

The payment window opens on top of the event page or the shopping list. It shows the order summary, a currency box, base price, VAT and total, and three payment methods (Visa, Mastercard and bank transfer) that are already filled in with demo details. After payment, the Booking Details window shows the transaction id, method, currency, event details, ticket codes and prices, with Download PDF and Close. This component does not call the server itself. It hands the result back to the booking component.

3.2.3 Component processing detail

- Start
- Payment window opens with the base price in AED
- System adds VAT and shows the prices in the preferred currency
- User may change the currency and the payment method
- User presses Pay
- System compares the form with the demo details
- If they match, the booking is saved and the Booking Details window opens
- User may download the receipt as a PDF

- End

3.2.3.1 Design Class hierarchy for component Pricing, Payment and Receipts

payment, pricing and booking_pdf are plain modules, not classes (Figure 20). This kept them easy to test: the tests call verify_demo_payment directly with a correct and an edited card.

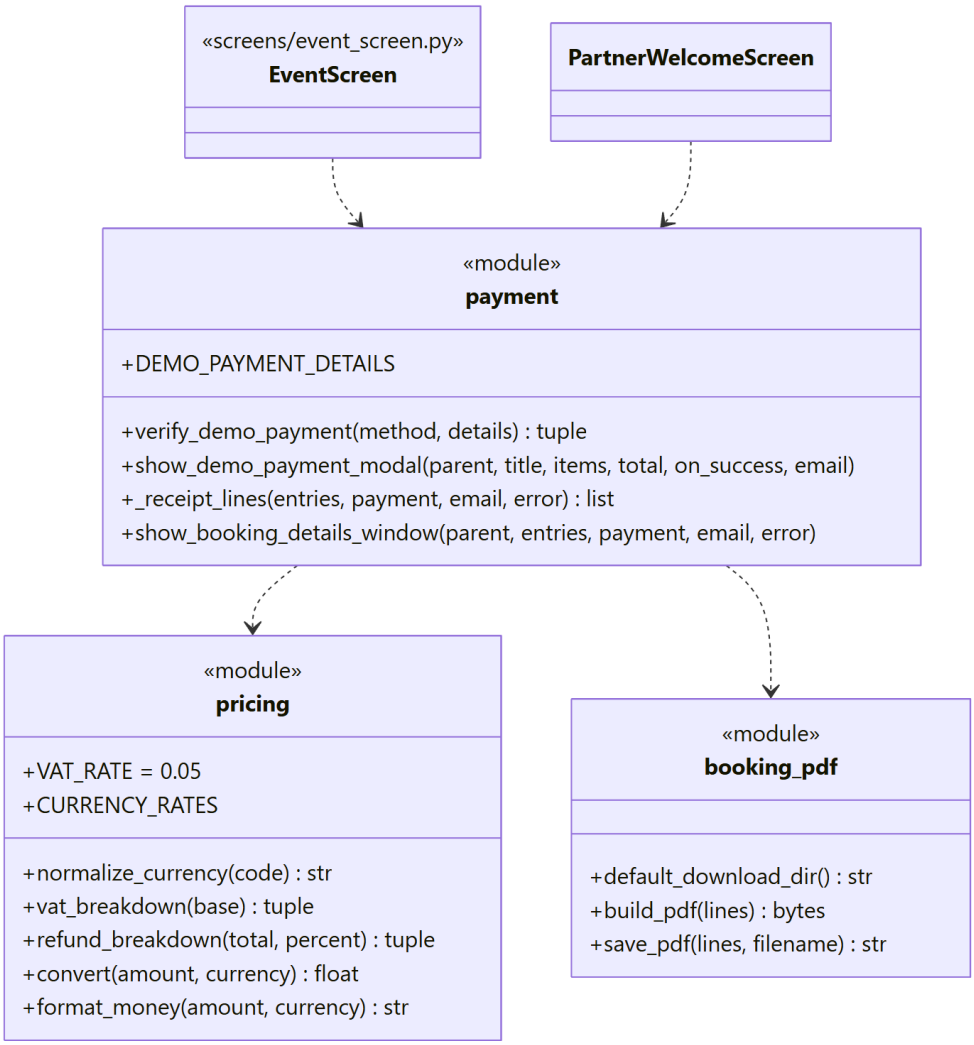


Figure 20: Class diagram of the Pricing, Payment and Receipts component

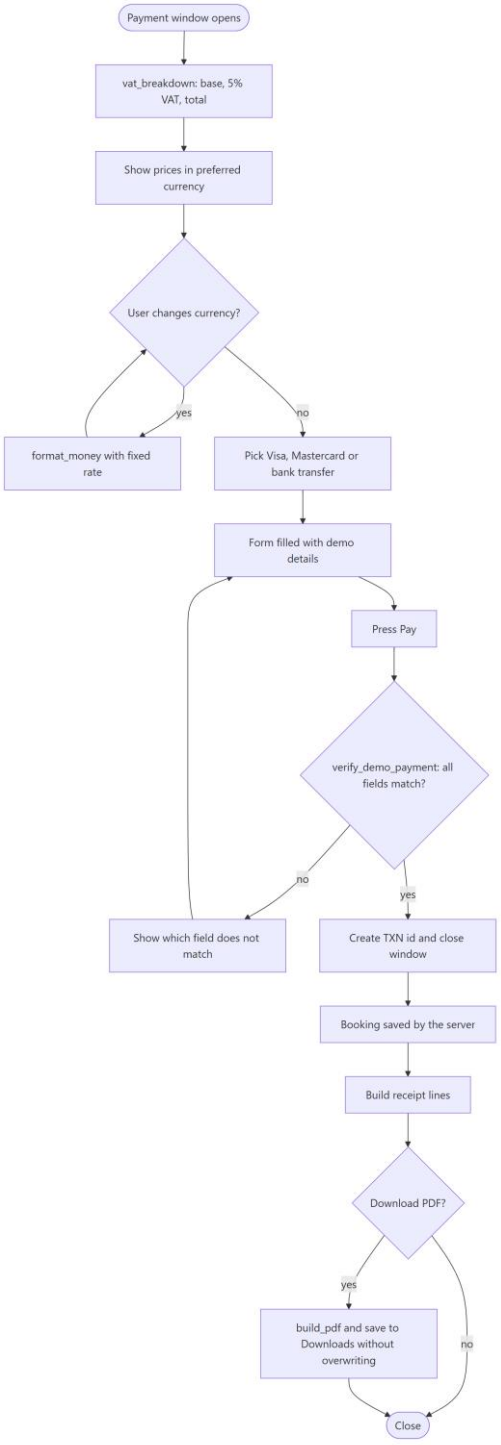


Figure 21: Activity diagram: the payment window and the receipt

Figure 21: a declined payment stays inside the payment window with a message about the field that does not match, so the user can fix it without starting again.

3.2.3.2 Restrictions/limitations for component Pricing, Payment and Receipts

The payment is a demo. It only checks that the form still has the fixed demo card or bank details, so changing a digit makes it fail on purpose. Currency rates are fixed numbers and everything is charged in AED. The PDF uses a basic font, so characters outside Latin-1 are replaced.

3.2.3.3 Performance issues for component Pricing, Payment and Receipts

All of this runs on the client and takes no time. Building the PDF only writes a few kilobytes.

3.2.3.4 Design constraints for component Pricing, Payment and Receipts

VAT is always 5% and always shown. The PDF is saved in the Downloads folder and never overwrites an old receipt.

3.2.3.5 Processing detail for each operation of component Pricing, Payment and Receipts

The component has the operations listed below:

```
+ vat_breakdown(base_aed: float): (float, float, float)
+ format_money(amount_aed: float, currency: str): str
+ verify_demo_payment(method: str, details: dict): (bool, str)
+ show_demo_payment_modal(parent, title, items, total_amount, on_success,
email): None
+ show_booking_details_window(parent, entries, payment, payer_email,
error, on_close): window
+ save_pdf(lines: list, filename: str): str
```

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. vat_breakdown() is invoked by the payment window and the receipt.

2. format_money() is invoked wherever a price is shown.

3. verify_demo_payment() is invoked when the user presses Pay.

4. show_demo_payment_modal() is invoked by the event page and the shopping list.

5. show_booking_details_window() is invoked after the booking was saved, or failed.

6. save_pdf() is invoked by Download PDF.

3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

1. vat_breakdown()

```
Start
Base = base rounded to 2 decimals
VAT = base x 0.05 rounded to 2 decimals
Return base, VAT and base + VAT
End
```

2. format_money()

```
Start
If the currency is unknown, use AED
Multiply by the fixed rate and round to 2 decimals
Return the number with thousands separators and the currency code
End
```

3. verify_demo_payment()

```
Start
If the method is a card, compare number, cardholder, expiry and CVV with
the demo card
If the method is bank transfer, compare bank name, account name, IBAN and
SWIFT
Return false with the first field that does not match, or true
End
```

4. show_demo_payment_modal()

```
Start
Work out base, VAT and total
Show summary, currency box and the form of the chosen method
On Pay, verify the details
If accepted, create a transaction id and call on_success
End
```

5. show_booking_details_window()

```
Start
Build the receipt lines
```

```
Show them with a green header (or red when the booking failed)
On Download PDF, save the PDF and show where it was saved
End
```

6. save_pdf()

```
Start
Find the Downloads folder (registry on Windows, otherwise home)
If the file name exists, add (1), (2) and so on
Write the PDF built by build_pdf
Return the full path
End
```

3.2 Description for Component “Cancellations and Refunds”

3.2.1 Processing narrative (PSPEC) for component

We wanted one refund policy that every role follows, written in one function. A customer or partner can cancel their own booking. They get 100% back if the event starts in 48 hours or more, 50% between 48 and 24 hours, and they cannot cancel in the last 24 hours. VAT is refunded in the same share as the price.

A vendor can cancel bookings on their own events, and the admin can cancel any booking. Both can do it right up to the start, always refund 100%, and must give the customer a reason. Every cancellation saves when it happened, who did it, their role, the reason, the percent and the amount. The seats are free again straight away.

3.2.2 Component interface description

Customers cancel from My Bookings and partners from Partner Bookings. Each card already shows what the refund would be, so there are no surprises. Vendors and admins select a row in their bookings table and press Cancel, and a small window asks for the reason. Cancelled bookings then show who cancelled them, why and how much was refunded.

Endpoint	Sends	Gets back
POST /api/bookings/<id>/cancel	customer_email	refund percent, base, VAT, total

Endpoint	Sends	Gets back
POST /api/bookings/<id>/vendor_cancel	vendor_email, reason	refund (vendors only)
POST /api/admin/bookings/<id>/cancel	admin_email, reason	refund (admins only)
GET /api/admin/bookings?admin_email=		every booking

Table 21: Server endpoints used by "Cancellations and Refunds"

3.2.3 Component processing detail

- Start
- User presses Cancel on a booking
- System shows the refund the user would get (or asks vendors and admins for a reason)
- User confirms
- System checks that the user may cancel this booking and that the event has not started
- System works out the refund percent from the hours left
- System marks the ticket and the booking as cancelled and saves the refund
- User sees the refund amount and the seats become free
- End

3.2.3.1 Design Class hierarchy for component Cancellations and Refunds

The client part is small (Figure 22). cancellation.py has the policy text, the confirm dialog and the reason dialog, and four different screens use it. On the server, cancel_booking, vendor_cancel_booking and admin_cancel_booking all call one shared function, _cancel, and only pass a different owner check.

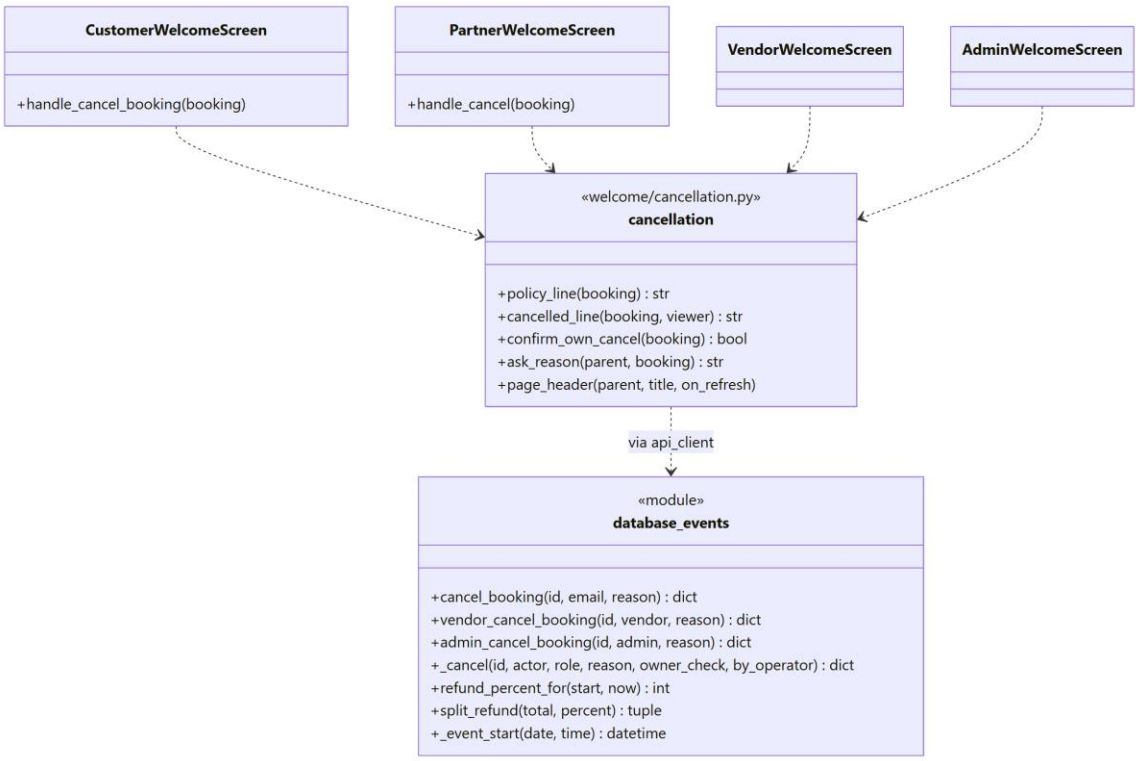


Figure 22: Class diagram of the Cancellations and Refunds component

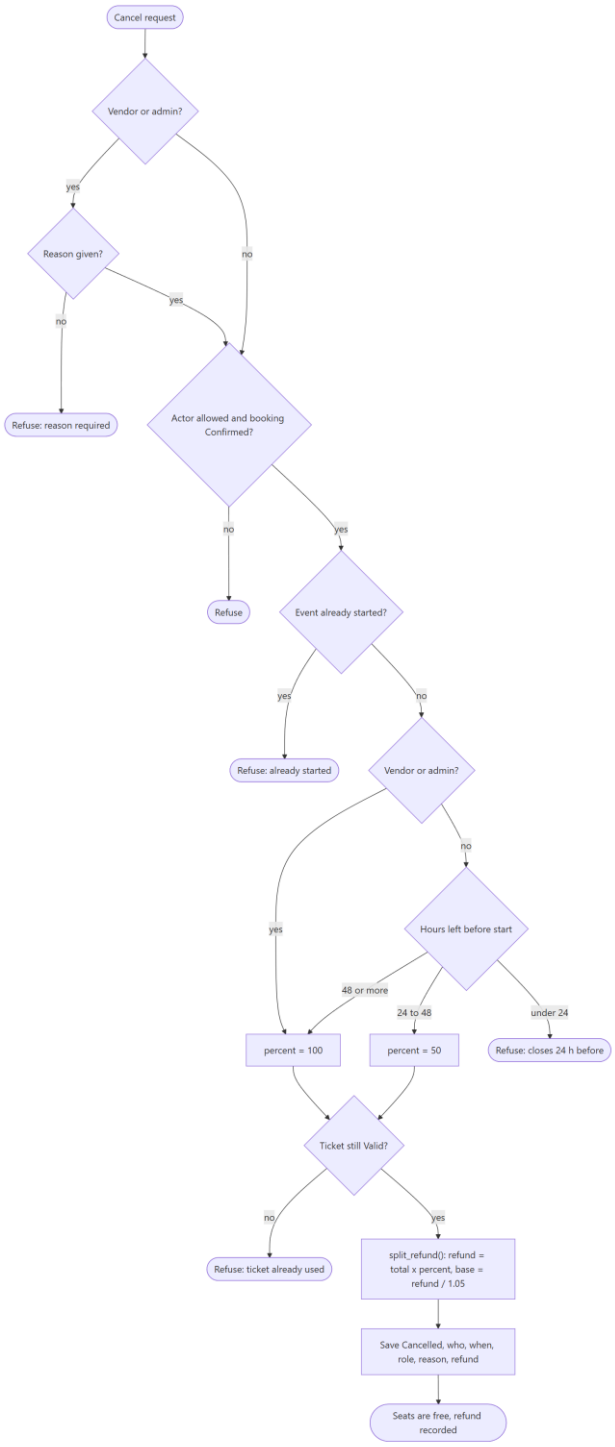


Figure 23: Activity diagram: the refund decision in _cancel()

Figure 23: the same function handles customers, partners, vendors and admins. The only differences are who is allowed to cancel the booking and whether the refund follows the hours left or is always 100%.

3.2.3.2 Restrictions/limitations for component Cancellations and Refunds

Refunds are written to the database only. No money moves, because the payments are a demo too. A booking with no time saved counts from midnight of its date.

3.2.3.3 Performance issues for component Cancellations and Refunds

Each cancellation is one short transaction, the same as a booking.

3.2.3.4 Design constraints for component Cancellations and Refunds

A checked-in ticket cannot be cancelled, and nobody can cancel after the event has started. The reason can be at most 300 characters.

3.2.3.5 Processing detail for each operation of component Cancellations and Refunds

The component has the operations listed below:

```
+ cancel_booking(booking_id: int, customer_email: str, reason: str): dict
+ vendor_cancel_booking(booking_id: int, vendor_email: str, reason: str):
dict
+ admin_cancel_booking(booking_id: int, admin_email: str, reason: str):
dict
- _cancel(booking_id, actor_email, actor_role, reason, owner_check,
by_operator): dict
+ refund_percent_for(start: datetime): int or None
+ split_refund(total_price: float, percent: int): (float, float, float)
```

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. cancel_booking() is invoked when a customer or partner cancels their own booking.

2. vendor_cancel_booking() is invoked from the vendor's Bookings table.

3. admin_cancel_booking() is invoked from the admin's Bookings page.

4. _cancel() is invoked by all three functions above.

5. refund_percent_for() is invoked by _cancel and also when bookings are listed, so the screen can show the refund in advance.

6. split_refund() is invoked by _cancel.

3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

1. cancel_booking()

```
Start
Owner check: the booking's email must be the caller's email
Call _cancel as a booker (refund from the hours left)
End
```

2. vendor_cancel_booking()

```
Start
Owner check: the booking's event must belong to the vendor
Call _cancel as an operator (100%, reason required)
End
```

3. admin_cancel_booking()

```
Start
If the caller is not an active admin, refuse
Call _cancel as an operator with no owner check
End
```

4. _cancel()

```
Start
If a vendor or admin gives no reason, refuse
Open a write transaction and read the booking
If the actor may not cancel it, or it is not Confirmed, refuse
If the event already started, refuse
Percent = 100 for vendor or admin, otherwise from the hours left
If the percent is none (under 24 hours), refuse
Mark the ticket Cancelled only if it is still Valid, otherwise refuse
Split the refund into base and VAT
Save the booking as Cancelled with who, when, why and the refund
End
```

5. refund_percent_for()

```
Start
Hours left = start time - now
If 48 or more, return 100
If 24 or more, return 50
Return none
End
```

6. split_refund()

```
Start
Refund = total x percent / 100, rounded
Base = refund / 1.05, rounded
VAT = refund - base
End
```

3.2 Description for Component “Venue Operations”

3.2.1 Processing narrative (PSPEC) for component

This component is for staff on the ground. Every staff account belongs to one venue, chosen at sign-up and changeable in Settings. The Venue Operations page shows that venue, its manager's email, a check-in card and today's schedule for the venue with the number of bookings, guests and guests already checked in.

We added the venue rule late in the project. Before it, any staff member could check in any ticket in the system. Now the server refuses a ticket for an event at another venue, the schedule only lists the staff member's own venue, and the Support Tickets page only shows questions about events there. Old support tickets from before customers had to pick an event are shown to all staff.

3.2.2 Component interface description

Staff type the 16-digit code into one box per digit and press Validate. The answer appears under the boxes, for example 'Access granted! Ticket valid for ...' followed by how many guests to admit. If the booking is for another day, the message says so.

Endpoint	Sends	Gets back
POST /api/tickets/verify	staff_email, ticket_code	access granted or the reason
GET /api/staff/venue/<email>		venue and manager
GET /api/events/today?staff_email=		today's events at the venue
GET /api/support/open?staff_email=		open questions for the venue
POST /api/support/<id>/resolve	staff_email	resolved or refused

Table 22: Server endpoints used by “Venue Operations”

3.2.3 Component processing detail

- Start
- Staff member opens Venue Operations
- System shows the venue, the manager and today's schedule
- Staff member types a ticket code and presses Validate
- System checks that the ticket exists, belongs to this venue and is still Valid
- System marks the ticket as Used and says how many guests to admit
- Staff member can open Support Tickets and mark questions as resolved
- End

3.2.3.1 Design Class hierarchy for component Venue Operations

StaffWelcomeScreen builds both pages and reuses DigitBoxes for the code (Figure 24). The venue settings live in BaseWelcomeScreen, because the Settings window is shared by every role, and only staff see the venue part of it.

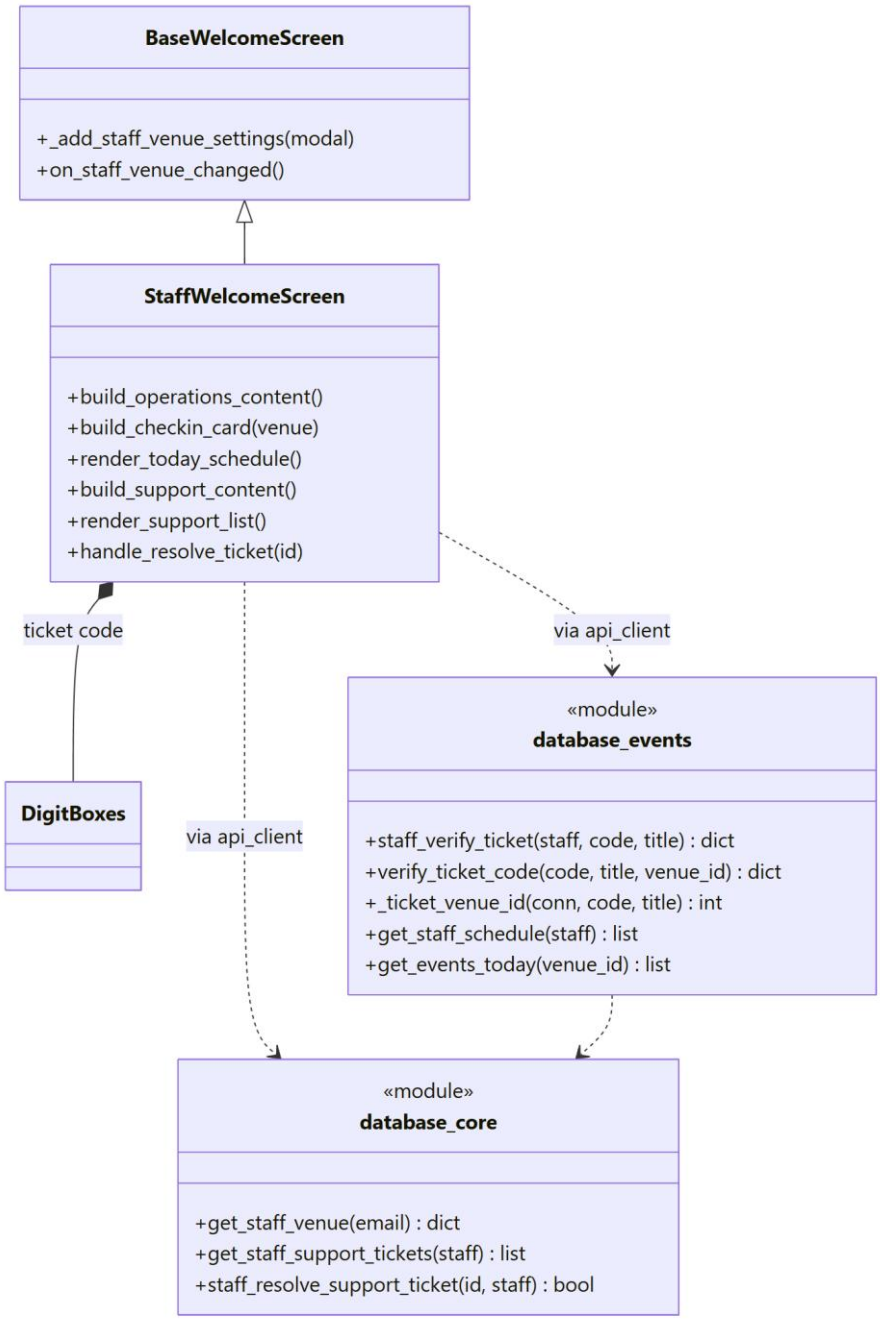


Figure 24: Class diagram of the Venue Operations component

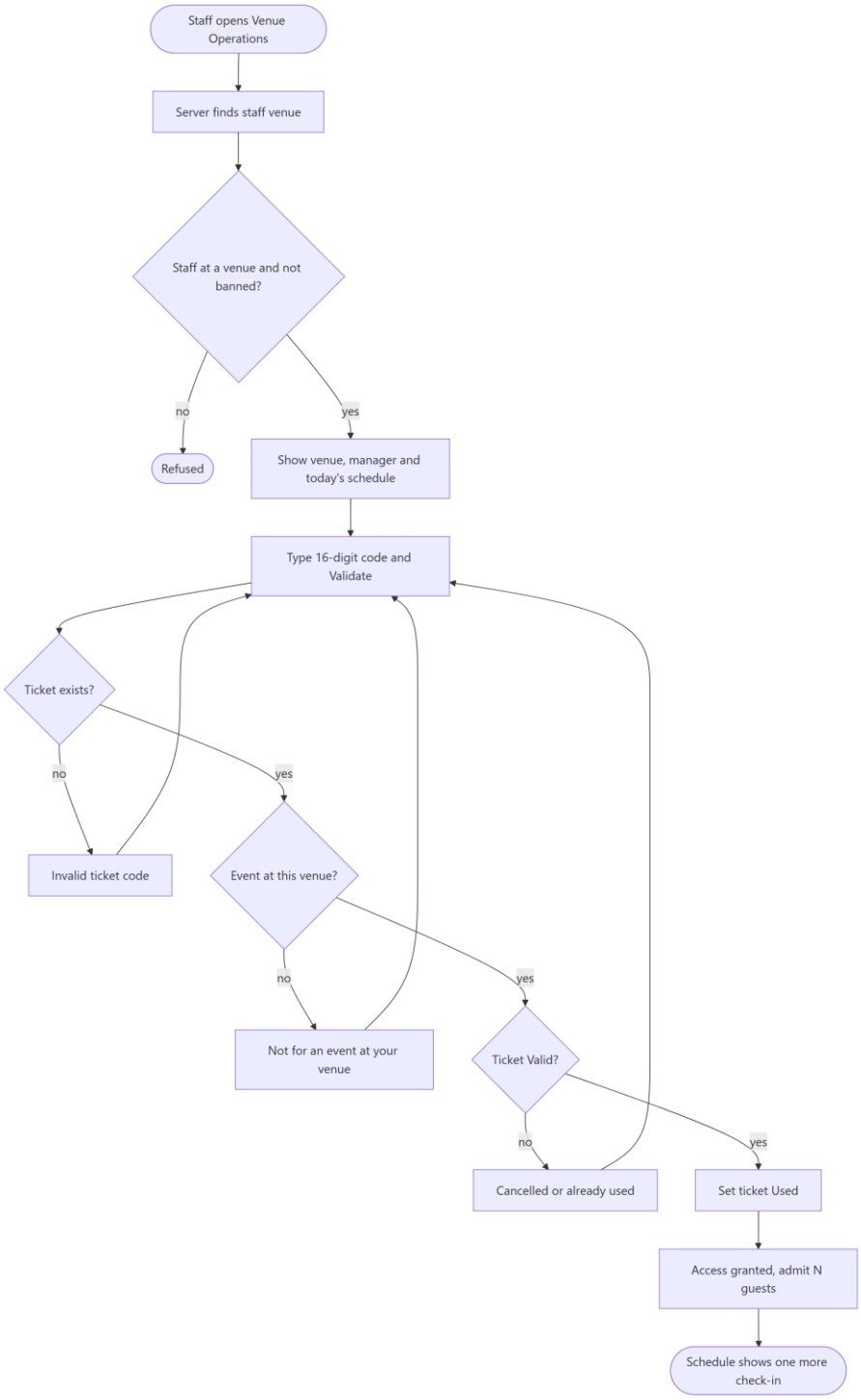


Figure 25: Activity diagram: checking a ticket in at the venue

Figure 25: every refusal sends the staff member back to the code boxes, so a queue of guests can be handled one after the other on the same screen.

3.2.3.2 Restrictions/limitations for component Venue Operations

Codes are typed by hand. There is no QR scanner, and the schedule only covers today.

3.2.3.3 Performance issues for component Venue Operations

The schedule runs two small queries per event of the venue. One venue only has a few events a day, so this is not a problem.

3.2.3.4 Design constraints for component Venue Operations

The update to Used only happens if the ticket is still Valid, so two staff members can never admit the same ticket, even at the same second.

3.2.3.5 Processing detail for each operation of component Venue Operations

The component has the operations listed below:

```
+ staff_verify_ticket(staff_email: str, ticket_code: str,
expected_event_title): dict
+ verify_ticket_code(ticket_code: str, expected_event_title, venue_id):
dict
- _ticket_venue_id(conn, ticket_code, event_title): int
+ get_staff_venue(email: str): dict or None
+ get_events_today(venue_id: int): list
+ staff_resolve_support_ticket(ticket_id: int, staff_email: str): bool
```

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. staff_verify_ticket() is invoked when staff press Validate.

2. verify_ticket_code() is invoked by staff_verify_ticket.

3. _ticket_venue_id() is invoked by verify_ticket_code.

4. get_staff_venue() is invoked by the page header, check-in and the schedule.

5. get_events_today() is invoked through get_staff_schedule when the page opens or Refresh is pressed.

6. staff_resolve_support_ticket() is invoked by Resolve on the Support Tickets page.

3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

1. staff_verify_ticket()

```
Start
Find the venue of this staff member (not banned)
If there is none, refuse
Call verify_ticket_code with that venue
End
```

2. verify_ticket_code()

```
Start
Find the ticket, refuse if it does not exist
If the ticket's event is at another venue, refuse
If it is for another event than expected, refuse
If the ticket is Cancelled or Used, refuse
Mark the ticket Used if it is still Valid
Return 'Access granted' with the number of guests
End
```

3. _ticket_venue_id()

```
Start
Find the venue through the ticket's booking and event
If there is no booking, use the first event with that title
End
```

4. get_staff_venue()

```
Start
Find the account with role Staff that is not banned
Return its venue with name, address, map link and manager email
End
```

5. get_events_today()

```
Start
For each event at the venue
   Keep it if it runs today (weekday or date)
   Count today's bookings, guests and checked-in guests
Return the list
End
```

6. staff_resolve_support_ticket()

```
Start
Find the staff member's venue
Mark the ticket Resolved only if it is open and about that venue
End
```

3.2 Description for Component “Fleet and Dispatch”

3.2.1 Processing narrative (PSPEC) for component

The dispatcher looks after the vehicles. They register vehicles with a license number, a type and a location, keep their status up to date, and give a group of vehicles to an event.

This is also where an event's seats come from. As soon as an event has vehicles, its capacity is the sum of their seats. For example two 4x4 Land Cruisers with 6 seats each give 12 seats, whatever the vendor typed. When the vehicles are freed, the event goes back to the vendor's number.

3.2.2 Component interface description

The dispatcher has three pages: Register Vehicles, Fleet Maintenance (a table with filters on the columns and check boxes to act on several vehicles) and Event Dispatch, where they pick an event, filter the available vehicles by type and location, and assign the ones they tick.

Endpoint	Sends	Gets back
GET /api/vehicle_types		types with seats
GET /api/vehicles/<dispatcher_email>		the dispatcher's vehicles
POST /api/vehicles, DELETE /api/vehicles/<id>	license_number, vehicle_type, location	success
PUT /api/vehicles/<id>/status, POST .../free	status	success
POST /api/dispatch	event_id, chosen vehicles	how many were assigned
GET /api/dispatch/<event_id>/vehicles		vehicles assigned to the event

Table 23: Server endpoints used by “Fleet and Dispatch”

3.2.3 Component processing detail

- Start

- Dispatcher registers vehicles
- Dispatcher opens Event Dispatch and picks an event
- System lists the available vehicles and the ones already assigned
- Dispatcher ticks vehicles and assigns them
- System marks them Assigned and the event's capacity becomes their seats
- Later the dispatcher frees a vehicle or sets it to Maintenance, which ends the assignment
- End

3.2.3.1 Design Class hierarchy for component Fleet and Dispatch

DispatcherWelcomeScreen is the only home screen that is built from two helper views, VehicleView and DispatchView, because its pages are large (Figure 26). Both use our shared table style.

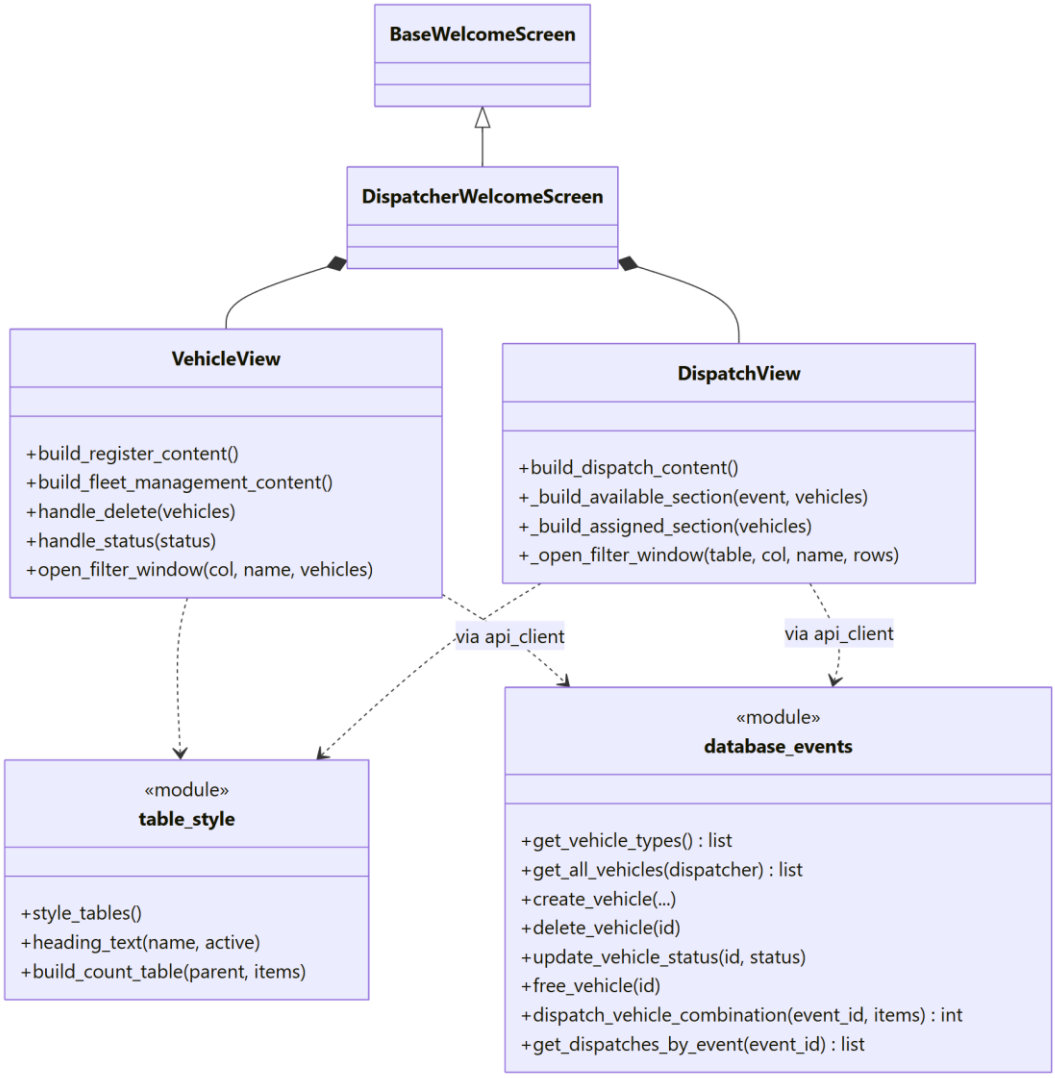


Figure 26: Class diagram of the Fleet and Dispatch component

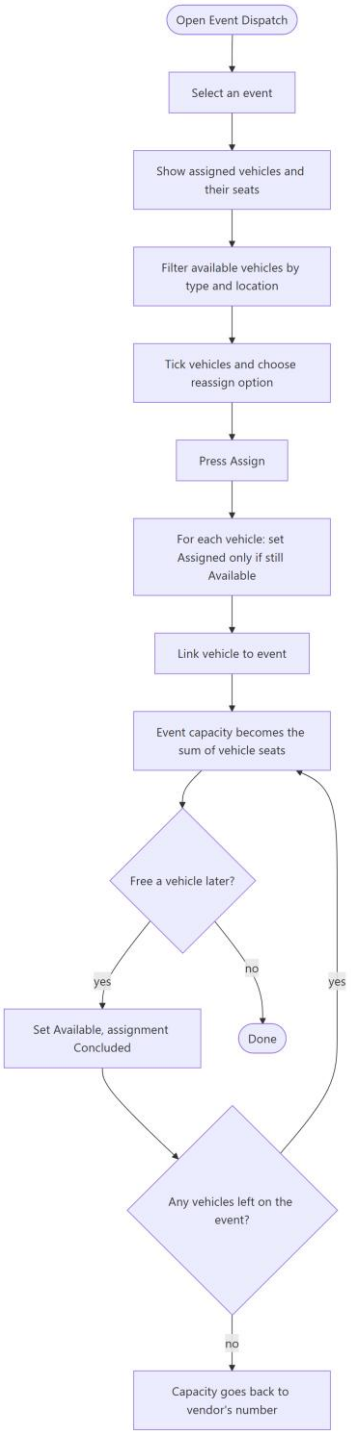


Figure 27: Activity diagram: assigning vehicles to an event

Figure 27: the capacity is never stored separately. It is worked out from the assigned vehicles every time, so it can never be out of date.

3.2.3.2 Restrictions/limitations for component Fleet and Dispatch

This is the weakest part of the server when it comes to security. The vehicle and dispatch endpoints do not check the caller's role or that the vehicle belongs to them. A vehicle is given to an event as a whole, not to one date. The auto_reassign_recurring flag is saved and shown, but nothing uses it yet.

3.2.3.3 Performance issues for component Fleet and Dispatch

Seats are worked out with one grouped query, so it stays quick with a large fleet.

3.2.3.4 Design constraints for component Fleet and Dispatch

Only vehicles that are Available can be assigned. The update has this condition in it, so the same vehicle can never be assigned twice. License numbers are unique.

3.2.3.5 Processing detail for each operation of component Fleet and Dispatch

The component has the operations listed below:

```
+ create_vehicle(license_number, vehicle_type, location, dispatcher_email,
auto_reassign_recurring): None
+ dispatch_vehicle_combination(event_id: int, assignments: list): int
+ update_vehicle_status(vehicle_id: int, status: str): None
+ free_vehicle(vehicle_id: int): None
+ delete_vehicle(vehicle_id: int): None
+ get_all_vehicles(dispatcher_email: str): list
```

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. create_vehicle() is invoked from Register Vehicles.

2. dispatch_vehicle_combination() is invoked when the dispatcher presses Assign. It returns how many vehicles were really assigned.

3. update_vehicle_status() is invoked by Set Maintenance and Set Available.

4. free_vehicle() is invoked from Event Dispatch.

5. delete_vehicle() is invoked from Fleet Maintenance.

6. get_all_vehicles() is invoked by Fleet Maintenance and Event Dispatch.

3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

1. create_vehicle()

```
Start
Save the vehicle as Available for this dispatcher
If the license number exists, the database refuses it
End
```

2. dispatch_vehicle_combination()

```
Start
For each chosen vehicle
   Set it to Assigned only if it is still Available
   If that worked, link it to the event and count it
Return the count
End
```

3. update_vehicle_status()

```
Start
If the new status is not Assigned, end the vehicle's event assignment
Save the new status
End
```

4. free_vehicle()

```
Start
Call update_vehicle_status with Available
End
```

5. delete_vehicle()

```
Start
Remove the vehicle's assignments
Delete the vehicle
End
```

6. get_all_vehicles()

```
Start
Read the dispatcher's vehicles with the seats of their type
If a type is unknown, count 4 seats
End
```

The life of a vehicle is short enough to show in one state diagram (Figure 28).

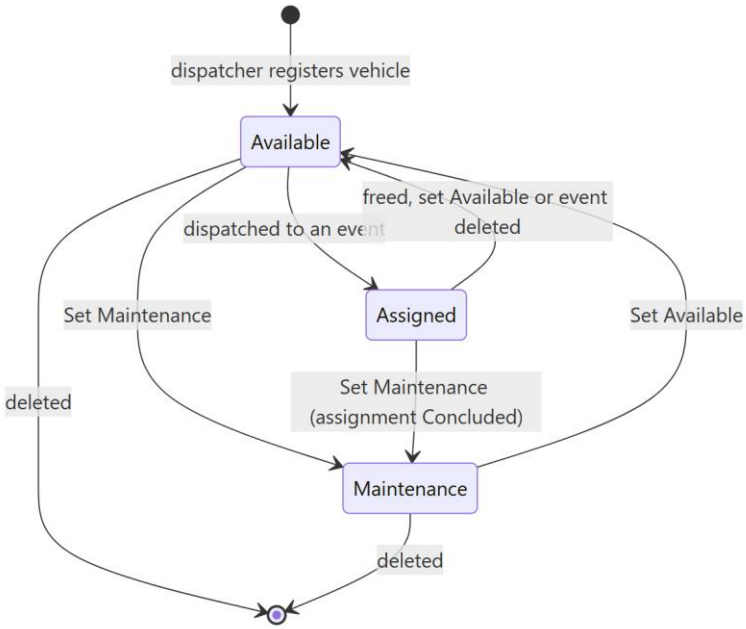


Figure 28: State diagram of a vehicle

3.2 Description for Component “Administration and Reports”

3.2.1 Processing narrative (PSPEC) for component

The admin keeps the platform clean. The User Accounts page lists every account with its role, sign-up date, how old the account is and whether it is banned. For staff it also shows the venue and the manager email. The admin can ban and restore accounts, but not their own account and not another admin.

Every other role has a Report button at the bottom of the side menu. The report form changes with the role, so a customer sees choices like 'Safety concern' while a dispatcher sees 'Vehicle misuse or damage'. Reports reach the admin's inbox, open ones first and the oldest at the top, where the admin can resolve them or ban the offender in one step.

Bans are enforced by the server, not by hiding buttons. A banned user cannot log in, and every request that changes data passes through a guard called _blocked, which refuses unknown and banned accounts.

3.2.2 Component interface description

The admin's home screen has User Accounts, Reports and Bookings. The User Accounts page starts with a small table that counts accounts per role, and the main table can be filtered by column. The Bookings page lists every booking in the system and lets the admin cancel any of them with a reason.

Endpoint	Sends	Gets back
GET /api/admin/users?admin_email=		all accounts
POST /api/admin/users/ban	admin_email, email, banned	success or reason
POST /api/reports	reporter, category, offender, summary, details	report id
GET /api/admin/reports?admin_email=		reports, open first
POST /api/admin/reports/<id>/resolve	admin_email, resolution	success

Table 24: Server endpoints used by "Administration and Reports"

3.2.3 Component processing detail

- Start
- User presses Report and fills in the form
- System saves the report (at most 5 open per user)
- Admin opens Reports and reads the oldest open report
- Admin resolves it or bans the offender
- System blocks the banned account from logging in and from changing data
- End

3.2.3.1 Design Class hierarchy for component Administration and Reports

AdminWelcomeScreen builds the three admin pages, and ReportView builds the report form from a table of choices per role called REPORT_OPTIONS (Figure 29). The guard _blocked sits in server.py and uses active_role from database_core.

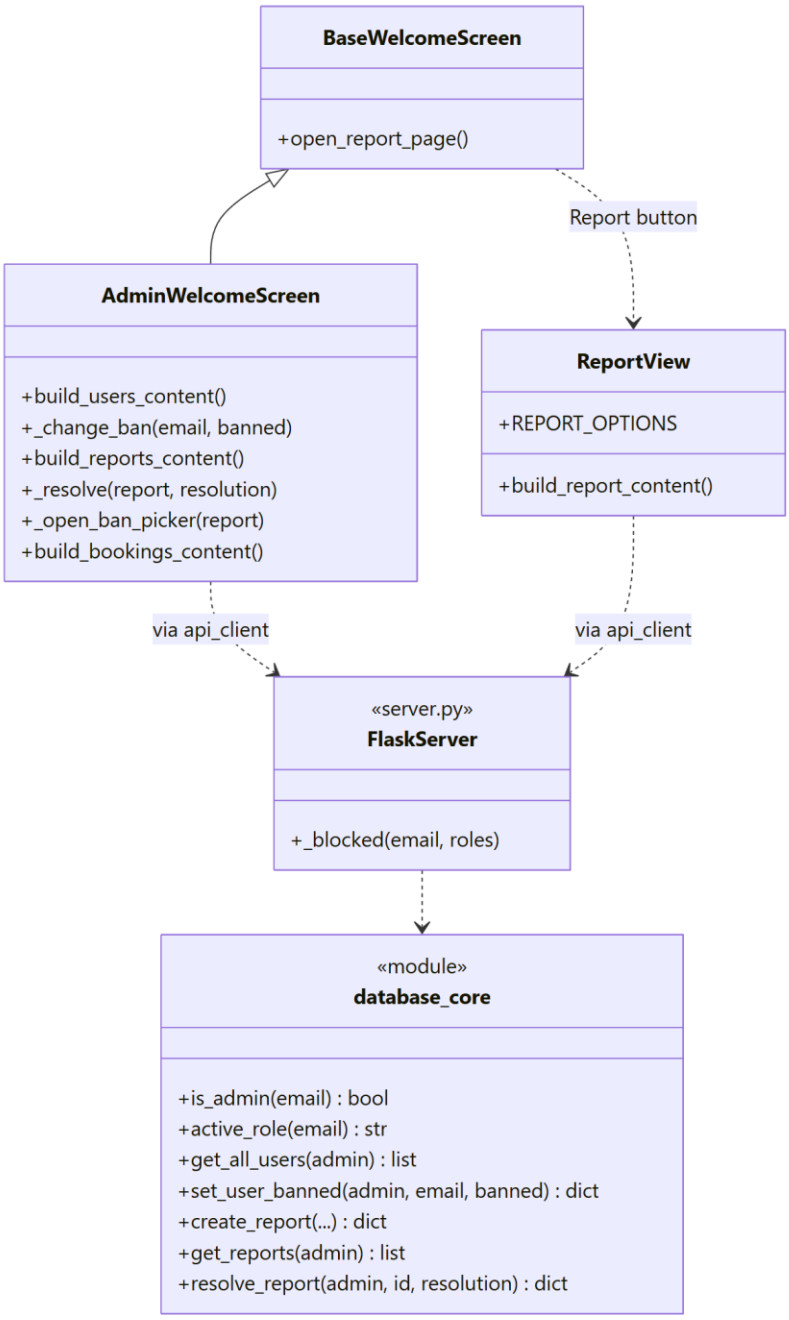


Figure 29: Class diagram of the Administration and Reports component

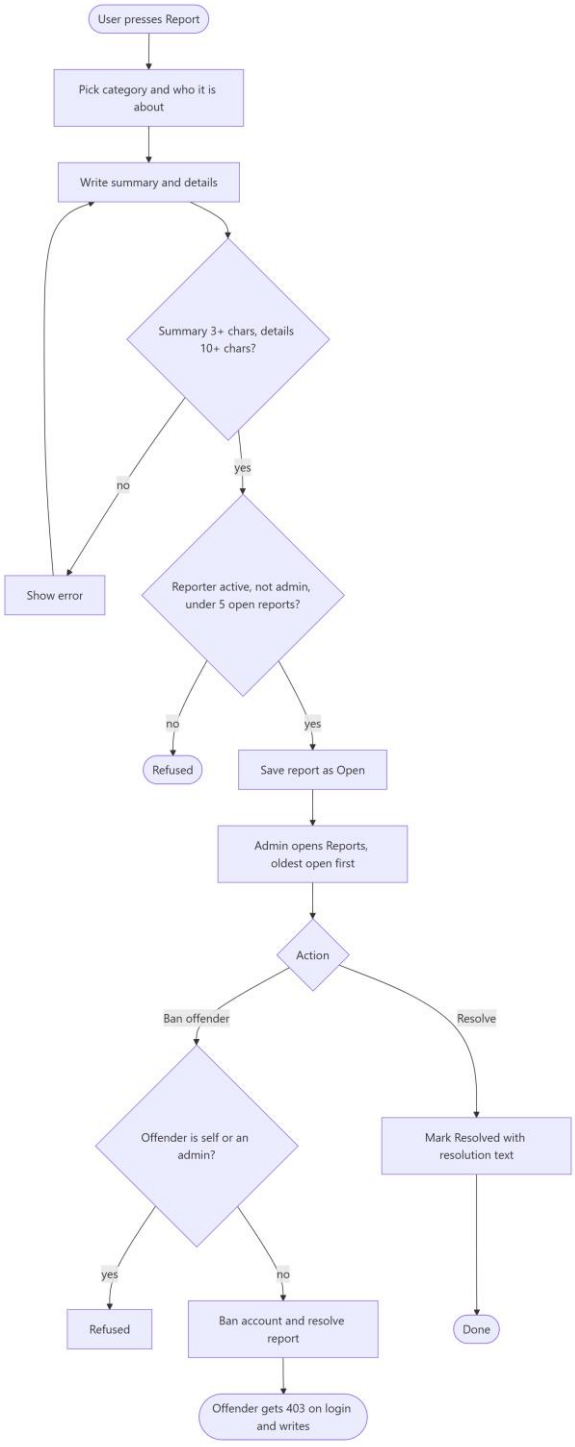


Figure 30: Activity diagram: from a report to a ban

Figure 30: banning from a report also resolves the report, so the admin does not have to do two steps.

3.2.3.2 Restrictions/limitations for component Administration and Reports

Admin requests carry the admin's email. The server checks that this email belongs to an active admin, but without tokens it cannot prove who sent the request. We explain this more in section 5.

3.2.3.3 Performance issues for component Administration and Reports

The user list and the report list are each read with a single query.

3.2.3.4 Design constraints for component Administration and Reports

Admins cannot file reports, and nobody can report their own account. A summary needs 3 characters and the details 10. Admin accounts can only come from the demo accounts, never from sign-up.

3.2.3.5 Processing detail for each operation of component Administration and Reports

The component has the operations listed below:

```
+ is_admin(email: str): bool
+ active_role(email: str): str or None
- _blocked(email: str, roles): response or None
+ get_all_users(admin_email: str): list or None
+ set_user_banned(admin_email: str, email: str, banned: bool): dict
+ create_report(reporter_email, category, offender_type, subject,
description, offender_email): dict
+ resolve_report(admin_email: str, report_id: int, resolution: str): dict
```

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. is_admin() is invoked at the start of every admin function.

2. active_role() is invoked by _blocked.

3. _blocked() is invoked at the start of every endpoint that changes data.

4. get_all_users() is invoked by User Accounts.

5. set_user_banned() is invoked by Ban and Restore in User Accounts and by Ban offender in Reports.

6. create_report() is invoked by Submit Report.

7. resolve_report() is invoked by Resolve and after Ban offender.

3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

1. is_admin()

```
Start
Return true if an account with this email has role Admin and is not banned
End
```

2. active_role()

```
Start
Return the role of the account if it exists and is not banned, otherwise
none
End
```

3. _blocked()

```
Start
Find the role with active_role
If there is no such account, answer 403
If roles are given and the role is not one of them, answer 403
Otherwise let the request continue
End
```

4. get_all_users()

```
Start
If the caller is not an admin, return none
Read every account, newest sign-up first, with venue names for staff
End
```

5. set_user_banned()

```
Start
If the caller is not an active admin, refuse
If the admin is banning themself or another admin, refuse
Save banned, the time and the admin's email (or clear them to restore)
End
```

6. create_report()

```
Start
Check category, summary and details
If the reporter is unknown, banned or an admin, refuse
```

```
If the reporter names their own email, refuse
If they already have 5 open reports, refuse
Save the report as Open
End
```

7. resolve_report()

```
Start
If the caller is not an admin, refuse
Mark the report Resolved with the admin, the time and the resolution text
End
```

3.2 Description for Component “Reviews and Support Tickets”

3.2.1 Processing narrative (PSPEC) for component

This component lets customers talk back. After an experience, a customer opens My Experiences, which lists bookings that were checked in or whose date has passed, and gives a rating from 0 to 5 in half steps with an optional comment. Rating the same booking again updates the first review instead of adding a second one. The explore page shows the average for each event, rounded to one decimal.

Customers can also send a support question about an event. The question goes to the staff of the venue where that event happens, who can mark it resolved.

3.2.2 Component interface description

My Experiences shows past trips as cards with the current rating. A small window opens for the stars and the comment. The Support page has an event list, a subject and a message box, and below it the customer's own tickets with their status.

Endpoint	Sends	Gets back
GET /api/experiences/customer/<email>		past bookings with their review
POST /api/reviews	customer_email, booking_id, rating, comment	thank you message
GET /api/events/ratings		average and count per event
POST /api/support	customer_email, event_title, subject, message	ticket id
GET /api/support/customer/<email>		the customer's tickets

Table 25: Server endpoints used by “Reviews and Support Tickets”

3.2.3 Component processing detail

- Start
- Customer opens My Experiences or Support
- For a review, customer picks a past booking and gives a rating
- System checks the rating and that the booking was attended, then saves it
- For support, customer picks an event and writes the question
- System checks the event and the limit of 3 open tickets, then saves it
- Staff of that venue see the ticket and resolve it
- End

3.2.3.1 Design Class hierarchy for component Reviews and Support Tickets

Both parts live in CustomerWelcomeScreen on the client, while the staff side is in StaffWelcomeScreen (Figure 31). On the server, reviews are in database_events and support tickets in database_core.

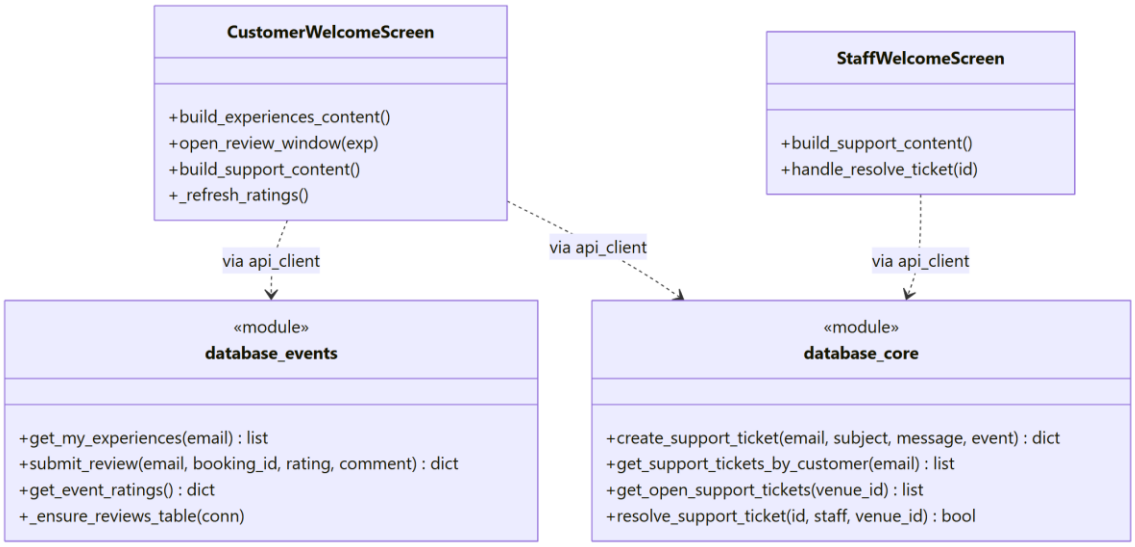


Figure 31: Class diagram of the Reviews and Support Tickets component

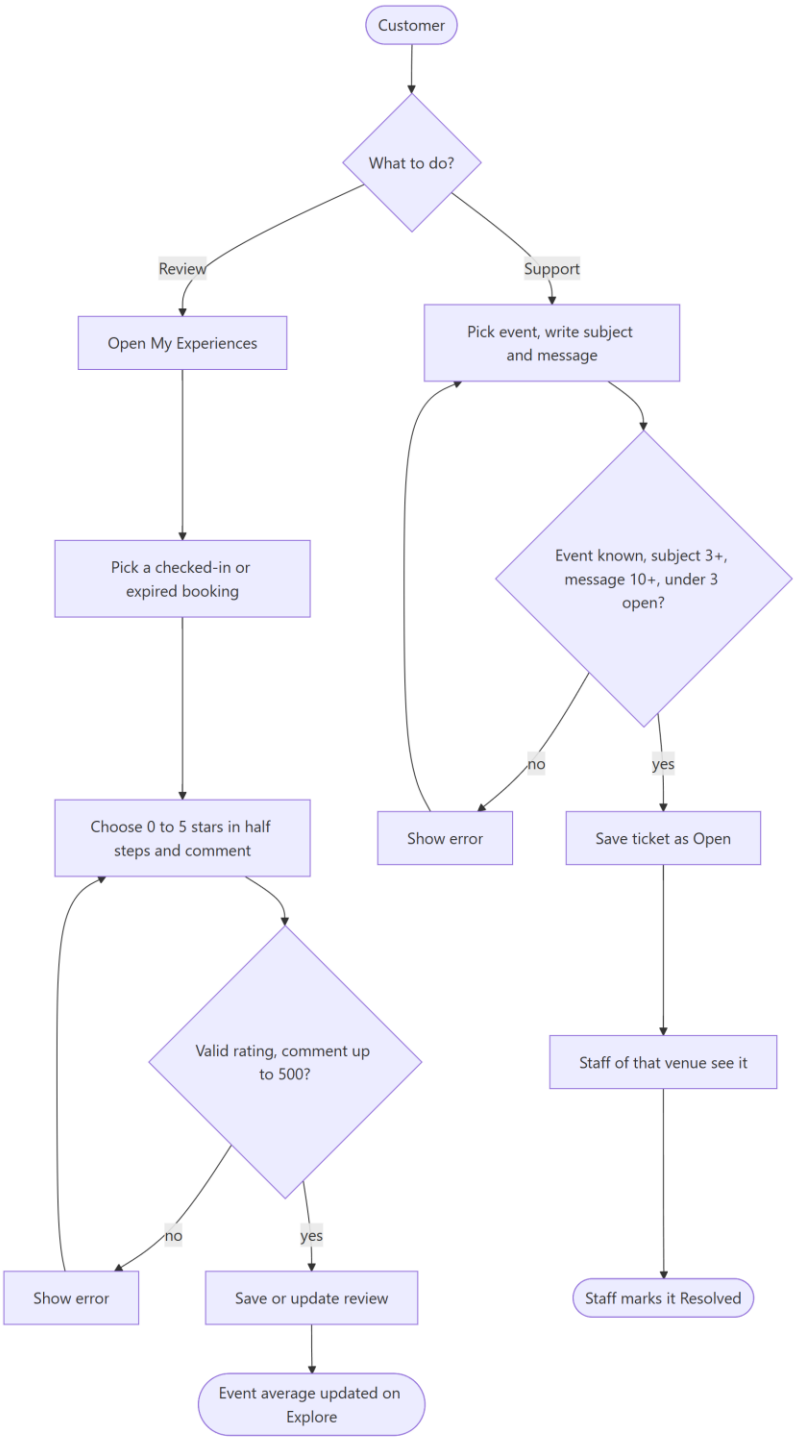


Figure 32: Activity diagram: reviews and support tickets

Figure 32: both paths check the input twice, once in the form and once on the server, and both end with something another person can see: the event's average or a ticket in the staff list.

3.2.3.2 Restrictions/limitations for component Reviews and Support Tickets

Staff can mark a ticket resolved but cannot write an answer inside the app. The reviews table is created the first time it is used, which is why the seeder has to create it on purpose.

3.2.3.3 Performance issues for component Reviews and Support Tickets

Ratings for all events are worked out with one GROUP BY query.

3.2.3.4 Design constraints for component Reviews and Support Tickets

A support ticket must name an event that is listed or that the customer booked before. The subject needs 3 characters and the message 10. A review comment can have 500 characters.

3.2.3.5 Processing detail for each operation of component Reviews and Support Tickets

The component has the operations listed below:

```
+ get_my_experiences(customer_email: str): list
+ submit_review(customer_email, booking_id, rating, comment): dict
+ get_event_ratings(): dict
+ create_support_ticket(customer_email, subject, message, event_title):
dict
+ get_support_tickets_by_customer(customer_email: str): list
```

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. get_my_experiences() is invoked when My Experiences opens.

2. submit_review() is invoked from the review window.

3. get_event_ratings() is invoked when the explore page loads.

4. create_support_ticket() is invoked by Send on the Support page.

5. get_support_tickets_by_customer() is invoked by the Support page to show the customer's own tickets.

3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

1. get_my_experiences()

```
Start
Take the customer's bookings that are Checked in or Expired, newest first
Add the rating and comment of each one if it was reviewed
End
```

2. submit_review()

```
Start
Check the rating is 0 to 5 in steps of 0.5 and the comment is short enough
Check the booking belongs to the user and was attended
Save the review, or update it if one already exists
End
```

3. get_event_ratings()

```
Start
Average the ratings per event
Round to one decimal with .x5 going up
End
```

4. create_support_ticket()

```
Start
Check the event was chosen, subject and message are long enough
Check the event is listed or was booked by the customer
If the customer already has 3 open tickets, refuse
Save the ticket as Open
End
```

5. get_support_tickets_by_customer()

```
Start
Return the customer's tickets, newest first, with their status
End
```

3.3 Dynamic Behavior for Components

This part shows how the classes work together while the program runs. As the template asks, we give a sequence diagram for each main use case. The numbers on the arrows show the order of the calls.

3.3.1 Interaction Diagrams

System sequence diagrams are different from design sequence diagrams. In a system sequence diagram, the whole system is one box talking to the actors. In a design sequence diagram, the modules inside the system talk to each other. The diagrams below are design sequence diagrams: each lifeline is a real module or screen of Rihla, and each message is a real function call or HTTP request. We tried to give one sequence diagram for each main use case.

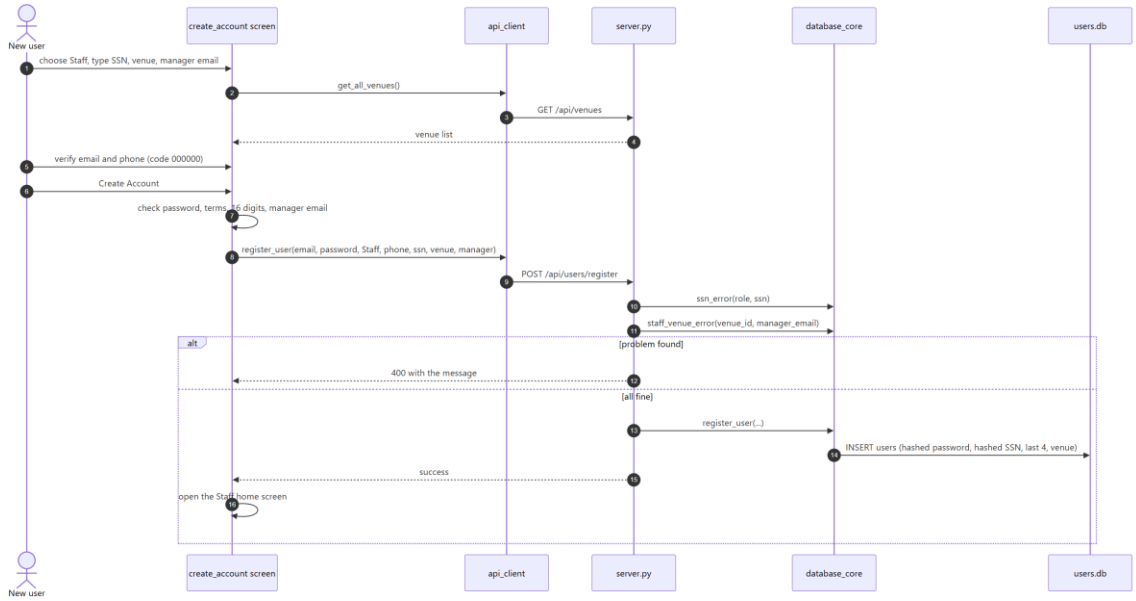


Figure 33: Sequence diagram: sign-up of a staff account

Figure 33: A staff sign-up is the longest version of sign-up, so we show that one. The client loads the venue list first, so the user can only pick a venue that exists. The server checks the SSN and the venue again before it saves anything, and it answers with a message the client shows as it is.

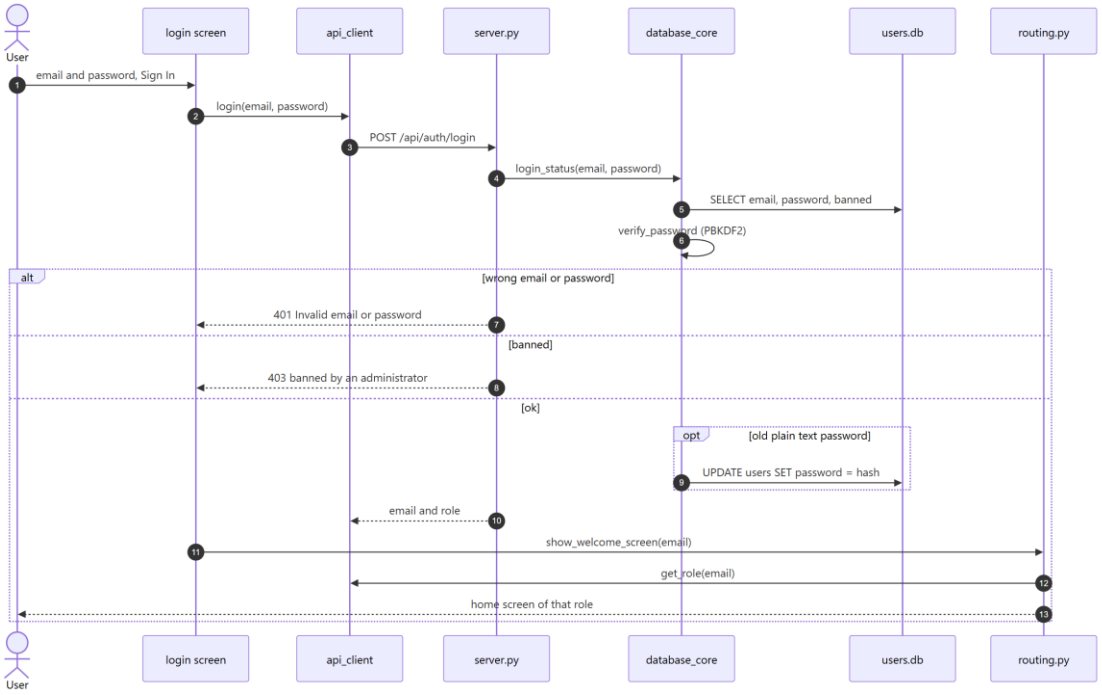


Figure 34: Sequence diagram: login with the ban check

Figure 34: The server tells apart a wrong password (401) and a banned account (403), so the user gets the right message. The client then asks for the role and routing.py opens the matching home screen.

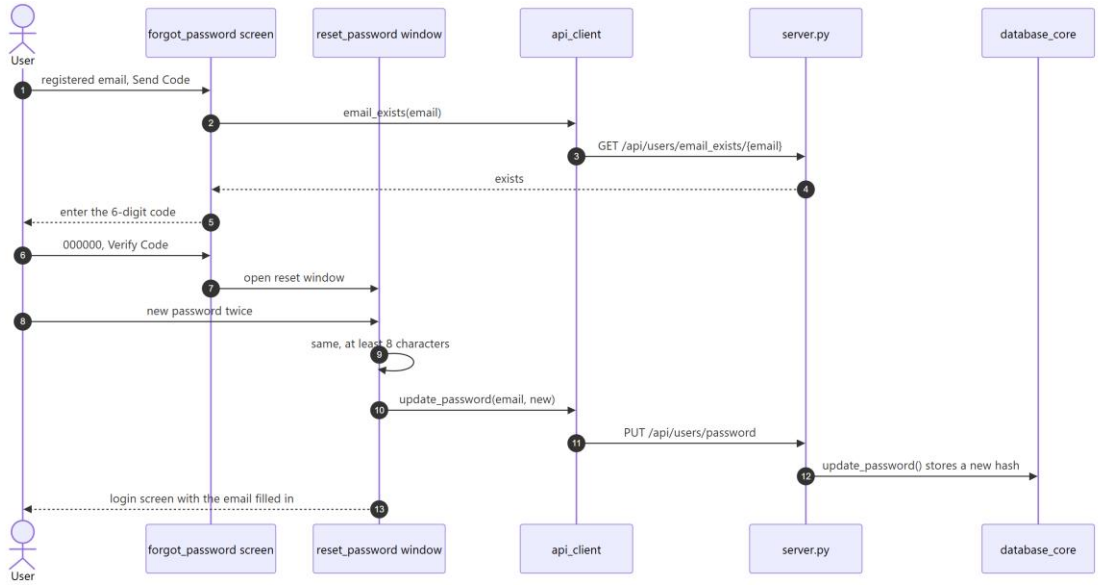


Figure 35: Sequence diagram: forgot password

Figure 35: Only the email check reaches the server before the new password is saved. The code itself is checked on the client and is always 000000, which is one of the limits listed in section 5.

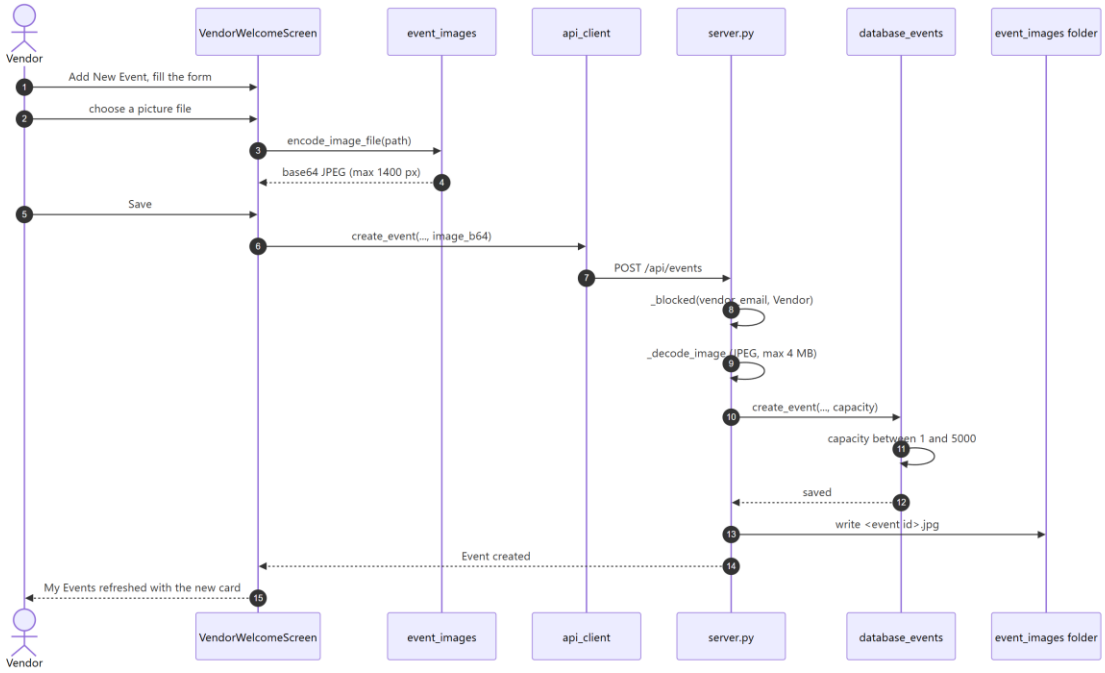


Figure 36: Sequence diagram: a vendor creates an event with a picture

Figure 36: The picture travels inside the JSON as base64. The server checks the role, decodes the picture and only writes the file after the event row was saved, so a bad picture never leaves a half-made event.

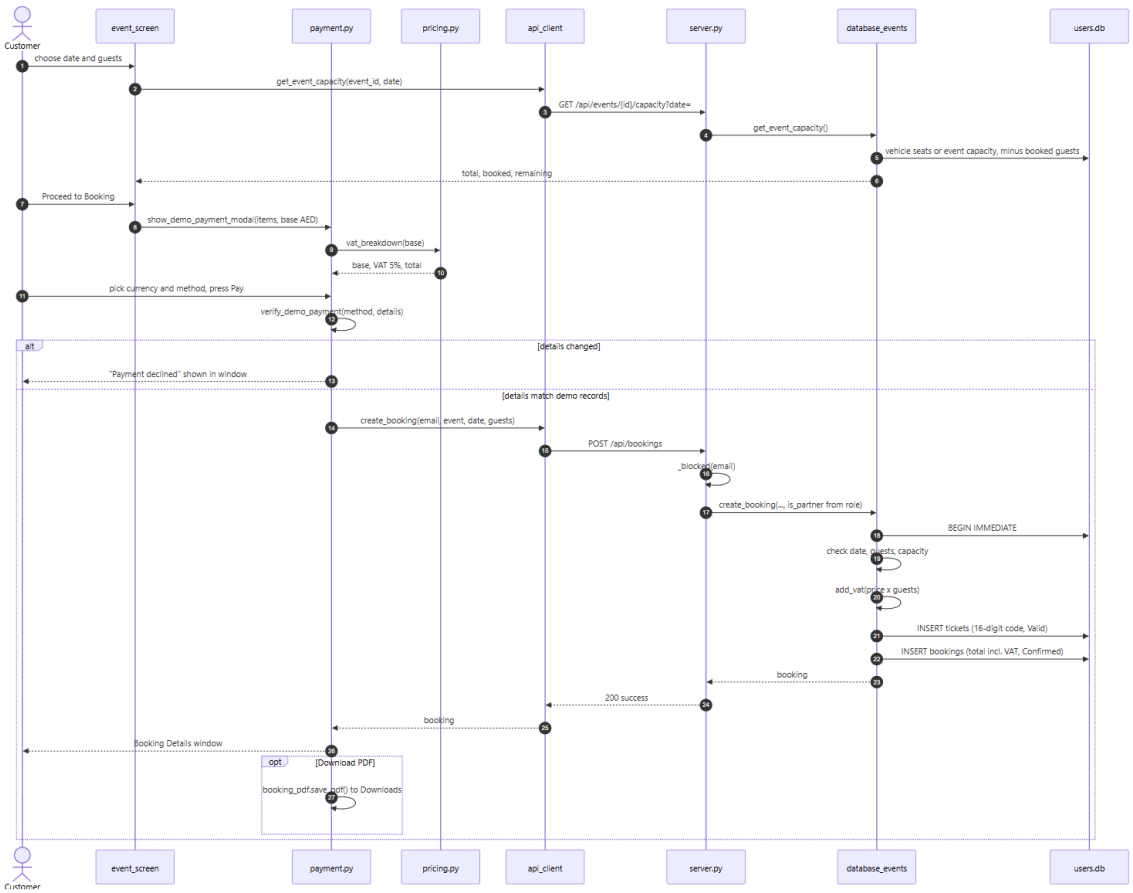


Figure 37: Sequence diagram: a customer books an event and pays

Figure 37: The capacity is read first for the chosen date. The payment window adds VAT and shows the chosen currency. Only after the demo payment is approved does the client send the booking, and the server checks the seats again inside its transaction before it creates the ticket.

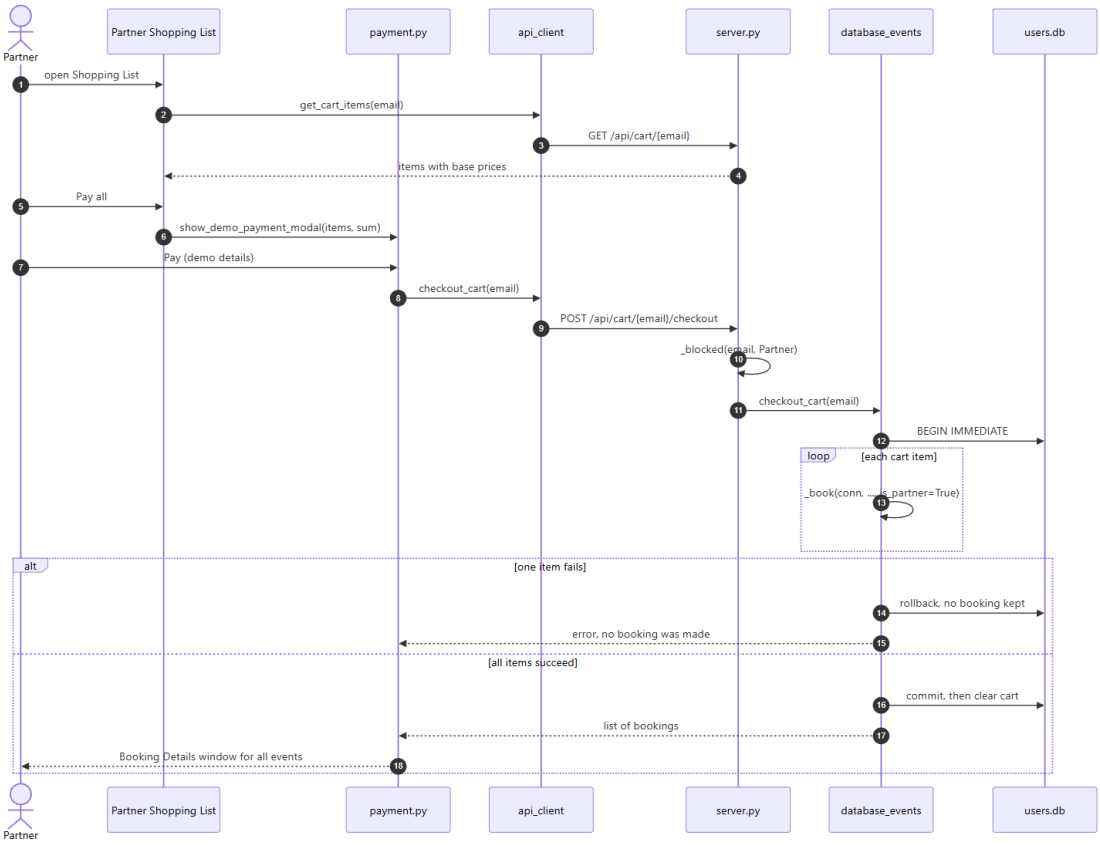


Figure 38: Sequence diagram: a partner pays for the whole shopping list

Figure 38: Every item in the cart is booked inside one transaction. If one of them fails, for example because the event filled up in the meantime, nothing is booked and the cart stays as it was.

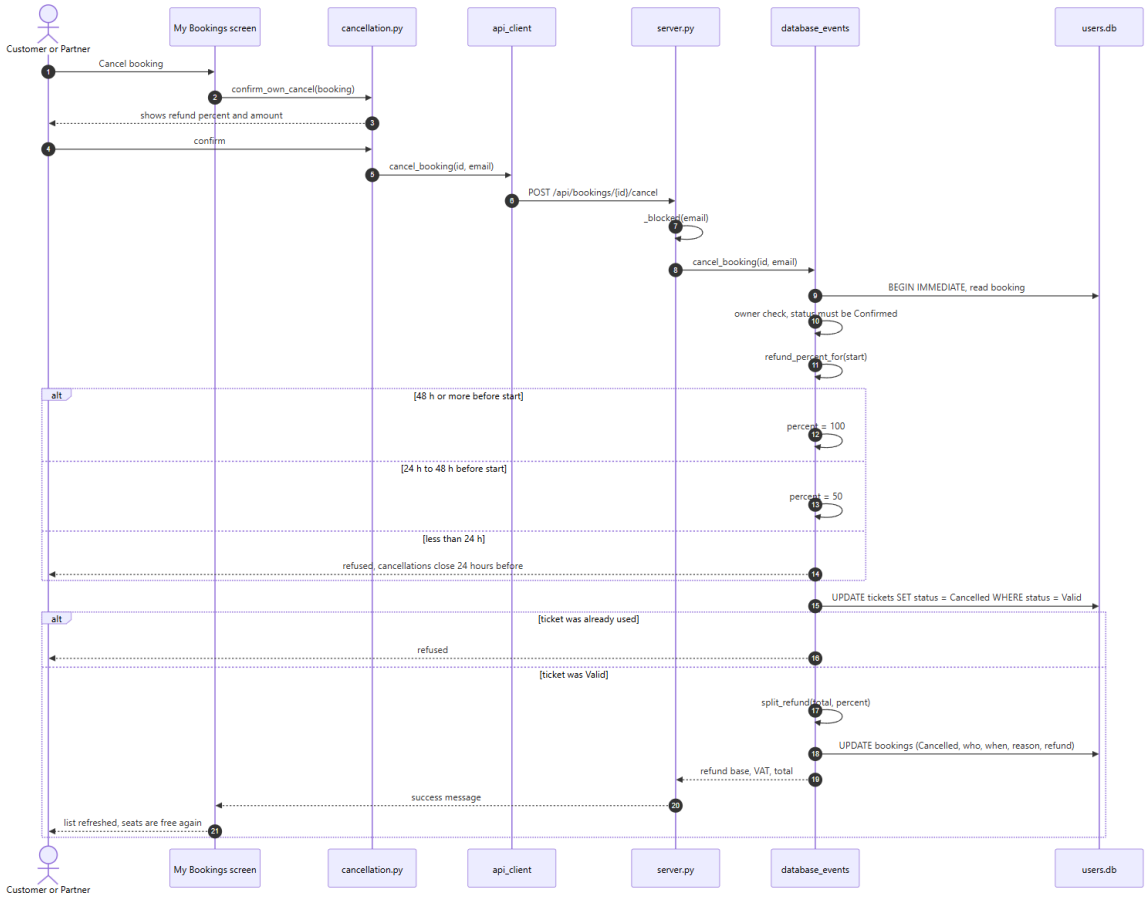


Figure 39: Sequence diagram: a booker cancels and gets a refund

Figure 39: The client shows the refund before the user confirms, but the server decides the real percent again when the request arrives, because time has passed in between.

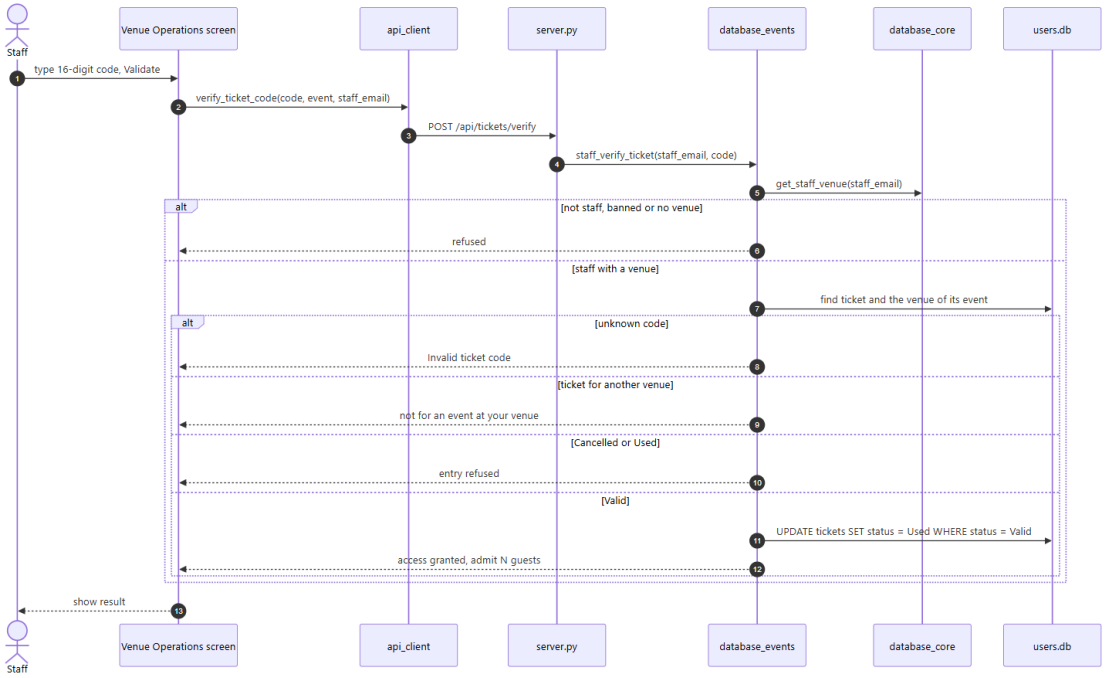


Figure 40: Sequence diagram: staff check a ticket in

Figure 40: The server finds the staff member's venue itself, from the email, so the client cannot choose another venue. A ticket for a different venue is refused without saying anything else about it.

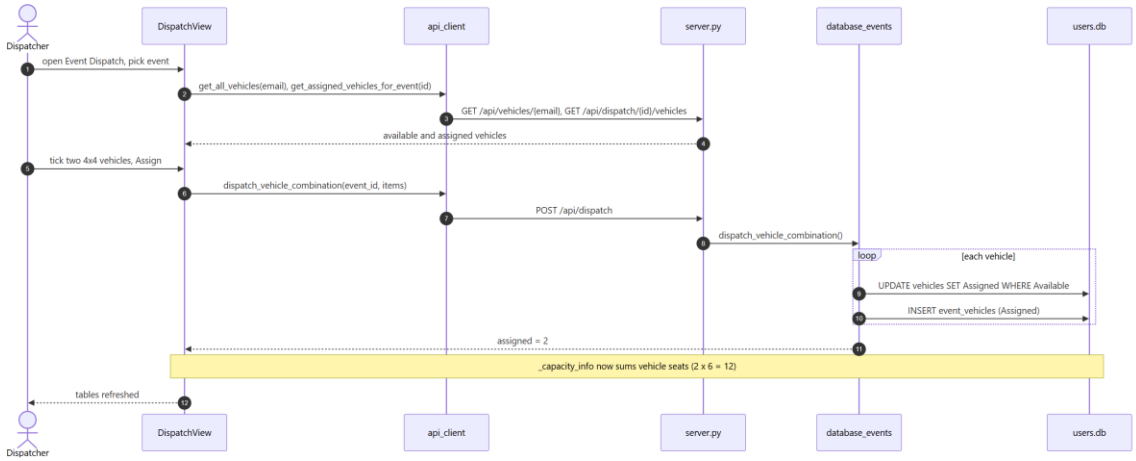


Figure 41: Sequence diagram: a dispatcher assigns vehicles to an event

Figure 41: After the two vehicles are assigned, the next capacity read for this event returns 12 seats, because _capacity_info adds up the seats of the assigned vehicles.

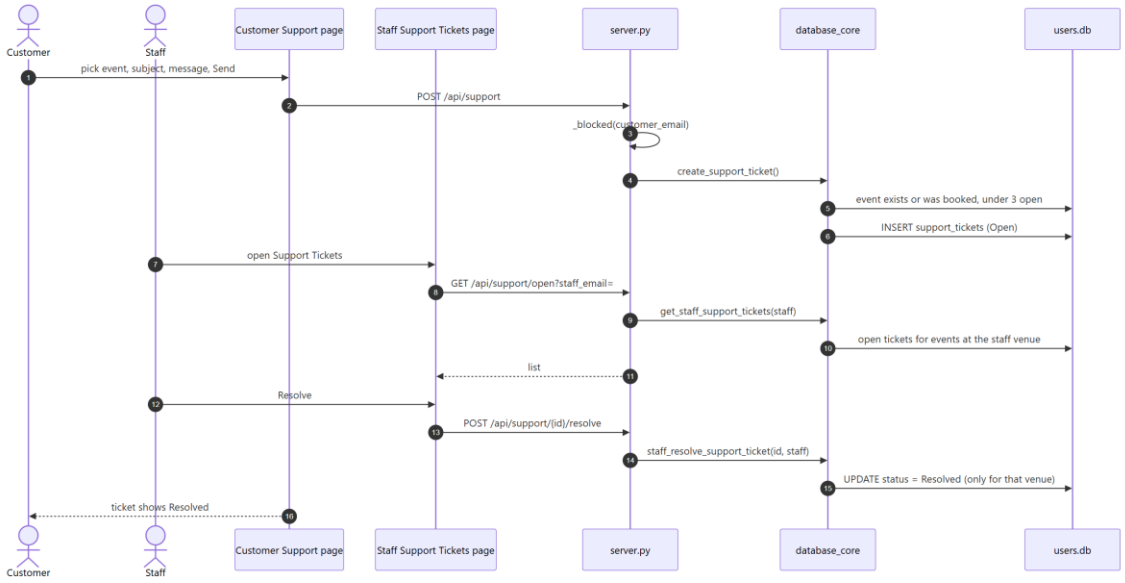


Figure 42: Sequence diagram: a support ticket from customer to venue staff

Figure 42: The ticket is not sent to a person directly. It is stored with its event, and each staff member's list is filtered by the venue of that event when they open the page.

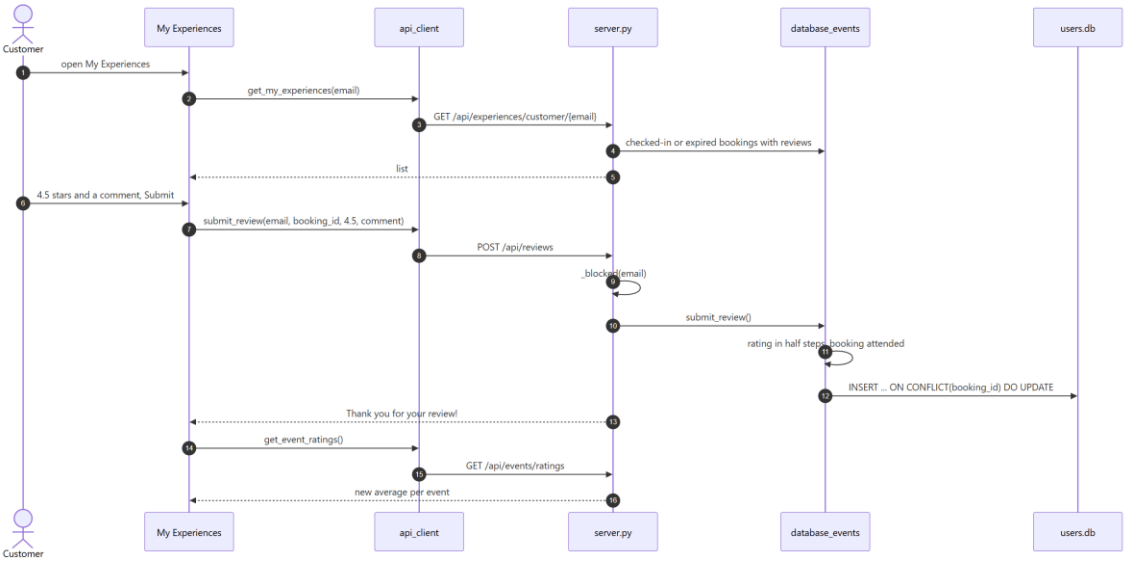


Figure 43: Sequence diagram: a customer reviews a past experience

Figure 43: The insert uses ON CONFLICT on the booking id, so the second review of the same booking updates the first one. The client then reloads the averages for the explore page.

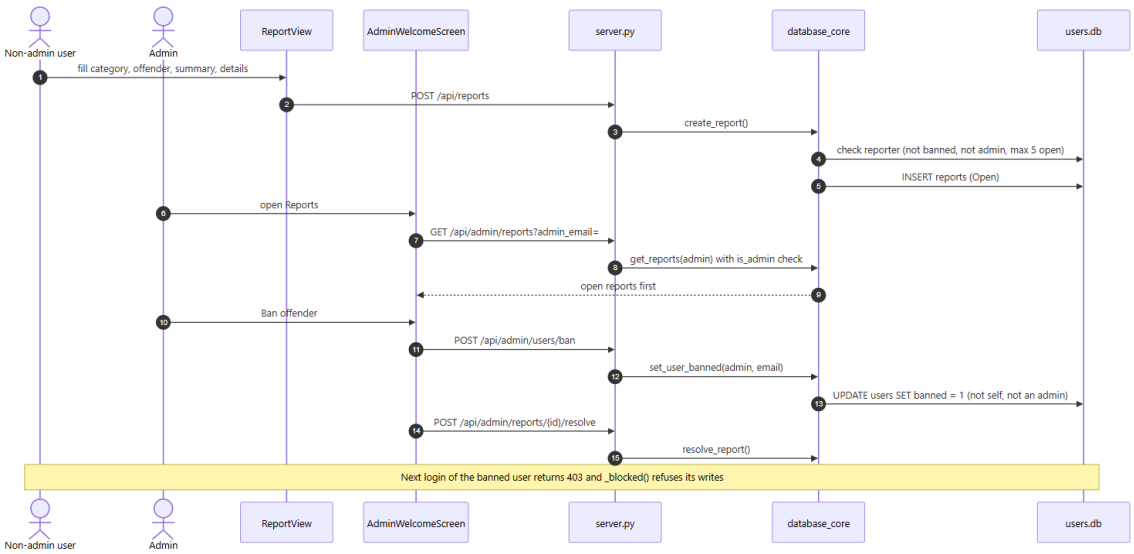


Figure 44: Sequence diagram: a report leads to a ban

Figure 44: After the ban the user cannot log in, and every request that changes data is refused by the _blocked guard in server.py.

3.3.2 State diagram of a booking

A booking moves through a few states (Figure 45). It starts as Confirmed when it is paid for. From there it is checked in by staff, cancelled under the refund rules, or it simply expires when its date passes. Checked in and Expired are not stored in the database. They are worked out from the ticket and the date each time the booking is read.

A sequence diagram covers one use case, while a statechart describes one object across several use cases. The booking is a good example. Its life is touched by booking, check-in, cancellation and reviews, which are four different use cases with four different actors.

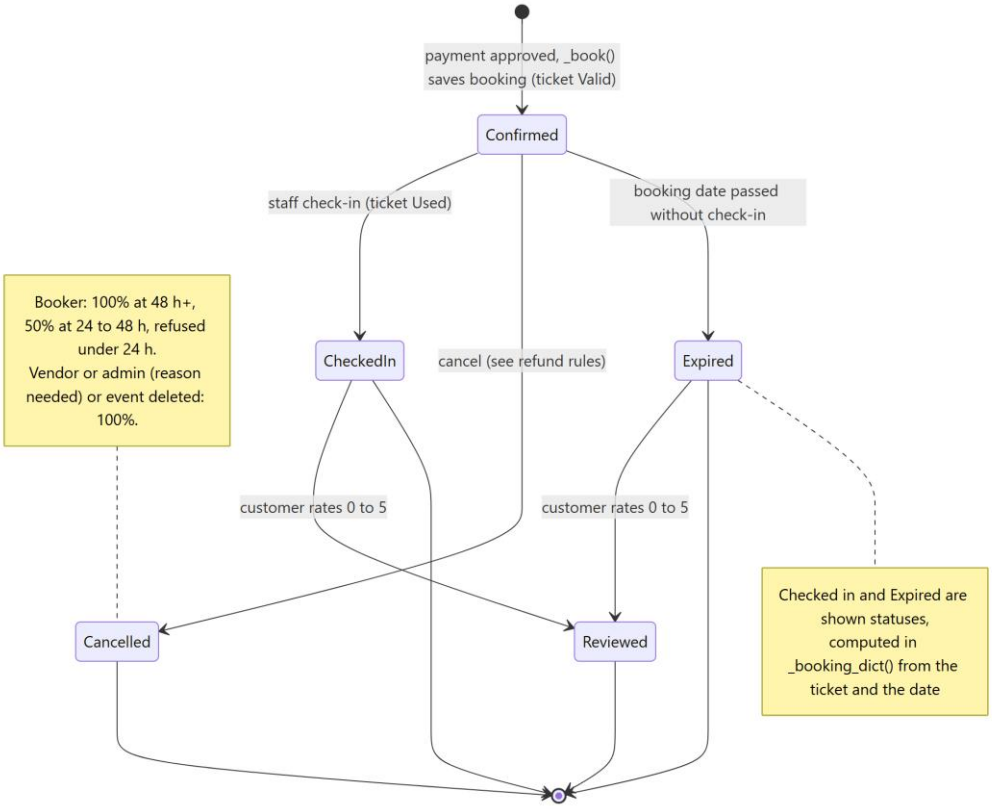


Figure 45: State diagram of a booking and its ticket

Section 4

4.0 User interface design

A description of the user interface design of Rihla is presented in this section.

4.1 Description of the user interface

4.1.1 Screen images

Rihla is a desktop app that opens full screen. We wanted it to feel calm and a little bit like a travel brochure, so the login and sign-up screens are split in two: a photo of Dubai with the Rihla logo on the left and a white form on the right. After login, every role gets the same frame: a dark blue side menu with the role's pages, a white top bar with the signed-in email and a live clock, and a light grey area for the content. Blue buttons are for normal actions and orange ones for the main action of a page, like Proceed to Booking or Pay.

Below are the screens of the app with the demo data loaded. The objects on them and what happens when you use them are described in 4.1.2.

4.1.1.1 Signing in and creating an account

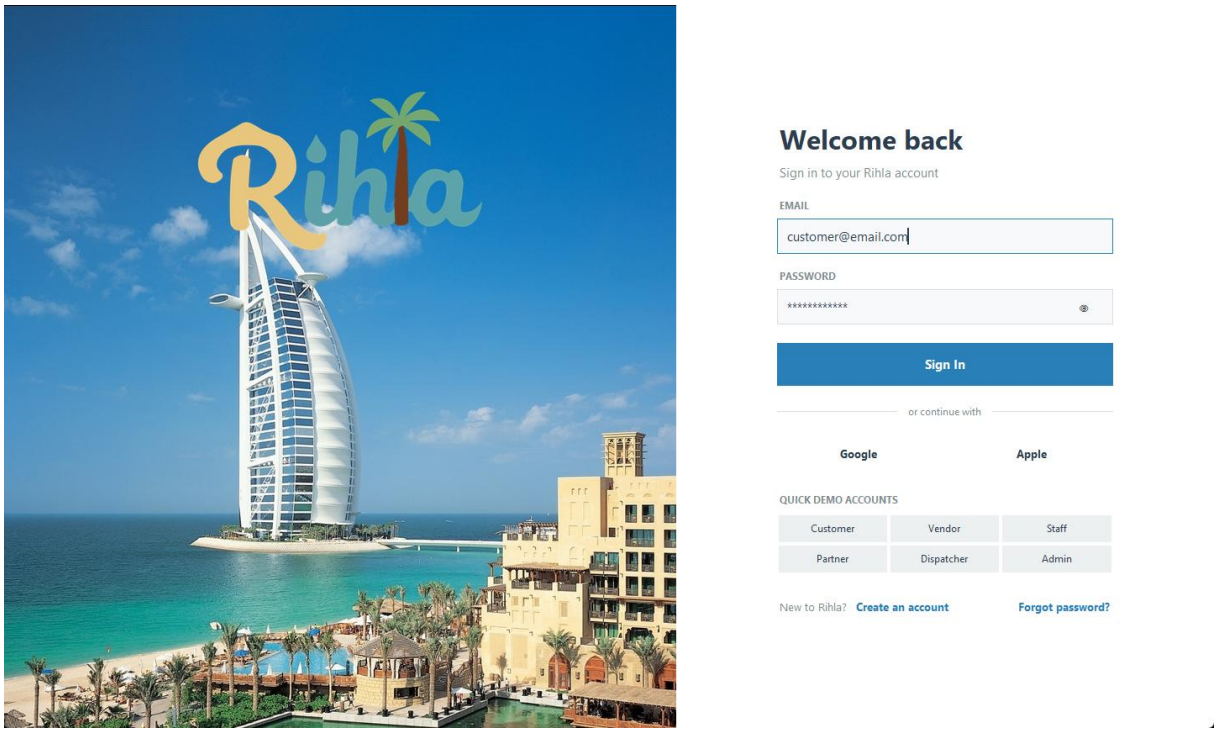


Figure 46: Login screen with Google and Apple buttons and quick demo accounts

The quick demo chips under the login form fill in the email and password of each role. They were meant for testing, but we kept them because they make the demo much faster.

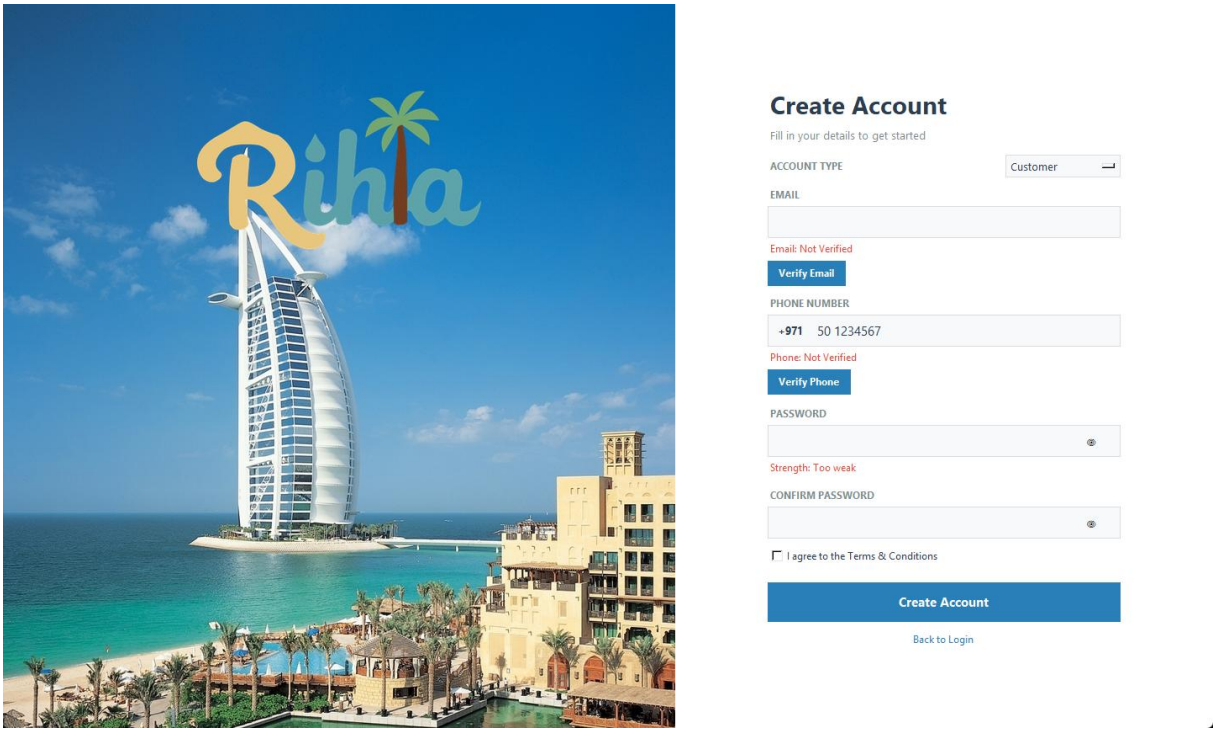


Figure 47: Create Account with the account type picker

4.1.1.2 Customer pages

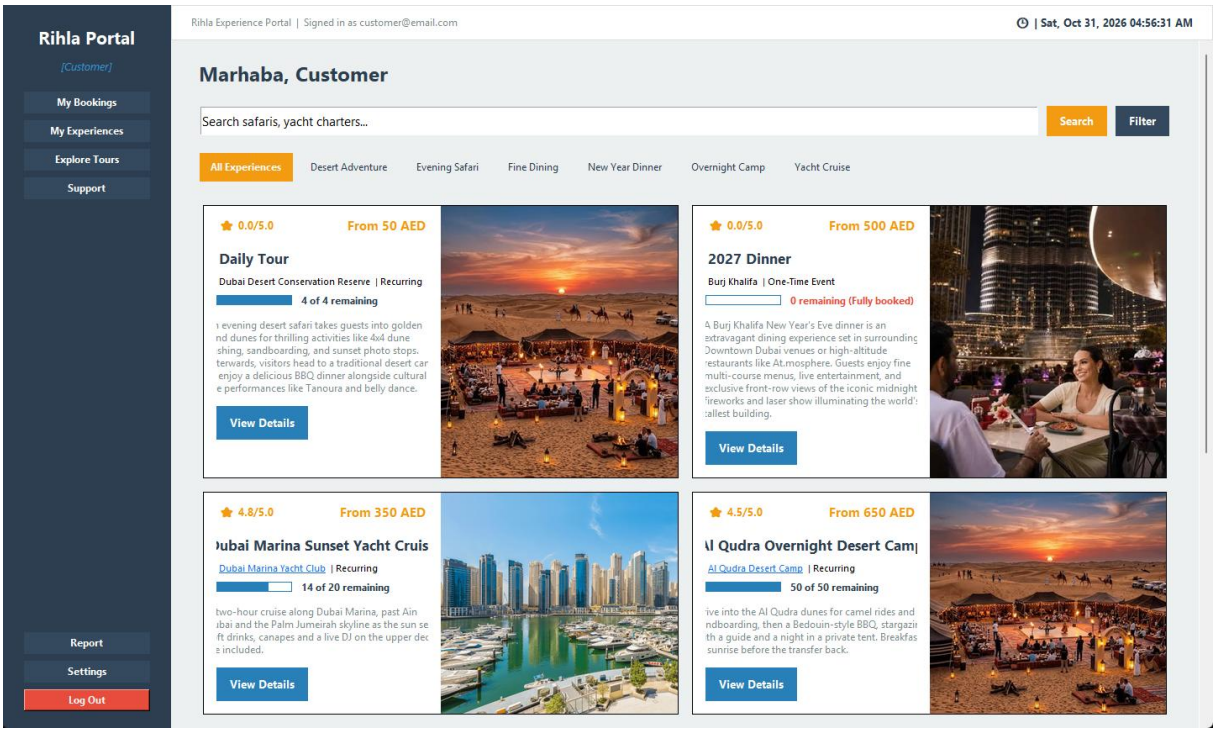


Figure 48: Explore Tours: categories, search, ratings and capacity bars

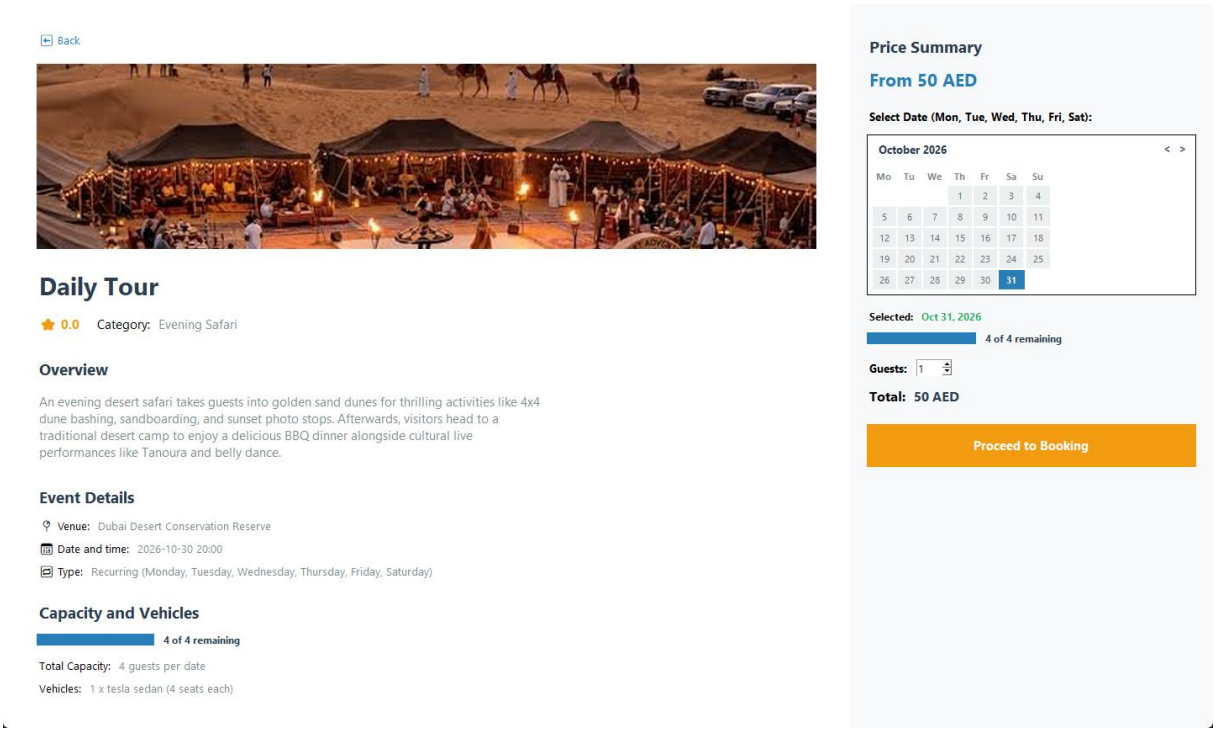


Figure 49: Event page: picture, details, seats, calendar and price summary

On the event page only the dates the event really runs on can be clicked. The capacity bar changes when another date is picked.

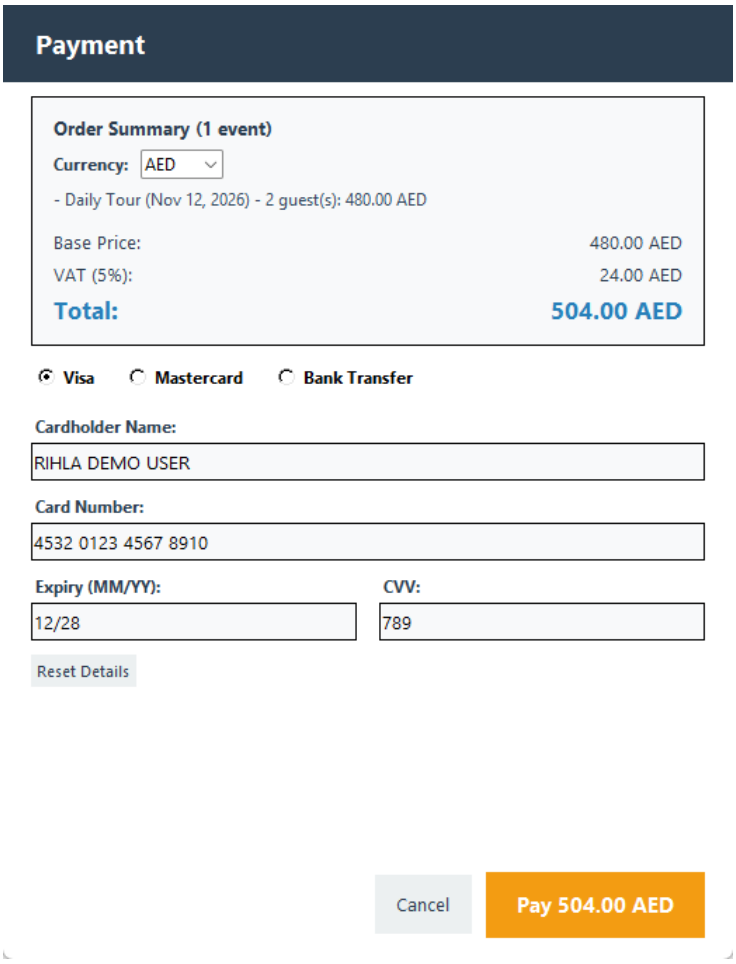


Figure 50: Payment window with currency, VAT line and prefilled demo card

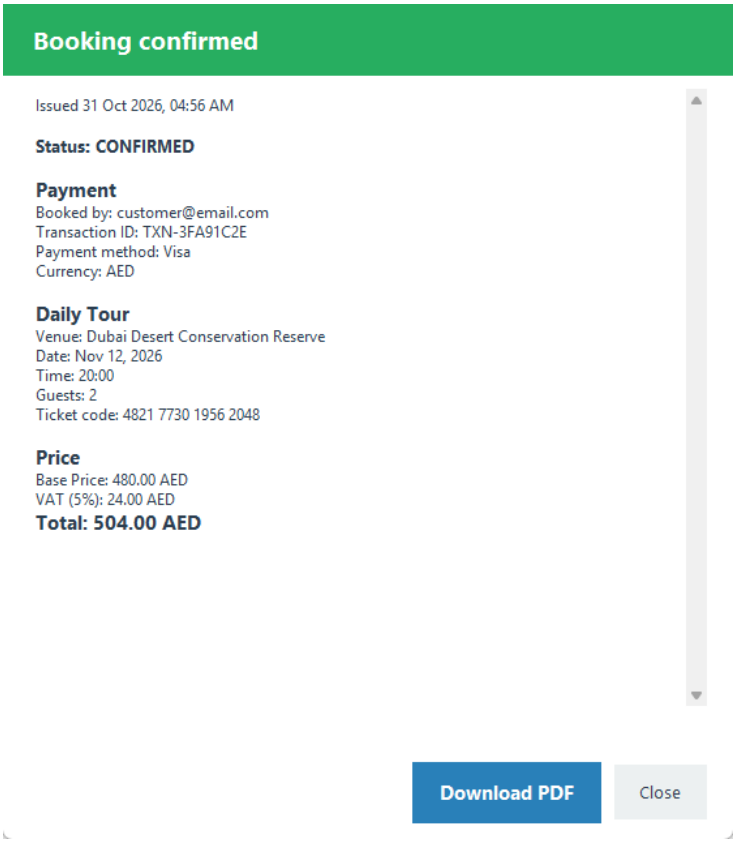


Figure 51: Booking Details after payment, with Download PDF

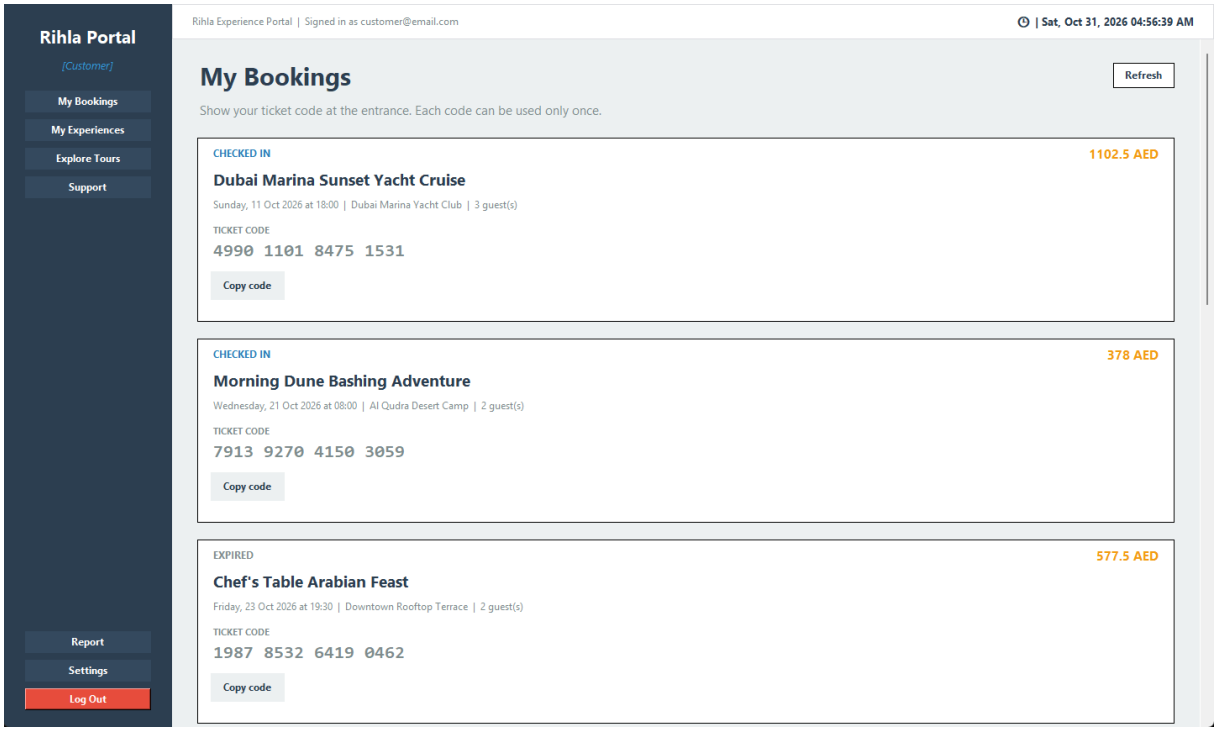


Figure 52: My Bookings with ticket codes and Copy code

4.1.1.3 Vendor pages

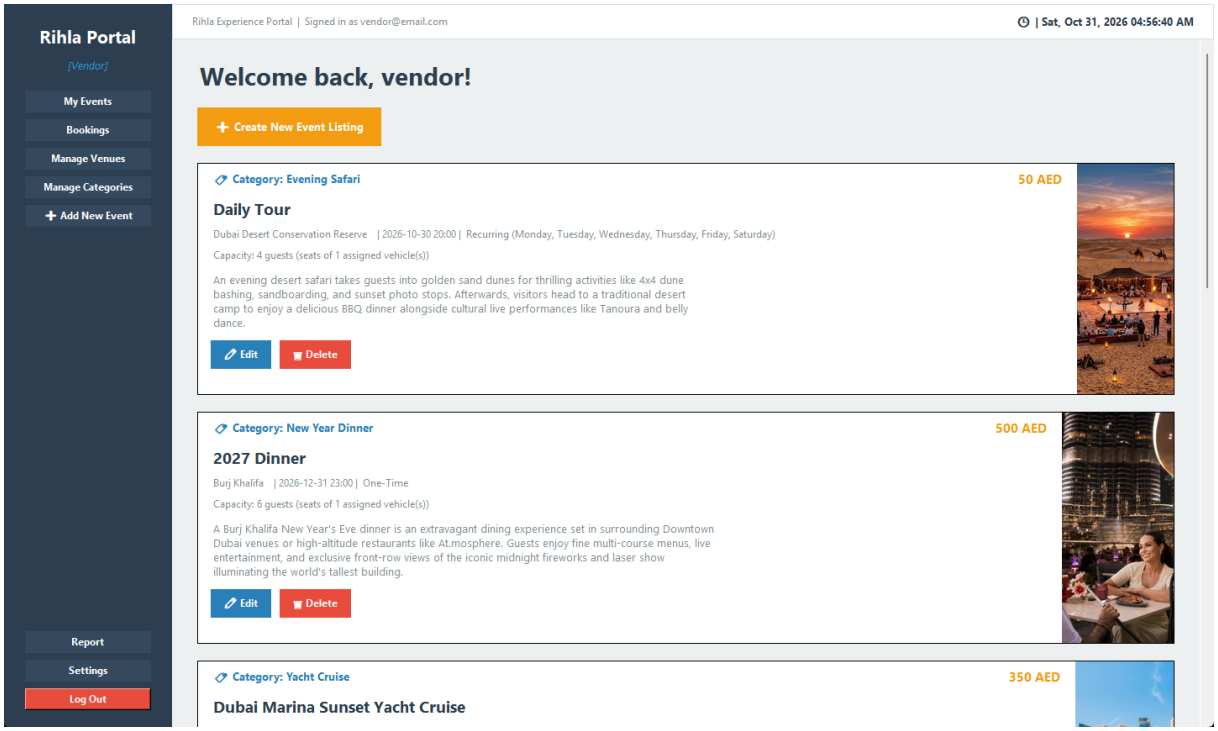


Figure 53: My Events with pictures, Edit and Delete

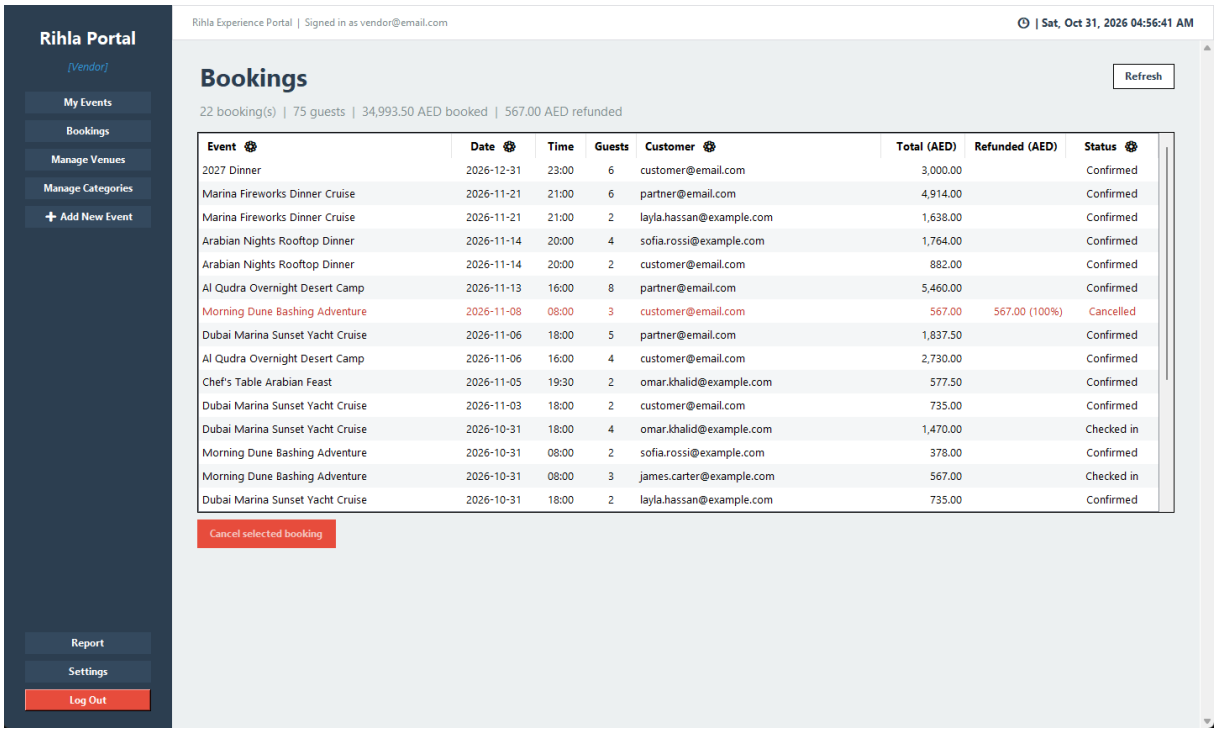


Figure 54: Vendor Bookings table with column filters and a cancelled booking

4.1.1.4 Staff, partner and dispatcher pages

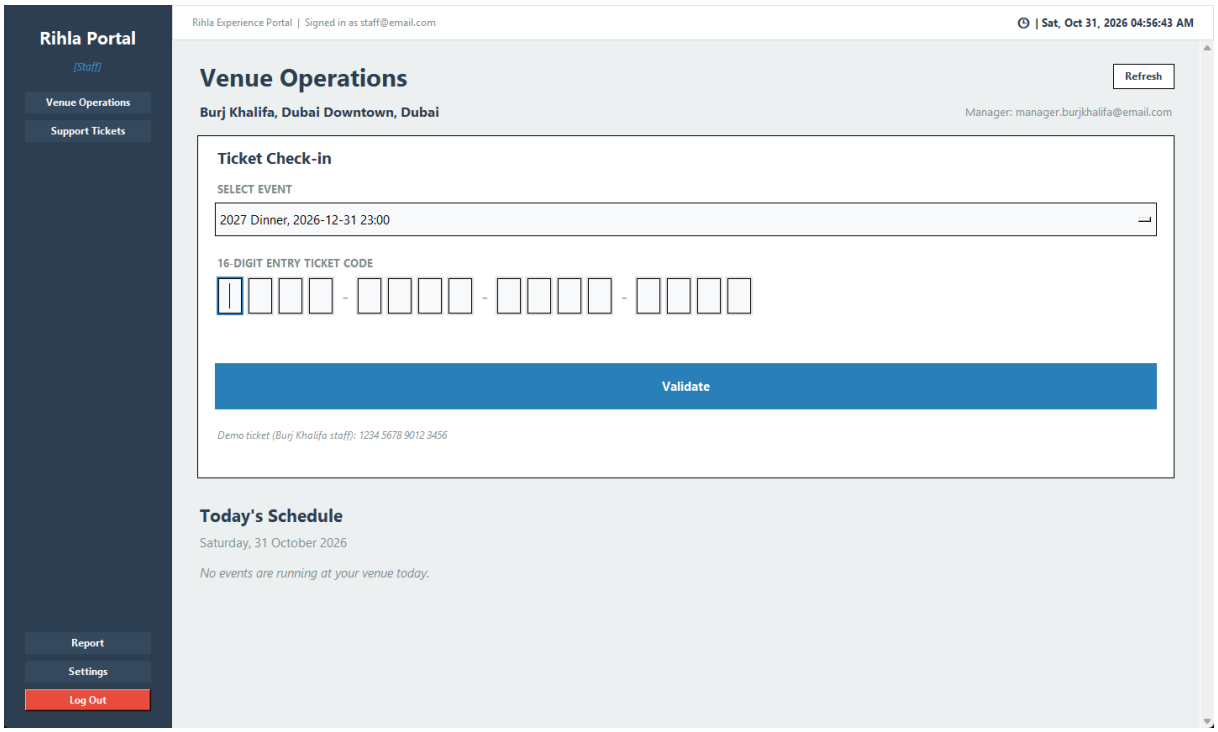


Figure 55: Venue Operations: venue, check-in and today's schedule

In this screenshot the schedule is empty because no event at Burj Khalifa runs on the day it was taken.

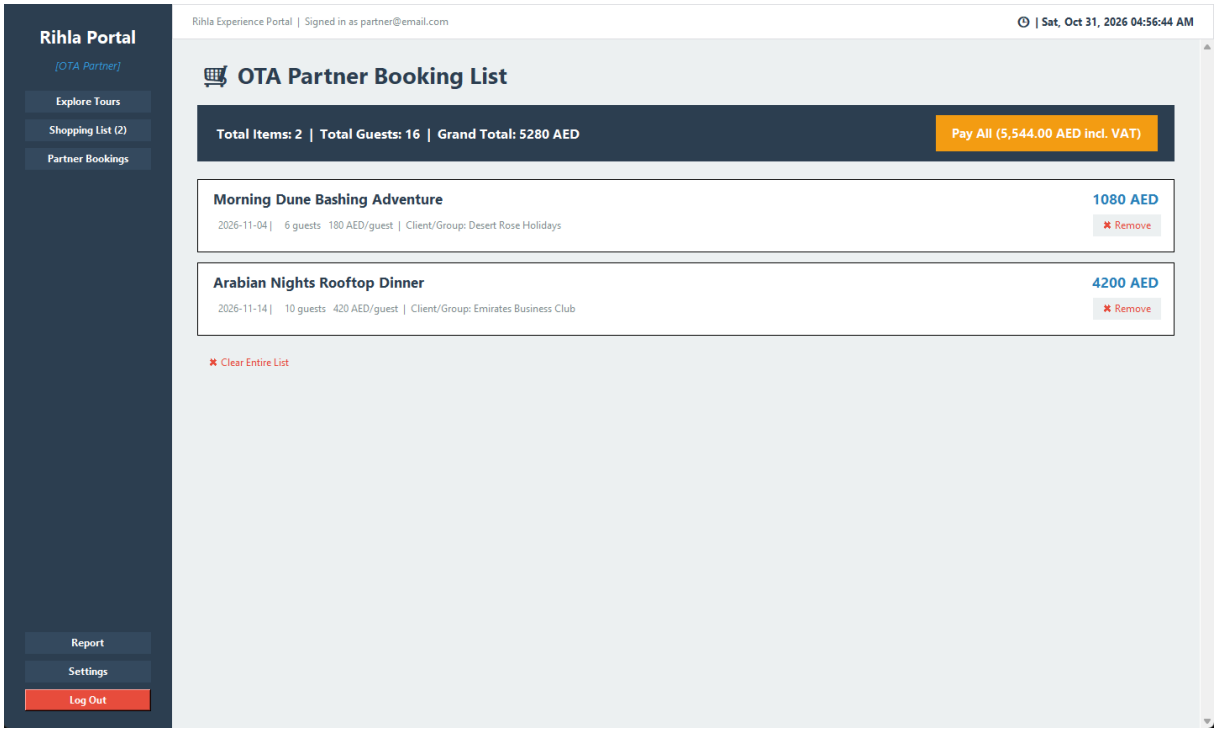


Figure 56: Partner shopping list paid in one go

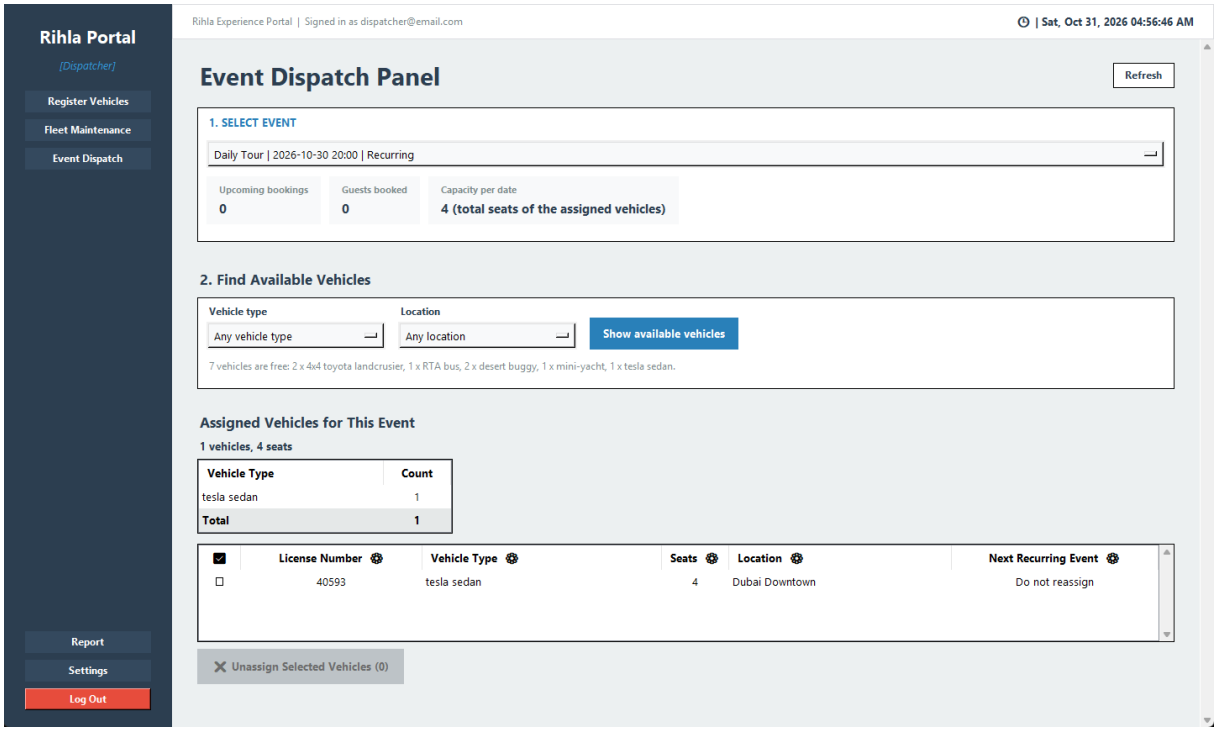


Figure 57: Event Dispatch with available and assigned vehicles

4.1.1.5 Admin pages and reports

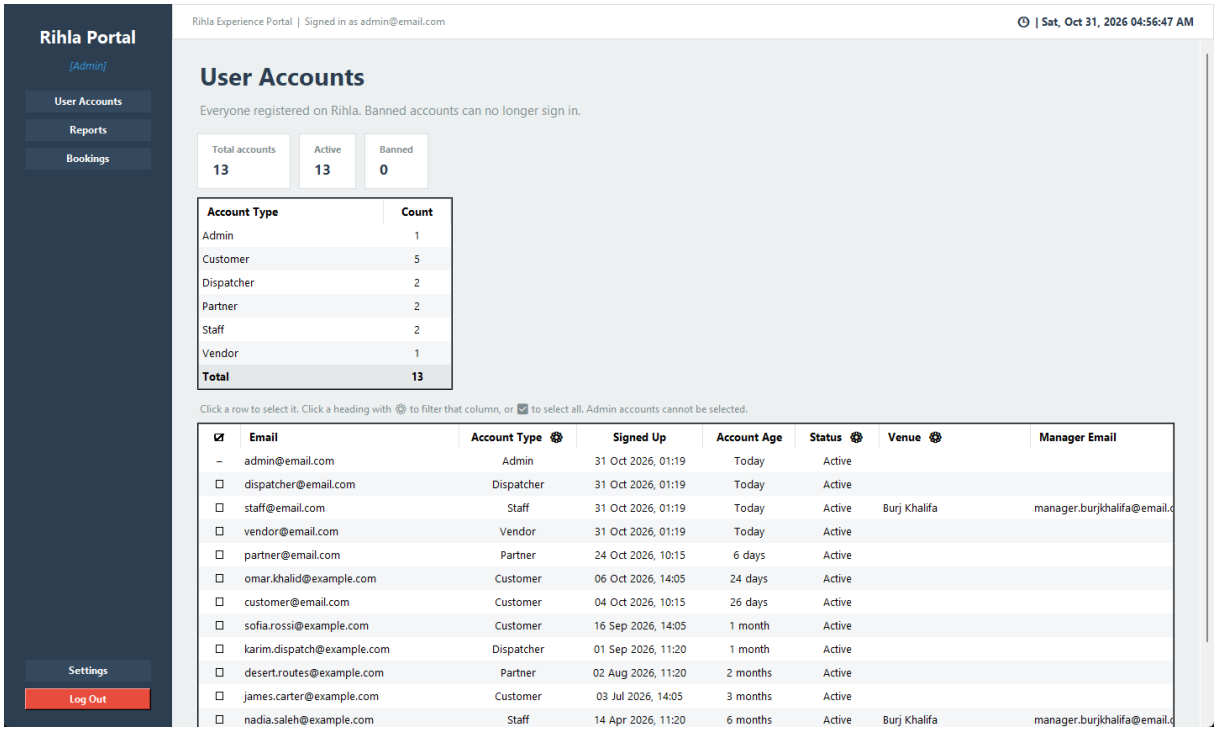


Figure 58: User Accounts with counts per role, sign-up date, age and status

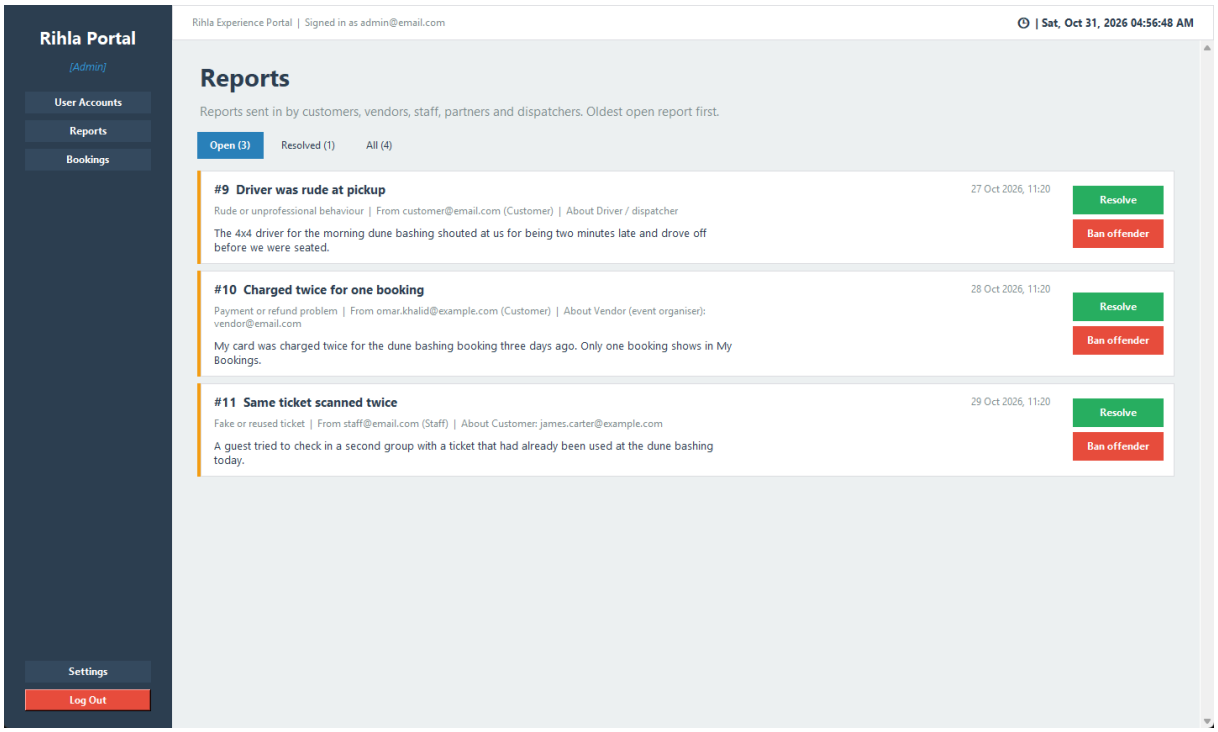


Figure 59: Reports inbox with Resolve and Ban offender

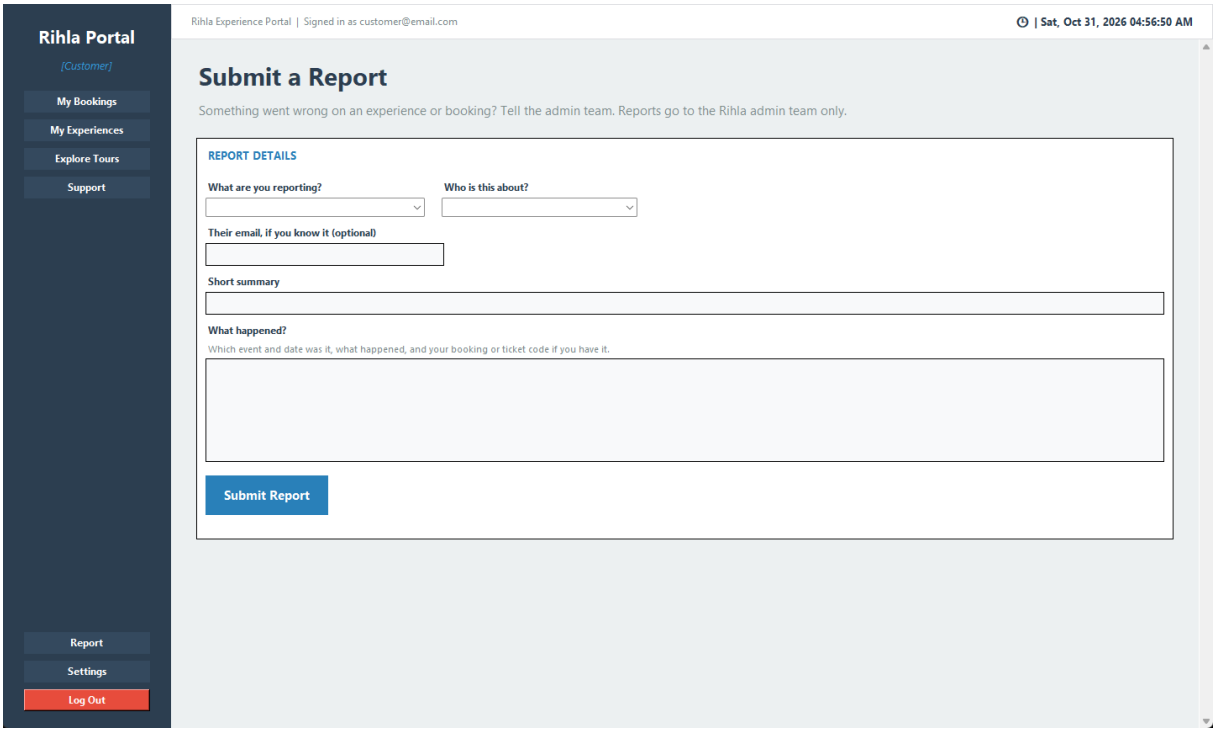


Figure 60: The Report form that every non-admin role can open

4.1.2 Objects and actions

The interface of Rihla is built from a small set of objects that come back on most screens:

- **Side menu:** the role's pages at the top, Report, Settings and Log Out at the bottom. Clicking a page replaces the content area.
- **Top bar:** shows who is signed in and the date and time, updated every second.
- **Event cards:** picture, title, venue, rating, price and a capacity bar. View Details opens the event page.
- **Category chips and search box:** filter the list of events. The Filter window adds a maximum price slider.
- **Calendar:** only allowed dates can be clicked, and the chosen date updates the seats and price.
- **Capacity bar:** a bar with 'N of TOTAL remaining', orange 'Hurry up!' when few seats are left and red 'Fully booked' at zero.
- **Digit boxes:** one box per digit for the SSN and ticket codes. They jump to the next box and accept a pasted code.

- **Payment window:** currency box, base price, VAT and total, payment method buttons and Pay.
- **Tables:** used by vendors, dispatchers and the admin. Columns with a gear icon can be filtered by clicking the heading, and the first column has check boxes.
- **Booking cards:** status, ticket code in groups of four digits, Copy code, the refund policy line and Cancel.
- **Pop-up windows:** for confirmations, the cancel reason, reviews, settings and errors.
- **Refresh buttons:** on the pages that show changes made by other people, such as bookings and dispatch.

4.2 Interface design rules

We followed the eight golden rules of interface design. Below is how each one shows up in Rihla.

1. **Strive for consistency.** All roles share one frame (BaseWelcomeScreen), one colour file (theme.py) and one table style (table_style.py). A table looks the same for a vendor and for the admin.
2. **Seek universal usability.** New users can try the app with the demo chips. Prices can be shown in four currencies, which matters for tourists.
3. **Offer informative feedback.** The capacity bar, the password strength label and messages such as 'Only 3 of 20 seats are left on that date' tell the user what is going on. A declined payment is explained inside the payment window instead of a separate pop-up.
4. **Design dialogs to yield closure.** Every booking ends with the Booking Details window, which sums up the whole booking and offers the PDF.
5. **Prevent errors.** Wrong dates cannot be picked, digit boxes only take digits, and the sign-up form hides the SSN and venue fields when they do not apply. The server still checks everything again.
6. **Permit easy reversal of actions.** Bookings can be cancelled under clear rules, cart items can be removed, banned users can be restored and the payment form has Reset Details.

7. **Keep users in control.** The refund is shown before the user confirms a cancellation, and pages with shared data have a Refresh button.
8. **Reduce short-term memory load.** Ticket codes are split into four groups of four digits and have a Copy code button. The signed-in email is always visible in the top bar.

4.3 Components available

We did not use a design tool for the screens. Everything is built with the standard tkinter library and ttk (Frame, Label, Button, Entry, Text, Canvas, Scrollbar, Scale, Spinbox, Radiobutton, Checkbutton, OptionMenu, Toplevel, Combobox and Treeview), plus Pillow for pictures and the optional tksvg for the Lucide icons. Where tkinter had nothing suitable we wrote small widgets ourselves: DigitBoxes, CapacityBar and the picture cards in event_images.py.

4.4 UIDS description

We did not have a user interface development system. The screens are written by hand in Python. To keep them consistent we rely on shared code instead: theme.py for colours and fonts, table_style.py for tables, navigate.py and routing.py for moving between screens. Adding a new role only needs a new welcome module and one line in routing.py.

Section 5

5.0 Restrictions, limitations, and constraints

This section lists the things that limited our design and the weak points we know about. We prefer to name them here rather than have them found during the demo.

Design constraints and restrictions:

- All rules about seats, money and access must live on the server, not in the screens.
- All money is kept in AED and the 5% VAT is always shown on its own line.
- Sensitive data such as passwords and SSNs is only stored as salted hashes.
- The app must run on a normal Windows laptop with Python 3.12 and no database server to install.
- The demo must be repeatable, so the seed data is relative to today and can be cleared again.

Major factors that hindered the design:

1. **No login tokens.** The server does not hand out a session token after login. Requests carry an email and the server trusts it. It does check that the account exists, is not banned and has the right role, but someone who writes their own HTTP requests could act as another user. This is the biggest gap we would close next.
2. **Missing ownership checks.** The password change endpoint does not ask for the old password. Updating and deleting events, deleting venues, and the vehicle and dispatch endpoints do not check that the record belongs to the caller.
3. **Local only.** The server runs on 127.0.0.1 with plain HTTP and Flask's development server. It is made for one machine and is not ready for the internet.
4. **Demo payments.** Payments only compare the form with fixed details, and refunds are only recorded. A real payment provider would also need the booking to be reserved before the money is taken.

5. **Fixed currency rates.** USD, EUR and GBP use fixed numbers in pricing.py, so the shown prices can be out of date.
6. **Fake verification.** Email, phone and password reset codes are always 000000, and the Google and Apple buttons are ordinary email logins.
7. **SQLite.** One file is easy to set up, but only one writer at a time is allowed. Our transactions keep the data correct, but many users at once would have to wait.
8. **Email as a key.** Many tables point to users by email, so an email address cannot be changed.
9. **Prices as text.** Prices like '240 AED' are stored as text and only the digits are read, so prices with decimals are not possible.
10. **Server time.** All times are the local time of the server machine.
11. **Phase 1 features we did not build.** QR codes, the offline driver app, live vehicle tracking, email and SMS messages and Arabic support.

Section 6

6.0 Testing Issues

Unlike Phase 1, we now have working code, so we used both white-box and black-box testing. The automated tests are in the tests folder and use Python's unittest. They point the database to a temporary file, so the real data is never touched. Most tests call the server functions directly, and some go through the Flask API with app.test_client() to check the role guards. There are 70 tests, and on 31 October 2026 all 70 passed. They run with:

```
python -m unittest discover -s tests
```

Next to the automated tests, we went through the demo script in README.md role by role after filling the database with dummy_data.ps1.

We separate verification ('are we building the product right?') from validation ('are we building the right product?'). Most of our automated tests are verification: they check the code against the rules we wrote down, such as the refund steps. The demo script is closer to validation, because it walks through each role's real tasks the way a user would.

We also followed the usual testing strategy, which moves from testing in the small to testing in the large:

- **Unit testing:** single functions such as refund_percent_for, split_refund and ssn_error are called directly with chosen inputs.
- **Integration testing:** ApiAccountGuardTests and StaffVenueTests go through the Flask app with app.test_client(), so the endpoint, the guard and the data function are tested together. This is close to use-based testing, where the classes of one use case are integrated.
- **System testing:** the full app with the server, the client and the seeded database, following the demo script.
- **Acceptance testing:** the final demo in front of the instructor is our acceptance test. We have not done it yet at the time of writing.

- **Regression testing:** after adding a feature we ran the whole suite again, so the old behaviour was checked too. The suite grew to 70 tests this way.
- **Static testing:** before submitting we read through the code again to find mistakes (the 'Final audit' commit in our history). This is the review and walkthrough kind of static testing.

One fault from our own history shows the difference between an error, a fault and a failure. While cleaning up the interface, we removed some helper labels (the error). Two places in the code still used those labels (the fault). Nothing happened until a user opened those screens, which then crashed (the failure). The final audit found and fixed both. It taught us to check for names that are used but no longer defined after any interface change.

6.1 Classes of tests

1. Unit testing (white box)

Expectations: Each rule should work on its own, including at its limits.

Example: refund_percent_for gives 100 at exactly 48 hours, 50 between 48 and 24, and nothing below 24 (CancellationTests). Two vehicles of the same type add their seats together (CapacityTests).

2. Functional testing (black box)

Expectations: Every function should do what the user expects, judged only by the result.

Example: Booking 3 guests for a 240 AED event stores 756 AED and a 16-digit code (BookingTests).

3. Transaction testing

Expectations: A group of actions should either all happen or not happen at all.

Example: A partner cart with one full event books nothing and keeps the cart (test_failed_cart_checkout_keeps_no_bookings).

4. Security testing

Expectations: Only the right, active accounts should be able to act.

Example: A banned or unknown account cannot book or open support tickets, and only partners can use the cart (ApiAccountGuardTests). Staff cannot check in another venue's ticket (StaffVenueTests).

5. Upgrade testing

Expectations: Databases made by older versions of our code should still work.

Example: Tests build an old database without the new columns and check that init_db adds them (for example capacity, ban columns, staff venues).

6. Interface testing

Expectations: Small widgets should show the right numbers.

Example: The capacity bar shows 'N of TOTAL remaining' with the right ratio, and the demo payment fails as soon as a field is changed (PartnerAndPaymentTests).

6.2 Expected software response

Expected response through equivalence partitioning:

Restriction	User	Valid classes	Invalid classes
Customer books 1 to 10 guests	Customer	1, 5, 10	0, 11
Partner has no 10 guest limit	Partner	11, 25 (if seats are left)	more than the seats left
Booking date is today or later and fits the event	Customer, Partner	Tuesday for a Tue/Thu event	yesterday, a Monday
SSN is 16 digits, not one digit repeated	Vendor, Staff, Partner, Dispatcher	4000100020003001	15 digits, 1111111111111111, letters
Password has at least 8 characters	All	customer1234	abc123
Rating is 0 to 5 in steps of 0.5	Customer	0, 3.5, 5	4.3, 5.5, -1
Event capacity is 1 to 5000	Vendor	1, 30, 5000	0, 5001
Cancel reason up to 300 characters, required for operators	Vendor, Admin	a short reason	empty, 301 characters

Table 26: Equivalence classes

Expected response for the main test cases:

Test case	Input	Expected response
Booking price	3 guests, 240 AED event	Base 720, VAT 36, total 756 AED and a 16-digit code
Full event	More guests than seats left	Refused with the seats left and the total
Vehicle seats	Two 4x4 Land Cruisers assigned	Capacity becomes 12
Check-in twice	Same valid code twice	First: access granted. Second: already used

Test case	Input	Expected response
Other venue	Al Qudra staff, Burj Khalifa ticket	Refused: not for an event at your venue
Cancel early	Customer, 3 days before	100% refund, seats free again
Cancel late	Customer, 30 hours before	50% refund, VAT refunded in the same share
Cancel too late	Customer, 10 hours before	Refused: closes 24 hours before
Operator without reason	Vendor cancels with no reason	Refused: a reason is required
Edited card	One digit of the card changed	Payment declined, shown in the window
Admin sign-up	Role Admin at sign-up	Never creates an admin account
Banned user	Login and booking after a ban	Both answered with 403
Self ban	Admin bans their own email	Refused

Table 27: Expected responses

Boundary value analysis:

Defects like to hide at the edges between equivalence classes, so for each edge we test the value on the boundary and the values just below and just above it. Our refund test is a direct example: test_refund_policy_steps checks exactly 48 hours (100%), one minute less (50%), exactly 24 hours (50%) and one minute less again (refused).

Input	Boundary values tested or expected	Expected result
Hours before start (booker cancels)	48:00, 47:59, 24:00, 23:59	100%, 50%, 50%, refused
Guests per customer booking	0, 1, 10, 11	refused, ok, ok, refused
Event capacity	0, 1, 5000, 5001	refused, ok, ok, refused
Rating	-0.5, 0, 5, 5.5	refused, ok, ok, refused
SSN length	15, 16, 17 digits	refused, ok, refused
Support subject / message length	2 and 3 / 9 and 10 characters	refused and ok
Open support tickets per customer	2 open (third allowed), 3 open (fourth refused)	ok, refused
Cancel reason	300 and 301 characters	ok, refused

Table 28: Boundary values

Decision table for the refund policy:

The refund rules combine several conditions, which is exactly the kind of business logic that is best tested with a decision table. A dash means the condition does not matter for that rule.

	R1	R2	R3	R4	R5	R6	R7
Event already started?	Y	N	N	N	N	N	N
Ticket still Valid?	-	N	Y	Y	Y	Y	Y
Vendor or admin?	-	-	Y	N	N	N	Y
Reason given?	-	-	Y	-	-	-	N
Hours left	-	-	-	48 or more	24 to 48	under 24	-

	R1	R2	R3	R4	R5	R6	R7
Action: refuse	X	X				X	X
Action: refund 100%			X	X			
Action: refund 50%					X		

Table 29: Decision table for _cancel()

White-box testing with basis paths:

For white-box testing we used McCabe's cyclomatic complexity, V(G) = number of decisions + 1, as the number of independent paths that need a test. refund_percent_for has two decisions (48 hours or more, 24 hours or more), so V(G) = 3 and there are three basis paths: return 100, return 50 and return nothing. The four checks in test_refund_policy_steps cover all three paths, and also the boundaries between them. We did not measure statement or branch coverage with a tool for the whole project, so we cannot give a coverage percentage.

```
refund_percent_for(start, now)
  1  hours_left = start - now
  2  if hours_left >= 48 -> return 100
  3  if hours_left >= 24 -> return 50
  4  return None
```

```
decision nodes: 2
 V(G) = 2 + 1 = 3
  path A: 1, 2, return 100
  path B: 1, 2, 3, return 50
  path C: 1, 2, 3, 4
```

Validation and error messages:

A large part of our testing was making sure that wrong input gives a clear message instead of a crash. These are the exact messages the user sees for the most common mistakes.

Situation	Message shown to the user
Wrong email or password	Invalid email or password.
Banned account logs in	This account has been banned by an administrator.
Banned or unknown account sends a request	This account is unknown or has been banned.
Short or weak password at sign-up	Please choose a valid, secure password (at least 8 characters).
Email or phone not verified	You must verify both email and phone.
SSN with the wrong length	The SSN must be exactly 16 digits (0-9).
SSN with one digit repeated	Please enter your real SSN.
Staff without a venue	Please choose the venue you work at.
Customer books 11 guests	You can book 1 to 10 guests at a time.
Date in the past	This date is in the past. Please choose another date.
Day the event does not run	This event is not available on the date you chose.
No seats left	Sorry, this event is fully booked on that date (capacity N).

Situation	Message shown to the user
Cancel in the last 24 hours	Cancellations close 24 hours before the event starts.
Cancel after the start	This event has already started, so the booking cannot be cancelled.
Vendor or admin cancels without a reason	Please give the customer a reason for the cancellation.
Ticket from another venue	This ticket is not for an event at your venue. Entry refused.
Ticket used twice	Ticket already used! Checked in previously.
Fourth open support ticket	You already have 3 open tickets. Please wait for staff to reply.
Rating 4.3	The rating must be from 0 to 5, in steps of 0.5.
Review of a future booking	You can only review experiences that you already attended.
Admin bans own account	You cannot ban your own account.
Picture is not a JPEG	The picture must be a JPEG.
Venue still in use	Venue is in use by events or staff and cannot be deleted.

Table 30: Validation and error messages

Traceability from requirements to tests:

To make sure every essential function from section 1.2 is really tested, we traced each one to its component and to the tests that cover it. Where a function has no automated test, we say so.

Function (section 1.2)	Component	Covered by
Sign up and log in for six roles	Accounts and Sign-in	AdminTests (demo admin, no admin sign-up, sign-up time, ban and login)
16-digit SSN for non-customers	Accounts and Sign-in	StaffVenueTests sign-up tests, manual demo script
Events, venues and categories	Events and Venues	test_rename_then_delete_event, test_edit_event_capacity, test_venue_with_staff_cannot_be_deleted
Seat check per date	Booking and Shopping List	CapacityTests (13 tests)
Booking with VAT	Booking and Shopping List	test_booking_price_and_code, test_refusals
Currency display	Pricing, Payment and Receipts	Manual only (demo script)
Demo payment	Pricing, Payment and Receipts	test_demo_payment_verification_success and test_demo_payment_verification_failure_when_edited
Booking Details and PDF	Pricing, Payment and Receipts	Manual only (demo script)
Ticket check-in	Venue Operations	test_scanner_accepts_once_and_reports_guests, test_staff_cannot_check_in_another_venues_ticket
Cancellation and refunds	Cancellations and Refunds	CancellationTests (11 tests)
Partner shopping list	Booking and Shopping List	test_partner_shopping_list_and_checkout, test_failed_cart_checkout_keeps_no_bookings
Vehicle dispatch	Fleet and Dispatch	test_vehicle_combination_sets_the_capacity, test_freeing_a_vehicle_goes_back_to_vendor_capacity
Admin, bans and reports	Administration and Reports	AdminTests, ReportTests, ApiAccountGuardTests

Function (section 1.2)	Component	Covered by
Support tickets	Reviews and Support Tickets	test_support_tickets, test_support_tickets_are_scoped_to_the_staff_venue
Reviews	Reviews and Support Tickets	Manual only (demo script)

Table 31: Traceability from functions to components and tests

6.3 Performance bounds

- **Response time:** a booking, cancellation or check-in is one short SQLite transaction and shows its result without a visible delay on our laptops.
- **Timeouts:** the client waits at most 10 seconds for the server and then tells the user it cannot connect, instead of freezing.
- **Pictures:** at most 1400 pixels after upload and 4 MB on the server, so the event lists stay quick.
- **Limits:** 10 guests per customer booking, 5000 seats per event, 3 open support tickets per customer, 5 open reports per user, 300 characters per cancel reason and 500 per review comment.
- **Test run:** the full suite takes about five minutes, mostly because some tests open windows and the password hashing is slow on purpose.

Software measurement

Measuring size and complexity is a basis for judging quality. We measured our own code with the radon tool for Python, on 31 October 2026. Lines of code (LOC) count everything, and source lines (SLOC) leave out blank lines and docstrings.

Part	LOC	SLOC	Comment lines
Server (server/)	2,536	1,895	150
Client (src/)	7,123	5,441	286
Tests (tests/)	1,009	810	37
Total	10,668	8,146	473

Table 32: Size of the code

LOC measures length, not real size or effort, so we also looked at complexity. Over the 434 functions in the client and the server, the average cyclomatic complexity is 3.8. 361 functions have a complexity of 5 or less, 40 are between 6 and 10, and 33 are above 10. McCabe suggests keeping every module under 10, so those 33 are our weak spots. Most of them are long functions that build a whole screen, such as

show_event_screen (47) and open_event_form (28), and init_db (37), which creates and upgrades every table. Splitting them into smaller functions would be the first refactoring we would do. In the server logic, the most complex functions are validate_maps_url (20), create_report (18), _book (15) and _cancel (14). Note that radon also counts 'and' and 'or' as decisions, so its numbers are a little higher than a hand count of the if statements.

As a small example of Halstead's metrics, refund_percent_for has 4 distinct operators and 9 distinct operands (n1 = 4, n2 = 9), used 5 and 10 times (N1 = 5, N2 = 10). That gives a vocabulary of 13, a length of 15, a volume of about 55.5, a difficulty of about 2.2 and an effort of about 123, which confirms that this key function is short and easy to reason about.

Finally, we made a rough function point count of Rihla as it is built, using the average weight for every item. We counted each server endpoint that changes data as an external input, each read endpoint that only returns stored data as an external inquiry, the read endpoints that calculate something (capacity, demand, today's schedule, ratings) and the PDF receipt as external outputs, and each database table as an internal logical file. Rihla has no external interface files, because it does not read data kept by another system.

Function type	Count	Average weight	Points
External inputs (EI)	33	4	132
External outputs (EO)	5	5	25
External inquiries (EQ)	28	4	112
Internal logical files (ILF)	13	10	130
External interface files (EIF)	0	7	0
Unadjusted function points (UFP)			399

Table 33: Function point count (unadjusted)

We did not apply the 14 adjustment factors, because rating them honestly would need a person outside the team, so 399 is the unadjusted size.

6.4 Identification of critical components

These are the parts we test most carefully, because a mistake there costs someone money, a seat or their trust in the app:

1. **Seat check (_book and _capacity_info).** If it fails, two people can get the same seat, which is the exact problem Rihla was made to solve.
2. **Refund rules (_cancel, refund_percent_for, split_refund).** If they fail, customers get the wrong amount back. The 48 and 24 hour edges must be exact.
3. **Cart checkout.** If it fails, a partner could pay for a group and only get part of the bookings.
4. **Ticket check-in.** If it fails, one ticket could let people in twice, or at the wrong venue.
5. **Access guards (login_status, _blocked, set_user_banned).** If they fail, banned people keep using the app.
6. **SSN handling.** If it fails, a real SSN could end up stored as plain text.
7. **Database upgrade (init_db).** If it fails, the server would not start on an older database.

7.0 Work distribution

Team member	Part of this document
Salem Al Tamimi	Section 1 Introduction and Section 2 Data design
Hamzah and Hamad	Section 3 Architectural and component-level design
Mir Umer	Section 4 User interface design
Omar Arif	Section 5 Restrictions, limitations, and constraints
Oumar Ibrahim	Section 6 Testing issues

Table 34: Who wrote which part

Appendix A: Server endpoint reference

This is the full list of endpoints in server/server.py. The Guard column says what the server checks before it does the work. 'Role' means the _blocked guard with those roles, 'Any active' means _blocked without a role, and 'None' means the endpoint has no check. The 'None' rows that change data are the gaps we describe in section 5.

Method	Path	Guard
POST	/api/auth/login	Password check, banned accounts get 403

Method	Path	Guard
GET	/api/auth/role/<email>	None (read only)
POST	/api/users/register	SSN and staff venue rules
PUT	/api/users/password	None
GET / PUT	/api/users/currency	None / known user and currency
GET / PUT	/api/staff/venue	Staff role, not banned
GET	/api/users/email_exists, phone_exists, email	None (read only)
GET	/api/admin/users	Admin
POST	/api/admin/users/ban	Admin, not self, not another admin
POST	/api/reports	Known, not banned, not admin
GET	/api/admin/reports	Admin
POST	/api/admin/reports/<id>/resolve	Admin
GET	/api/venues, /api/venues/vendor/<email>	None (read only)
POST	/api/venues	Role: Vendor
PUT	/api/venues/<id>/maps_url	Venue must belong to the vendor
DELETE	/api/venues/<id>	None (refused while in use)
GET / POST	/api/vendor_categories	None
DELETE	/api/vendor_categories/<id>	None
GET	/api/events, /vendor/<email>, /venue/<id>	None (read only)
GET	/api/events/today	Staff at a venue
POST	/api/events	Role: Vendor
PUT / DELETE	/api/events/<id>	None
GET	/api/events/<id>/capacity, /demand, /active_bookings	None (read only)
GET	/api/events/<id>/image	None (read only)
GET	/api/events/ratings	None (read only)
POST	/api/bookings	Any active account
GET	/api/bookings/customer/<email>, /vendor/<email>	None (read only)
POST	/api/bookings/<id>/cancel	Any active account, must be the booker
POST	/api/bookings/<id>/vendor_cancel	Role: Vendor, must own the event
GET	/api/admin/bookings	Admin
POST	/api/admin/bookings/<id>/cancel	Admin
POST	/api/tickets/verify	Staff at the ticket's venue
POST	/api/support	Any active account
GET	/api/support/customer/<email>	None (read only)
GET	/api/support/open	Staff at a venue
POST	/api/support/<id>/resolve	Staff at the ticket's venue
GET	/api/vehicle_types, /api/vehicles/<email>	None (read only)
POST / DELETE	/api/vehicles, /api/vehicles/<id>	None
PUT / POST	/api/vehicles/<id>/status, /free	None
GET / POST	/api/dispatch/<event_id>, /api/dispatch	None
GET	/api/dispatch/<event_id>/vehicles	None (read only)
GET	/api/cart/<email>, /count	None (read only)
POST	/api/cart	Role: Partner
POST	/api/cart/<id>/remove	Item must belong to the partner
POST	/api/cart/<email>/clear	None
POST	/api/cart/<email>/checkout	Role: Partner
GET	/api/experiences/customer/<email>	None (read only)
POST	/api/reviews	Any active account, booking attended

Table 35: Every server endpoint and its guard